▼ **Before-or-after atomicity: consistency under concurrency**

1 ✦ Before-or-after atomicity: consistency under concurrency

▼ Where concurrency comes from

2 ✦ Motivating example: from bank application to bank service

3 ✦ Relationship between latency and throughput: queued system model

4 ✦ Question: what if arrival rate (λ) > throughput (Thpt)?

5 ✦ Why maximum throughput matters for latency

6 ✦ Question: if Thpt > λ, is the queue always empty?

7 ✦ Where the burst comes from: the Poisson arrival pattern

8 ✦ Analysis: how long is the wait? (Informally)

9 ✦ Analysis: W ≈ (1 / Thpt) / (1 − U)

10 ✦ Question: how can we improve system throughput?

11 ✦ Using concurrent execution to scale the throughput

▼ The cost of concurrency: race condition

12 ✦ Motivating example: issue during concurrent operations

13 ✦ The issue is that the `+` operator is not atomic

14 ✦ Analysis: what happens when operations from multiple threads interleave

15 ✦ Race condition

16 ✦ Race condition is hard to control

17 ✦ Case study in real life: a benign race

18 ✦ Case study (a bad one): Therac-25

19 ✦ Therac-25: Concurrent reads and writes to the mode

20 ✦ Therac-25: a simplified trace of the missed edit

21 ✦ More case study: AWS us-east-1's outage (2025-10-20)

22 ✦ AWS hosts DNS Planner and DNS Enactor to update the record

23 ✦ Analysis: the interleaving that happened caused AWS's outage

24 ✦ Summary: race conditions are harmful in practice if not handled

▼ The guarantee we want

25 ✦ Rethink: What is a strong consistency model

26 ✦ Review: serial execution of reads/writes

27 ✦ One step further: serial execution of actions

28 ✦ Goal: before-or-after atomicity (a.k.a. isolation)

▼ Use locking to achieve before-or-after

29 ✦ Realizing before-or-after: Locks

30 ✦ Use locking to achieve before-or-after: global lock

31 ✦ Analysis: the same two deposits under a global lock

32 ✦ Global lock: trading performance for correctness

33 ✦ Question: how to optimize?

34 ✦ Use locking to achieve before-or-after: fine-grained locking

35 ✦ Fine-grained locking allows more concurrency

36 ✦ A more complex example: Transfer + Audit

37 ✦ Question: think about the following execution?

38 ✦ Problem: the intermediate state is exposed with improper lock scope

39 ✦ Use locking to achieve before-or-after: static locking

40 ✦ An example execution of static locking

41 ✦ The drawback of static locking: the lock is held far longer than it is used

42 ✦ Use locking to achieve before-or-after: two-phase locking (2PL)

43 ✦ Static locking vs 2PL

44 ✦ Two phases of 2PL: lock acquisition & release

▼ Serializability

45 ✦ Why 2PL is correct? It gives us serializability

46 ✦ The "appear" typically means the schedule is similar

47 ✦ Serial schedule: run the actions sequentially

48 ✦ Question: what if run concurrently (concurrent schedules) appears to be the same as a serial one?

49 ✦ Think: the result is right, but the reads are impossible

50 ✦ View serializability

51 ✦ Question: is view serializability possible to check?

52 ✦ Conflict serializability

53 ✦ The conflicts in our example

54 ✦ Conflict graph

55 ✦ The conflict graph of schedule C

56 ✦ Conflict equivalence

57 ✦ A schedule that is view but not conflict serializable

58 ✦ View serializability vs. conflict serializability

59 ✦ Why systems use conflict serializability

60 ✦ Proof of 2PL (by contradiction)

61 ✦ Correctness is application-level semantics

62 ✦ Correctness and serializability

▼ The deadlock issue of 2PL

63 ✦ The issue of 2PL: it is pessimistic

64 ✦ That Transfer + Audit under 2PL

65 ✦ Deadlock: what if Audit first acquires `lock[a]`?

66 ✦ Resolving deadlock

67 ✦ Summary

68 ✦ 参考文献

Before-or-after atomicity: consistency under concurrency

Xingda Wei

Motivating example: from bank application to bank service

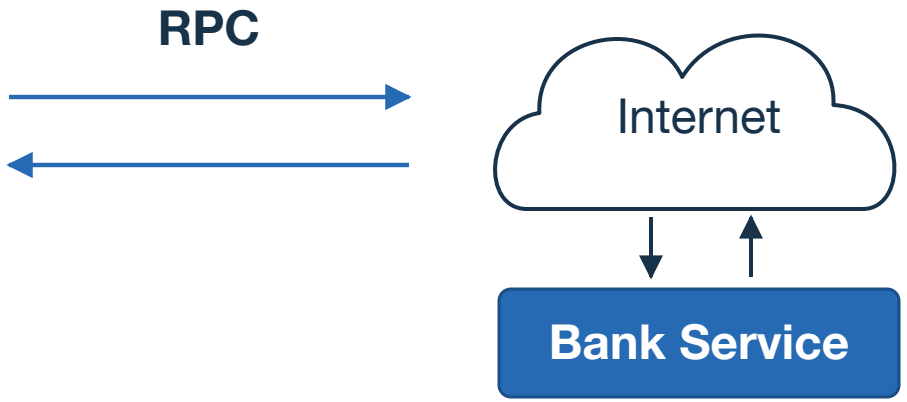
Suppose you are operating a bank service, deployed with a single-threaded setup

- Users send their requests from bank applications over RPC; the server handles one at a time



Users





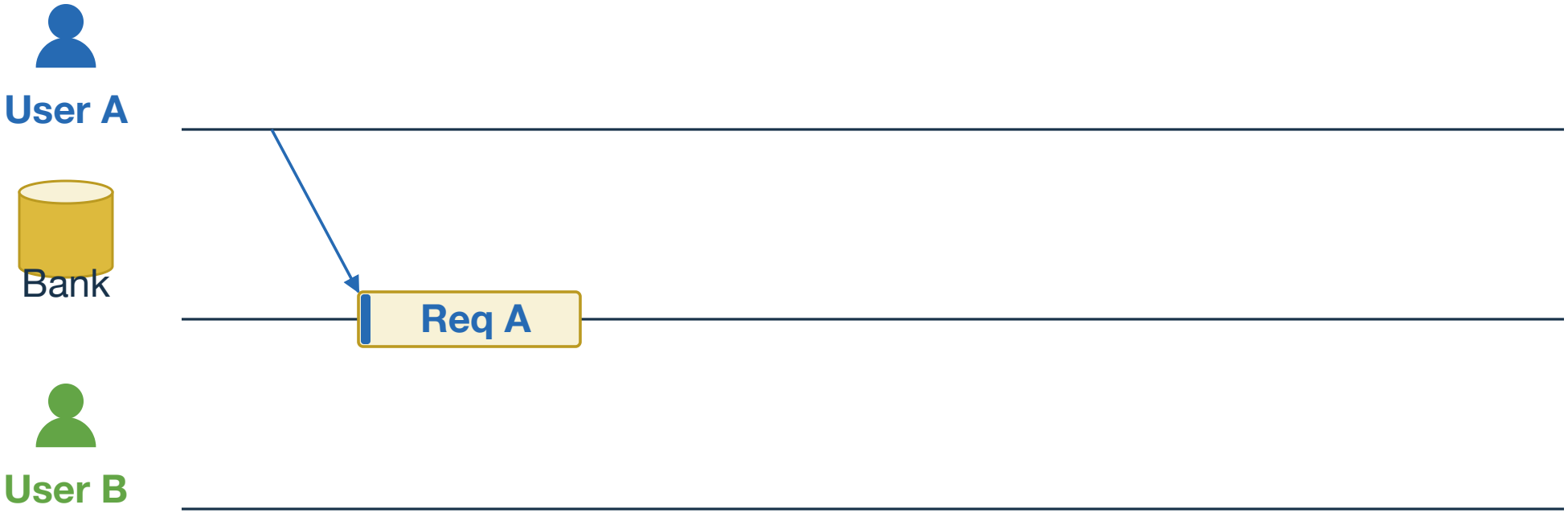
Question: what is the maximum throughput of it?

- And more importantly: why does it matter? Application only wants latency

Relationship between latency and throughput: queued system model

Think of the service has a queue to hold pending requests

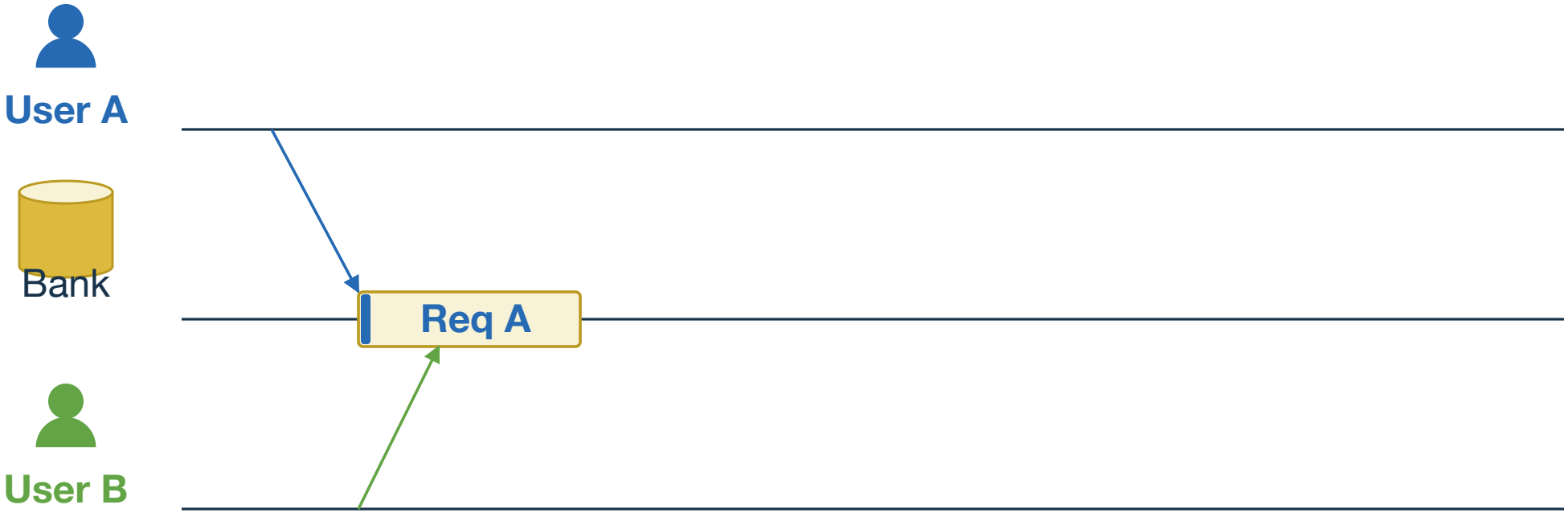
- Requests arrive at any time, the server handles one at a time, the rest wait
- So a user's latency = queued latency + process latency



Relationship between latency and throughput: queued system model

Think of the service has a queue to hold pending requests

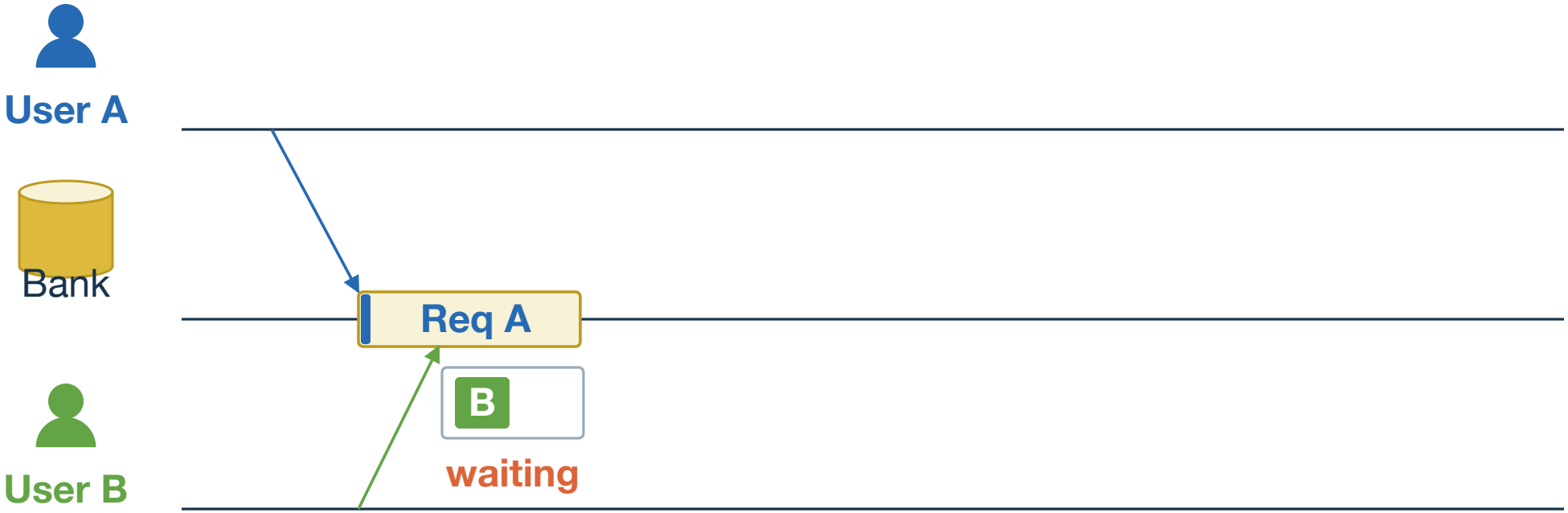
- Requests arrive at any time, the server handles one at a time, the rest wait
- So a user's latency = queued latency + process latency



Relationship between latency and throughput: queued system model

Think of the service has a queue to hold pending requests

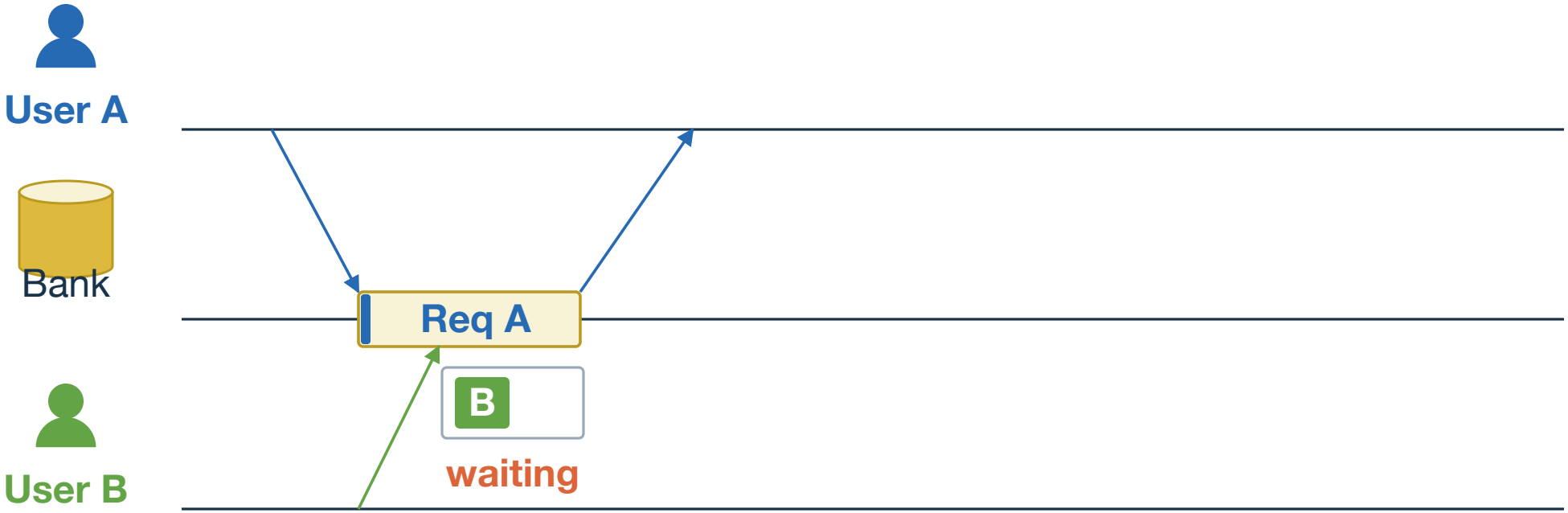
- Requests arrive at any time, the server handles one at a time, the rest wait
- So a user's latency = queued latency + process latency



Relationship between latency and throughput: queued system model

Think of the service has a queue to hold pending requests

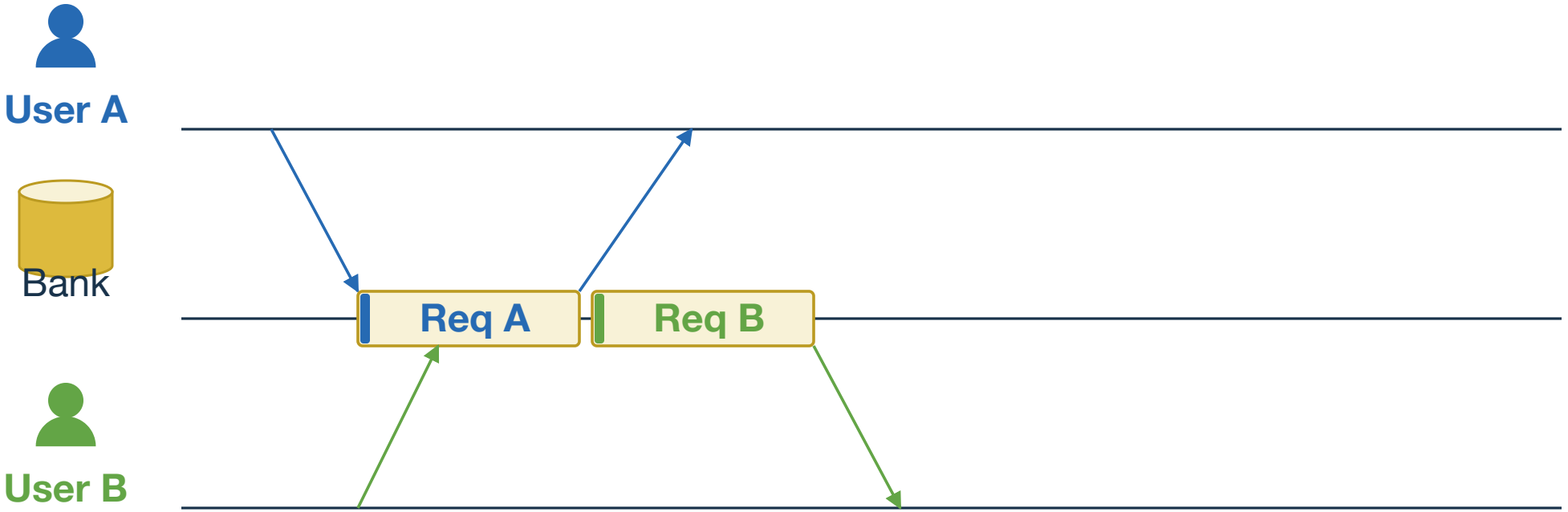
- Requests arrive at any time, the server handles one at a time, the rest wait
- So a user's latency = queued latency + process latency



Relationship between latency and throughput: queued system model

Think of the service has a queue to hold pending requests

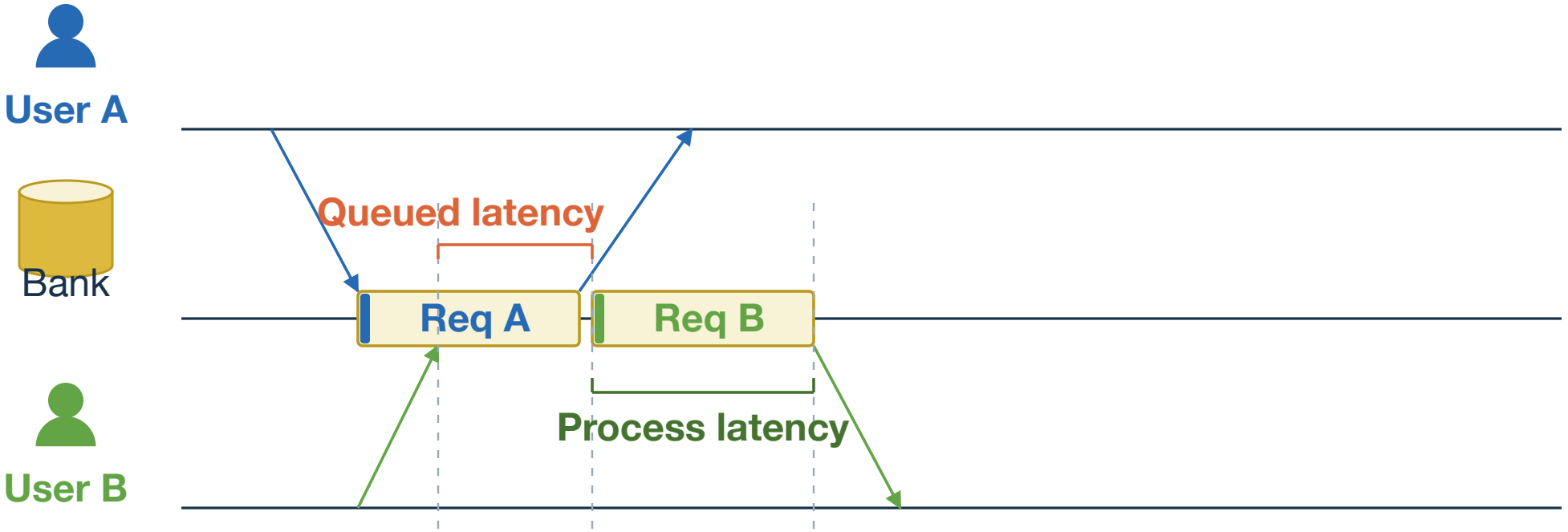
- Requests arrive at any time, the server handles one at a time, the rest wait
- So a user's latency = queued latency + process latency



Relationship between latency and throughput: queued system model

Think of the service has a queue to hold pending requests

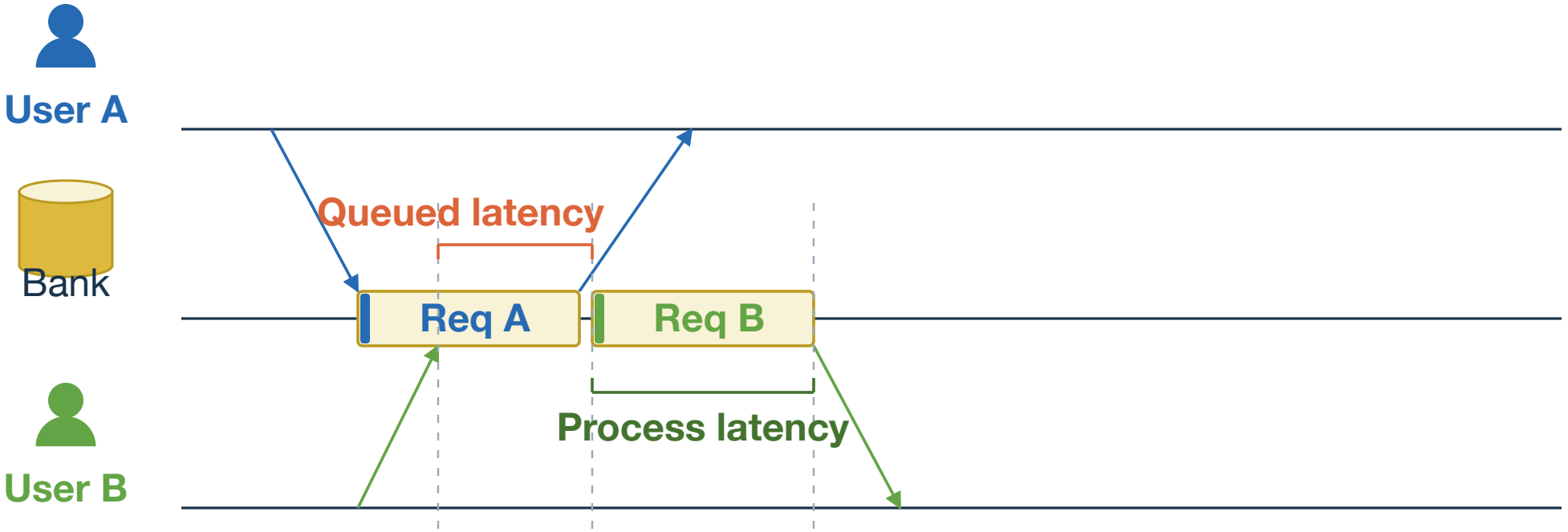
- Requests arrive at any time, the server handles one at a time, the rest wait
- So a user's latency = queued latency + process latency



Question: what if arrival rate (λ) > throughput (Thpt)?

The requests are kept in the queue

- #Queued requests ≈ (λ − Thpt) · T (T is the service time, assuming λ and T is static)
- As long as λ > Thpt, T keeps growing and queued latency has no upper bound



Why maximum throughput matters for latency

Suppose a workload's arrival rate λ is given to us; Thpt generally determines the observable latency

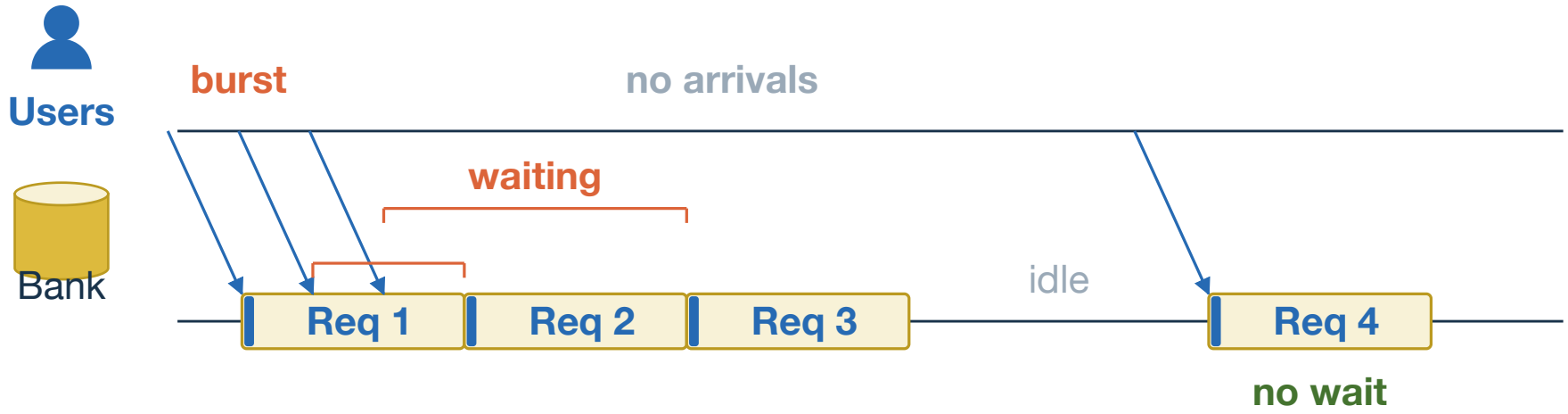
- Thpt > λ: the queue stays bounded (we have seen before)
- Thpt < λ: the queued latency can still have no upper bound!

Avoid queued latency under a given workload

Question: if Thpt > λ, is the queue always empty?

No — because requests arrive in bursts

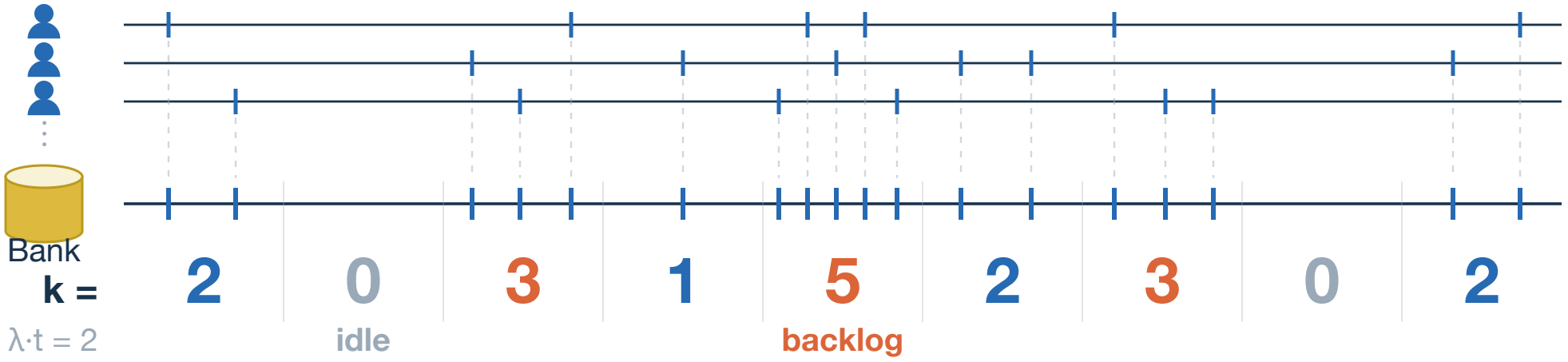
- A burst piles up requests faster than the server drains them, so queueing happens



Where the burst comes from: the Poisson arrival pattern

Many independent users, each sending rarely: nobody coordinates with anybody

- So requests land at random times, and randomly placed points aggregate



The request count within a time window t follows Poisson(λ·t)

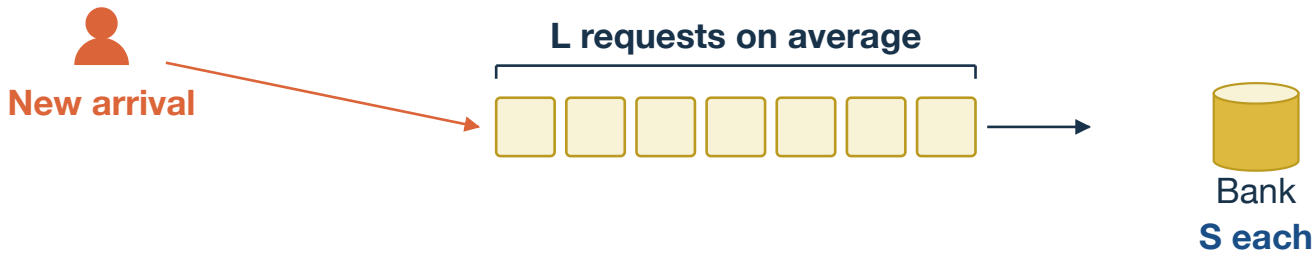
$$P(k \text{ arrivals in } t) = \frac{(\lambda t)^k e^{-\lambda t}}{k!}$$

Analysis: how long is the wait? (Informally)

Utilization $U = \lambda / Thpt$: the fraction of time the server is busy (U = 1 means saturated)

Two unknowns: the average latency W, and the average number of requests in the system L

- A new arrival waits for the L ahead, then its own turn: $W \approx (L + 1) / Thpt$
- Assumes Poisson arrivals: an arrival does not look at the queue, so it lands at a random moment and sees L



A second relation: the L ahead arrived earlier; in steady state they also stay W on average

- Little's Law: $L \approx \lambda \cdot W$

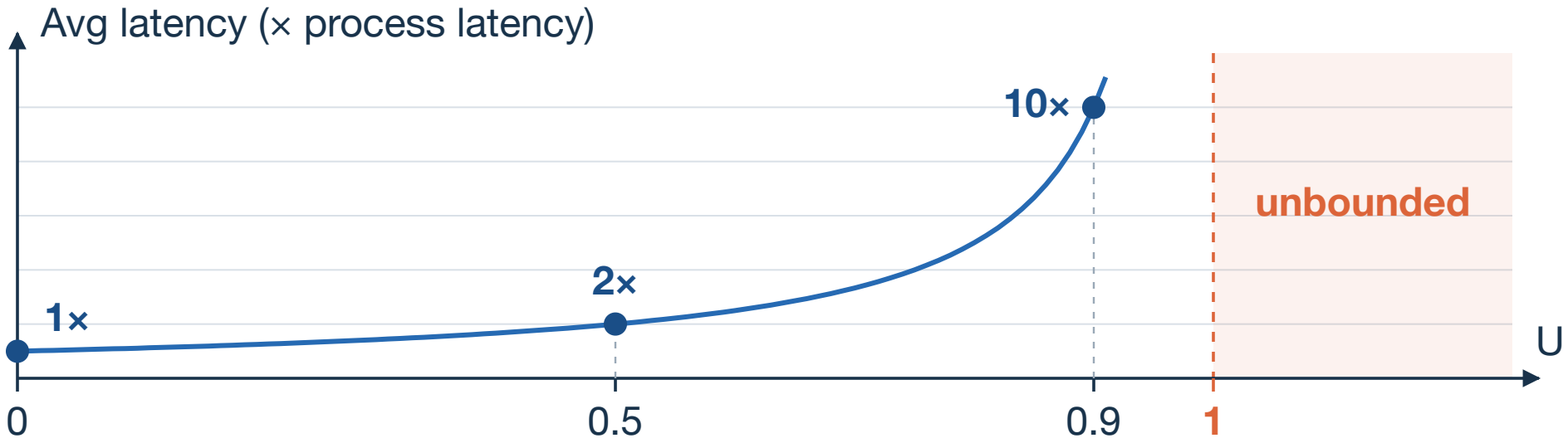
Eliminate L: $W \approx (\lambda \cdot W + 1) / Thpt = U \cdot W + 1 / Thpt$

- Solve for W: $W \approx (1 / Thpt) / (1 - U)$, the processing time scaled by $1 / (1 - U)$

Analysis: W ≈ (1 / Thpt) / (1 – U)

The latency curve:

- U = 0.5: every request takes 2× its processing time
- U = 0.9: 10×; as U approaches 1, the wait has no bound



This is why service providers deliberately leave their servers partly idle (e.g., U = 80%)

How to improve the throughput?

Question: how can we improve system throughput?

Before 2004: just wait 6 months and buy a new computer

- Single-core frequency kept rising, and not a line of code had to change

After 2004: single-core frequency stopped rising

- For more throughput, you have to write a parallel (concurrent) program yourself to boost the throughput

Using concurrent execution to scale the throughput

Concurrency/Parallelism: apps run on multiple execution resources (e.g., CPUs) at the same time

- Scale-up: run computations using multiple threads on one machine
- Scale-out: run computations using multiple machines



Scale up



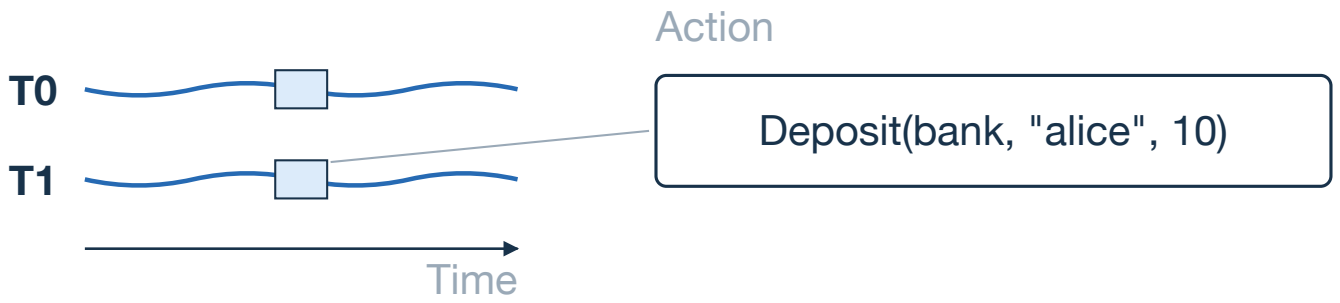
Scale out

However, taming concurrency is challenging!

Motivating example: issue during concurrent operations

Case study: Two threads deposit into the same account concurrently

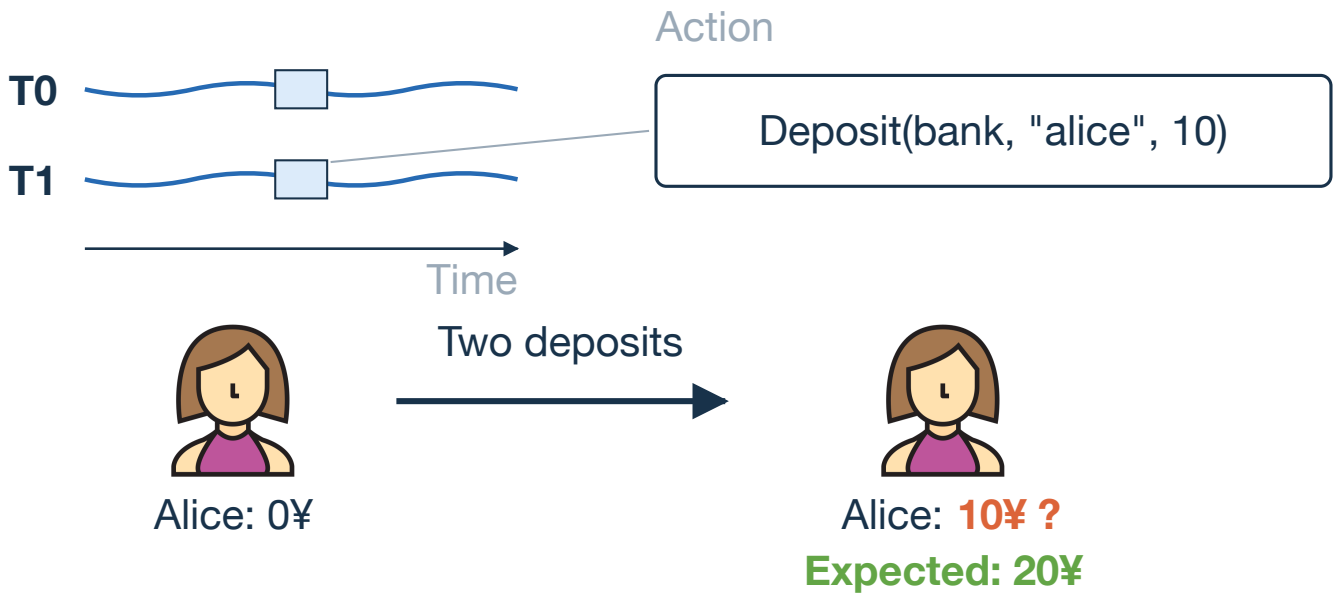
- To simplify understanding, suppose the accounts live in an in-memory array `bank`; each thread runs one `Deposit(bank, "alice", 10)`
- And there is no failure



Motivating example: issue during concurrent operations

Case study: Two threads deposit into the same account concurrently

- To simplify understanding, suppose the accounts live in an in-memory array `bank`; each thread runs one `Deposit(bank, "alice", 10)`
- And there is no failure



Question: what outcome does Alice expect?

The issue is that the `+` operator is not atomic

The developer writes one addition; the actual work is three instructions

```
Deposit(bank, acct, amt):
    bank[acct] += amt
```

```
mov    acct, %eax
add    $amt, %eax
mov    %eax, acct
```

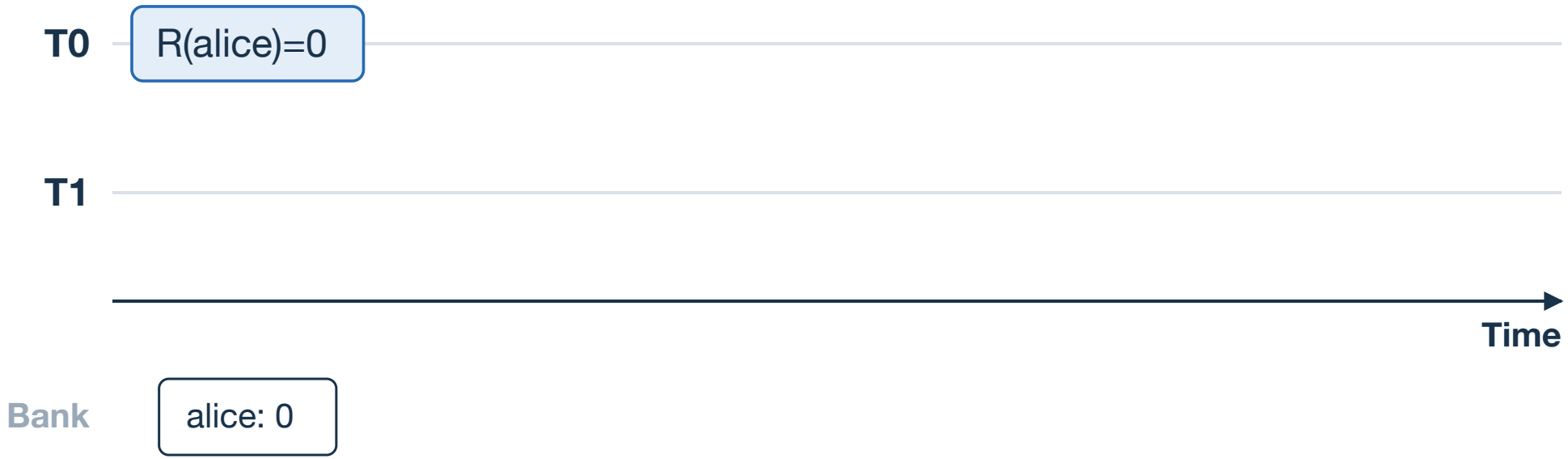
The CPU only guarantees the atomicity of one memory op, but this has two

- One read (`mov acct, %eax`), one write (`mov %eax, acct`)

Under concurrency, different operations can interleave, leading to issues

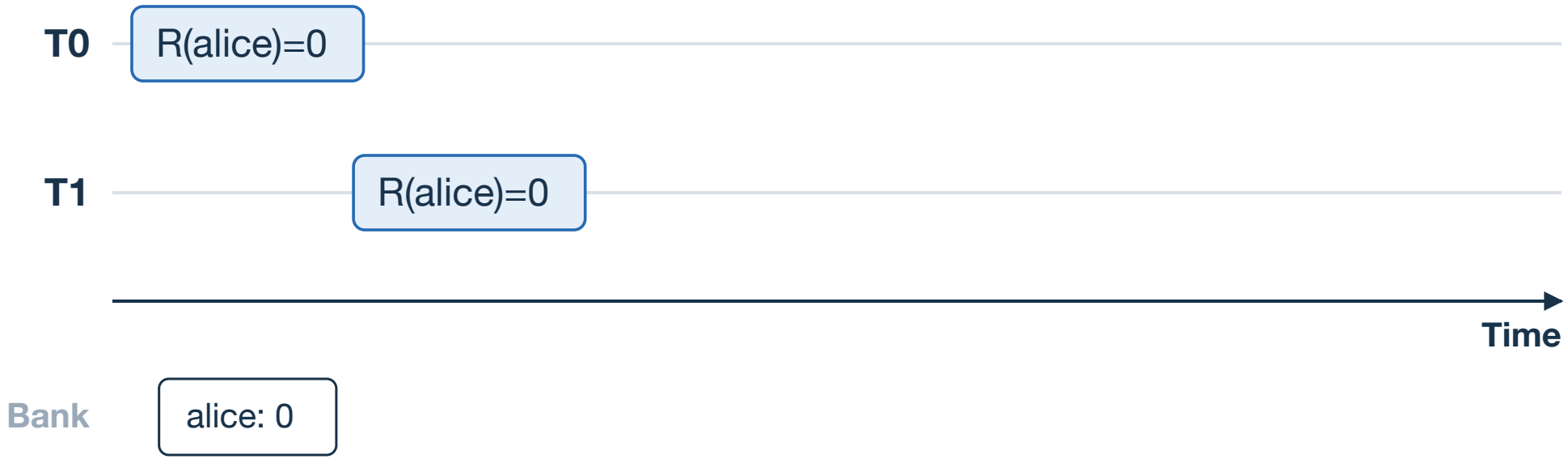
Analysis: what happens when operations from multiple threads interleave

The reads, increments and writes of the two threads interleave as below



Analysis: what happens when operations from multiple threads interleave

The reads, increments and writes of the two threads interleave as below



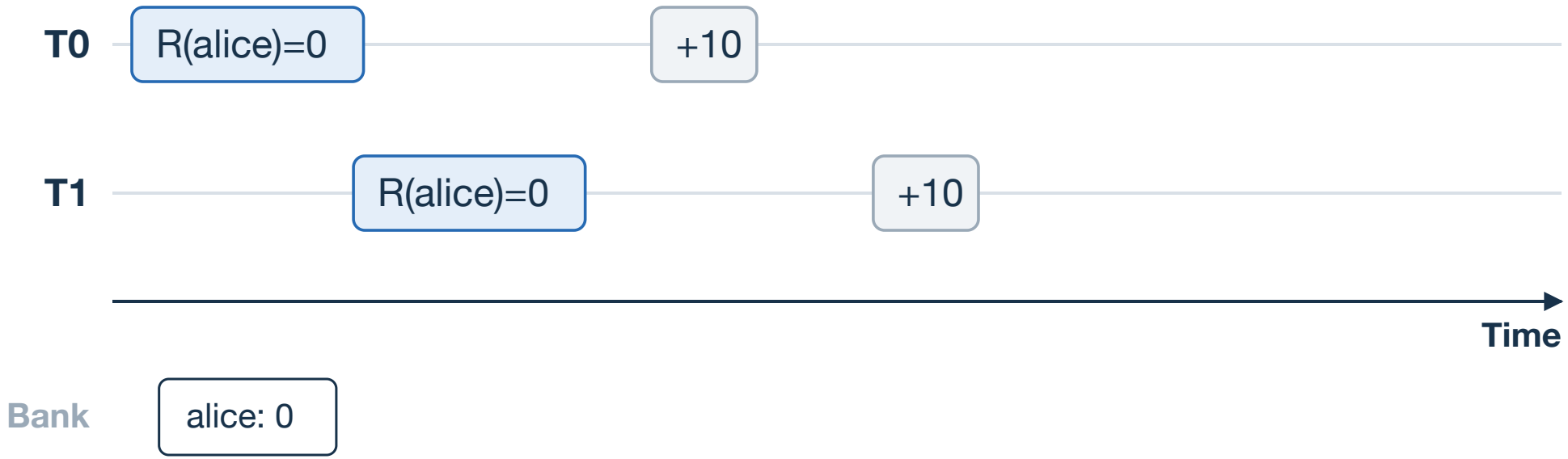
Analysis: what happens when operations from multiple threads interleave

The reads, increments and writes of the two threads interleave as below



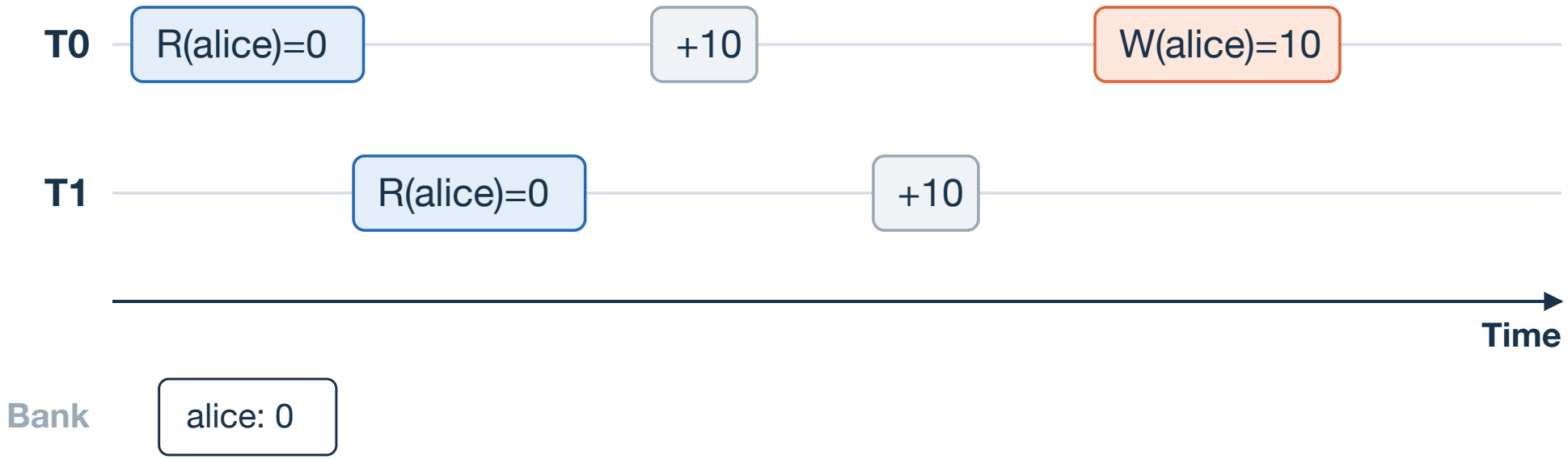
Analysis: what happens when operations from multiple threads interleave

The reads, increments and writes of the two threads interleave as below



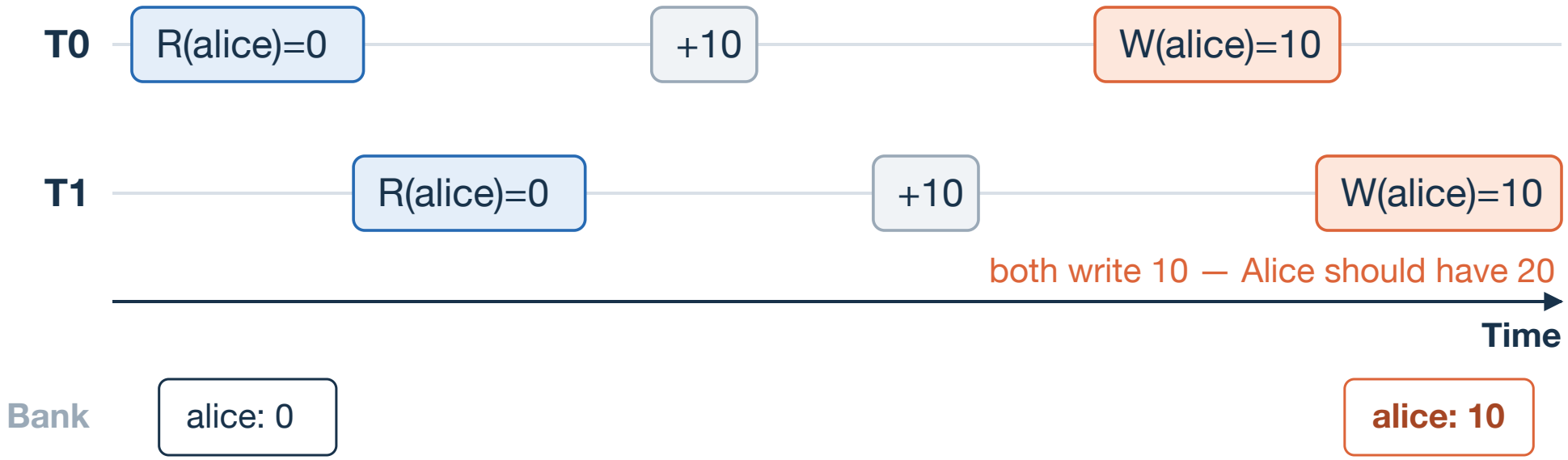
Analysis: what happens when operations from multiple threads interleave

The reads, increments and writes of the two threads interleave as below



Analysis: what happens when operations from multiple threads interleave

The reads, increments and writes of the two threads interleave as below



Race condition

Same code, two different schedules, two different results

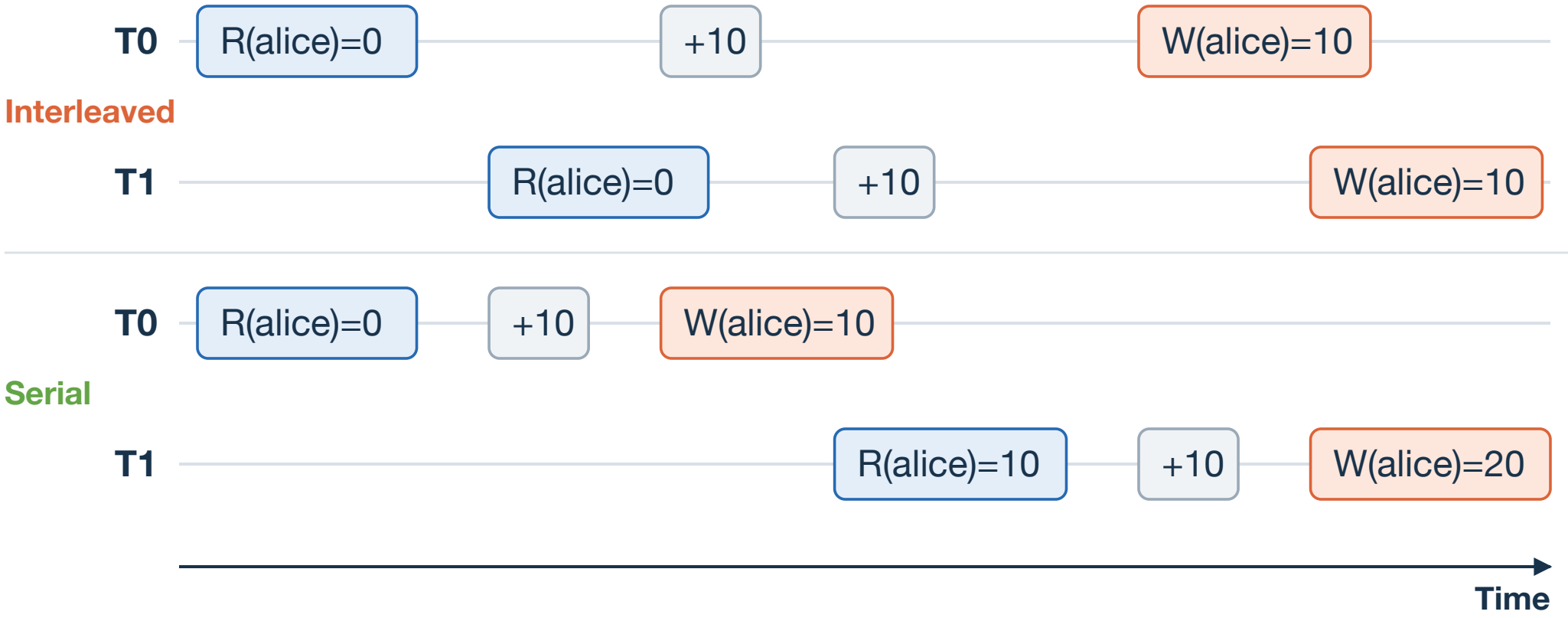
- Race condition: two or more threads access the same shared data, at least one is a write



Race condition

Same code, two different schedules, two different results

- Race condition: two or more threads access the same shared data, at least one is a write



Race condition is hard to control

To be correct, every possible schedule must be safe

- The number of schedule permutations is huge
- Bad schedules are a fraction of them, but can happen in practice

The errors are intermittent

- A small timing change between two invocations changes the behavior
- Add a `print` or attach a debugger, the timing shifts and the bug hides: Heisenbug

Systems people have tried to take the non-determinism away

- DMT (Deterministic Multi-Threading): the same input always takes the same schedule
- Record and replay: record the schedule, replay it afterwards
- Both are hard in either completeness or cost, or both

Case study in real life: a benign race

Real-world systems embrace concurrency, so races are likely to happen everywhere

- Some of them do not break correctness, they just look wrong to the user, so we call them benign



[1]

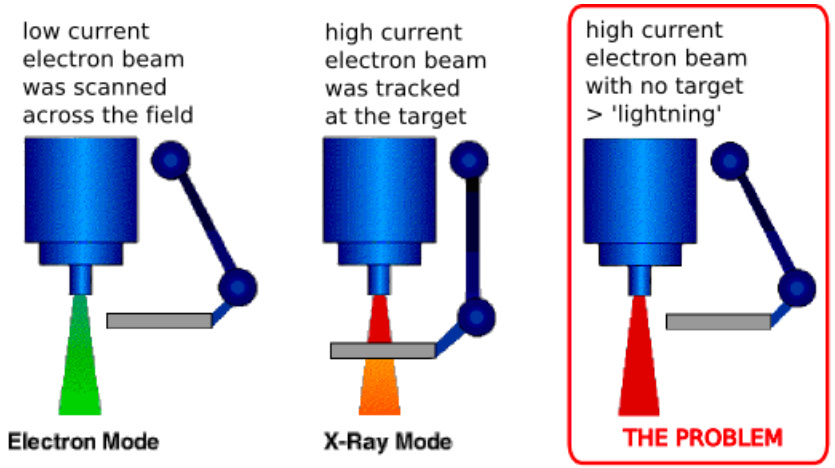
[1] Screenshot from the Zhihu app

Case study (a bad one): Therac-25

Therac-25 is a radiation therapy machine with two modes; each mode sets two things

- X-ray mode: high power (100×), with a metal plate in the beam
- Electron mode: low power, plate out





tray including the target, a flattening filter, the collimator jaws and an ion chamber was moved OUT for "electron" mode, and IN for "photon" mode.

[1]

Dangerous: high power with the plate out

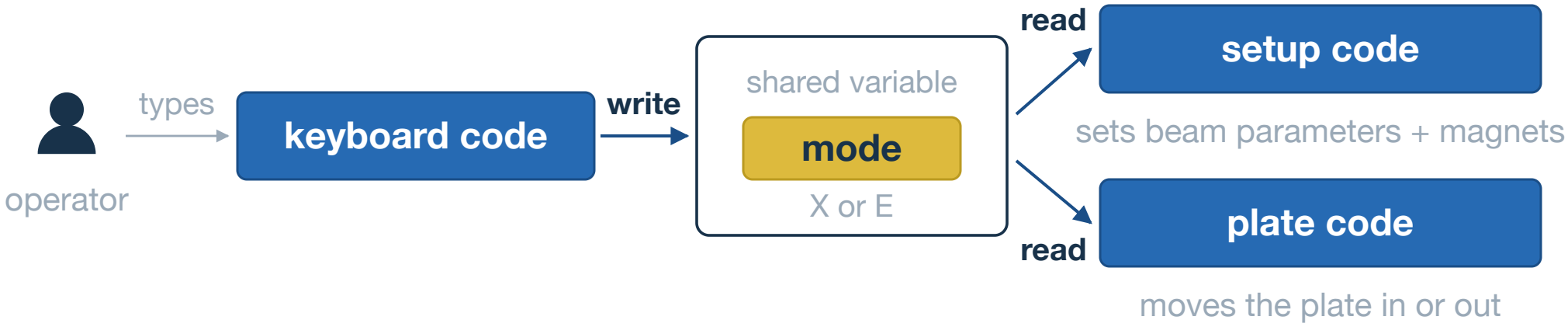
- Question: the mode is a variable in software; who writes it, and who reads it?

[1] Leveson, *Medical Devices: The Therac-25* (appendix of *Safeware*, 1995), sunnyday.mit.edu/papers/therac.pdf

Therac-25: Concurrent reads and writes to the mode

The operator types the mode; the keyboard code writes it into a variable `mode`

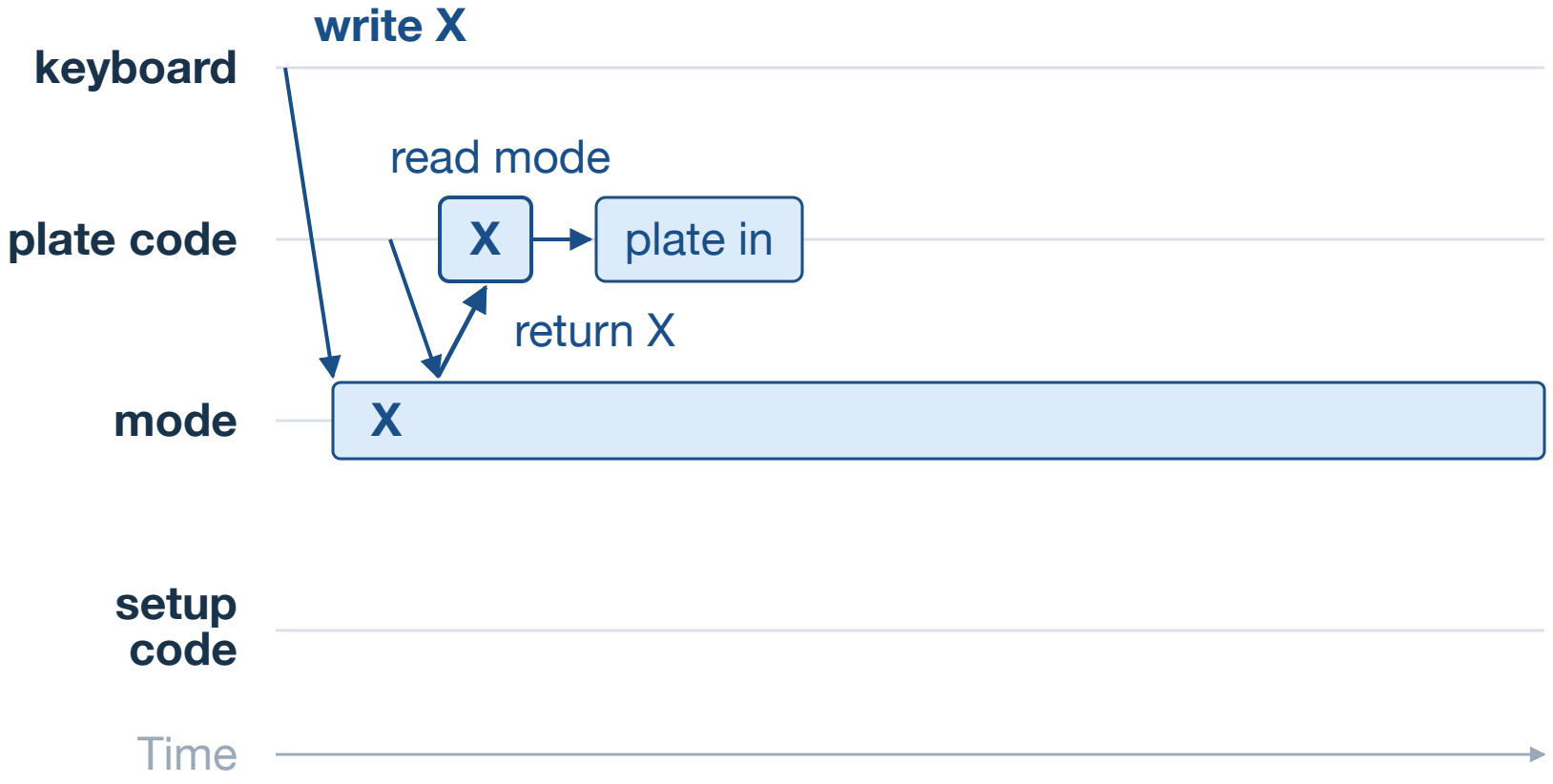
- The setup code reads `mode`, sets beam parameters, then sets magnets (~8 s)
- The plate code reads `mode` and moves the plate in or out



Simplified model: what if an edit reaches the plate code but the setup code misses it?

Therac-25: a simplified trace of the missed edit

First configure X; then edit to E while magnet setup is still running



Device state

power: pending



beam off



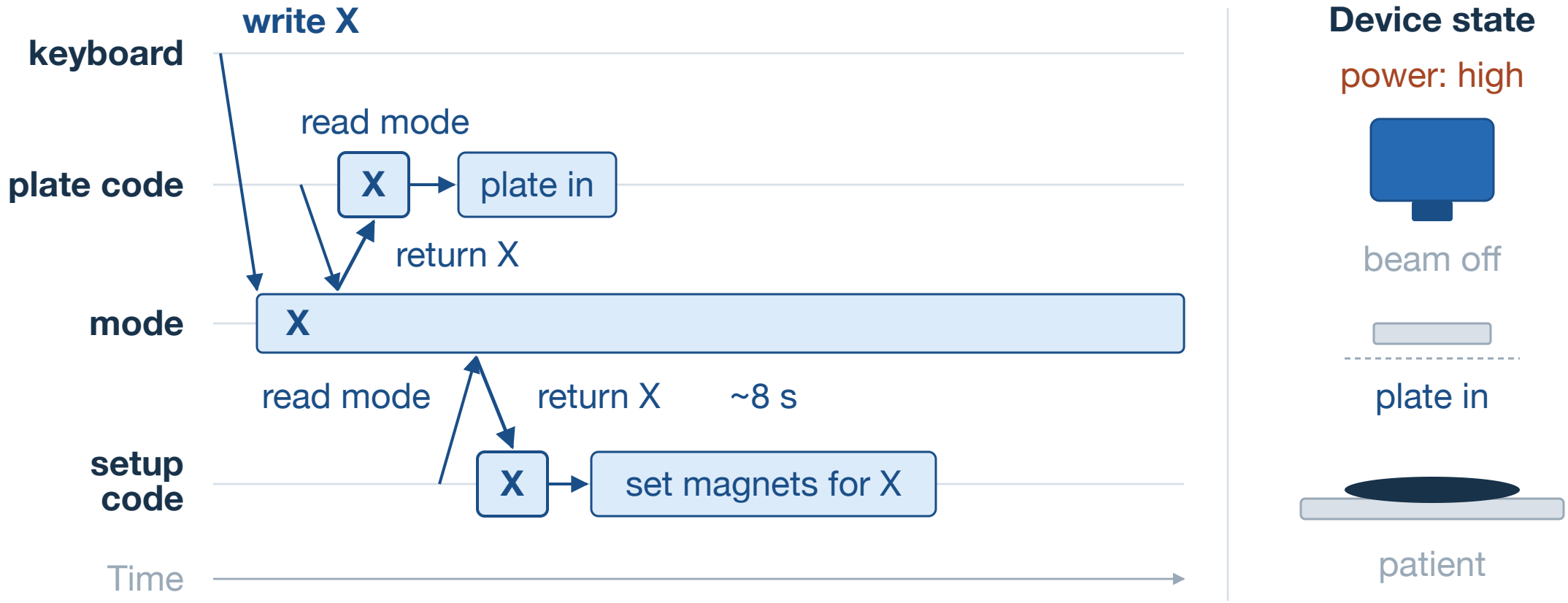
plate in



patient

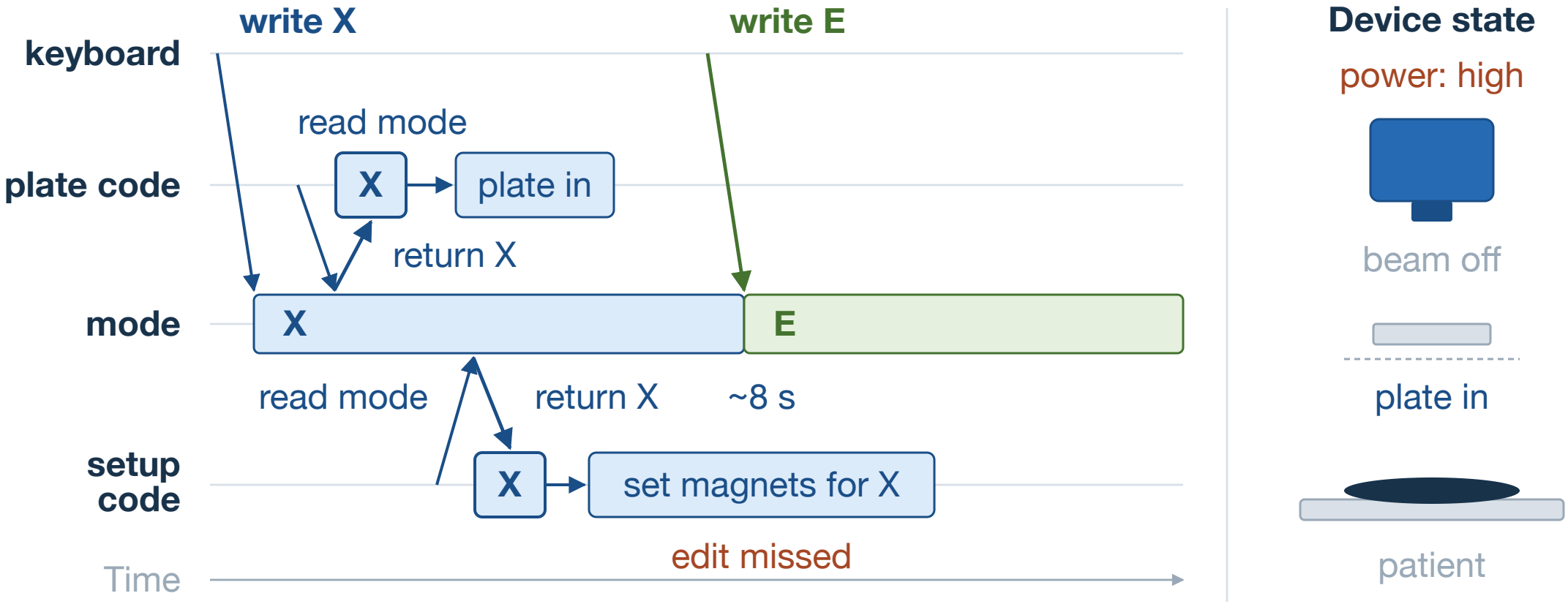
Therac-25: a simplified trace of the missed edit

First configure X; then edit to E while magnet setup is still running



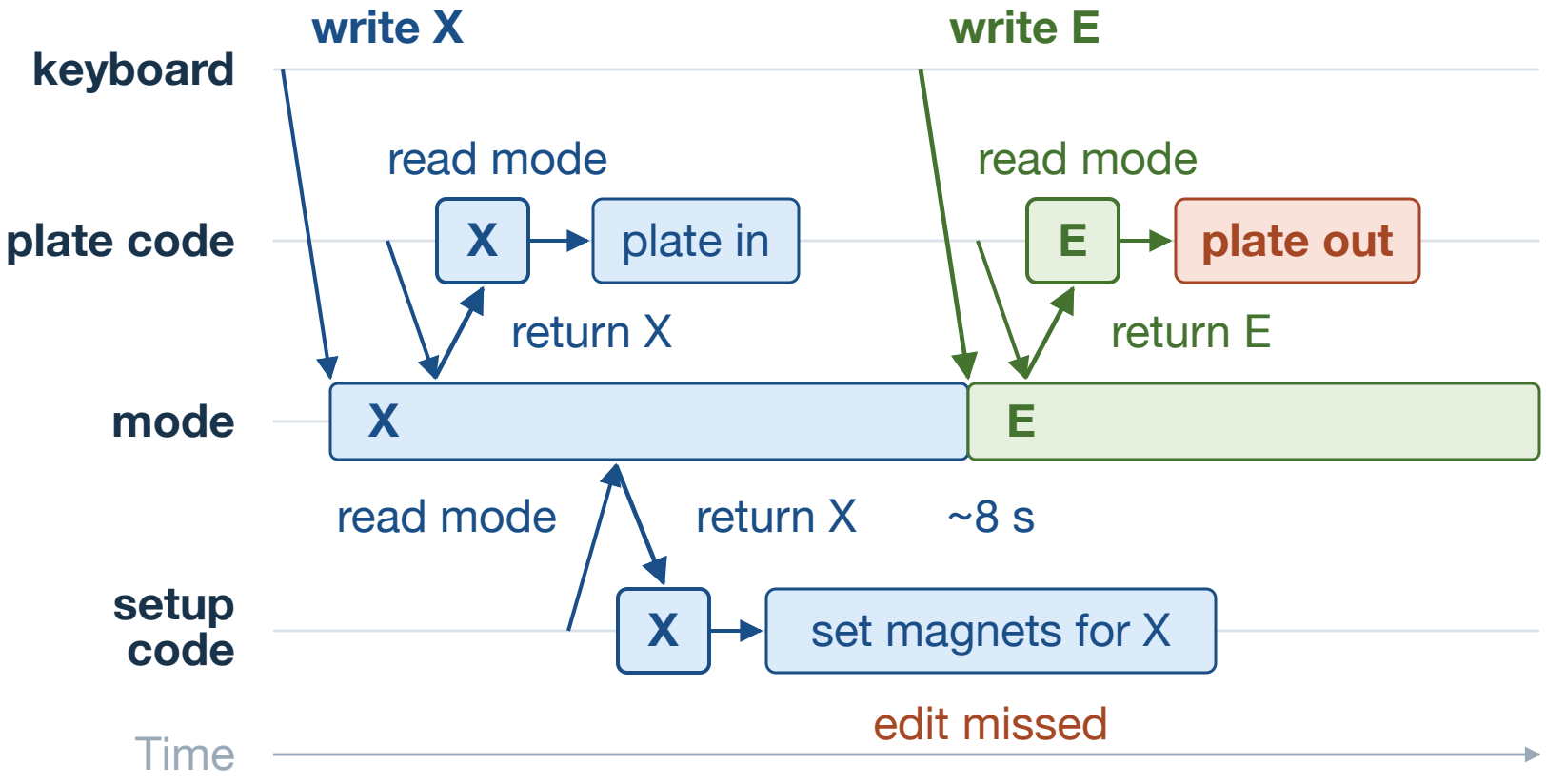
Therac-25: a simplified trace of the missed edit

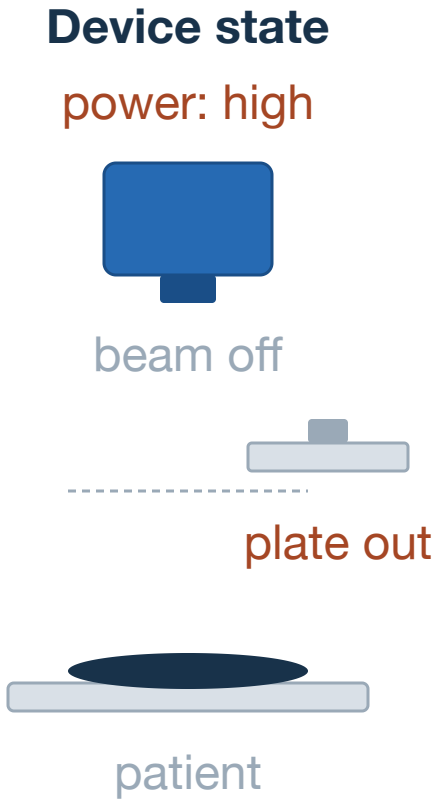
First configure X; then edit to E while magnet setup is still running



Therac-25: a simplified trace of the missed edit

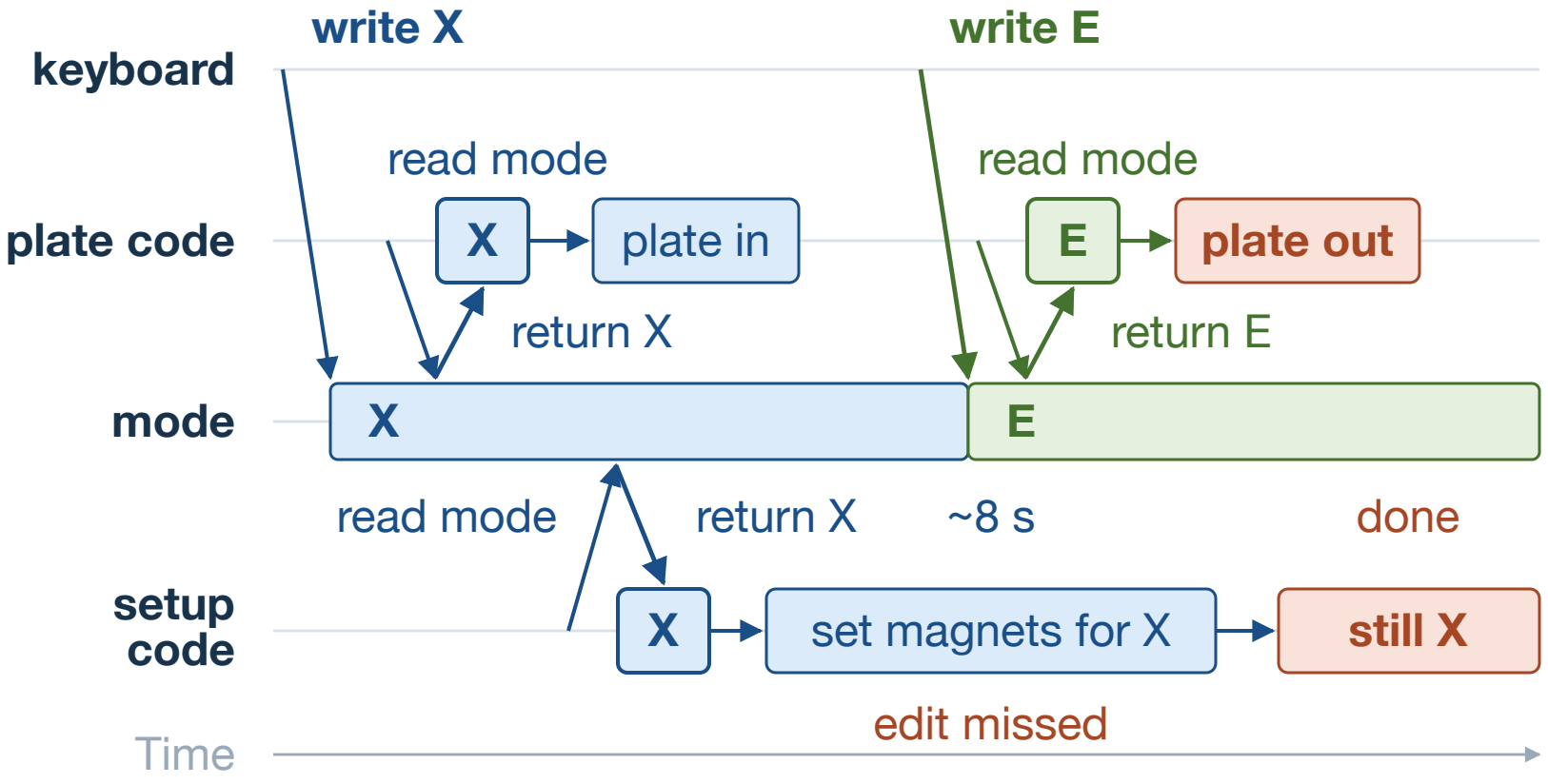
First configure X; then edit to E while magnet setup is still running

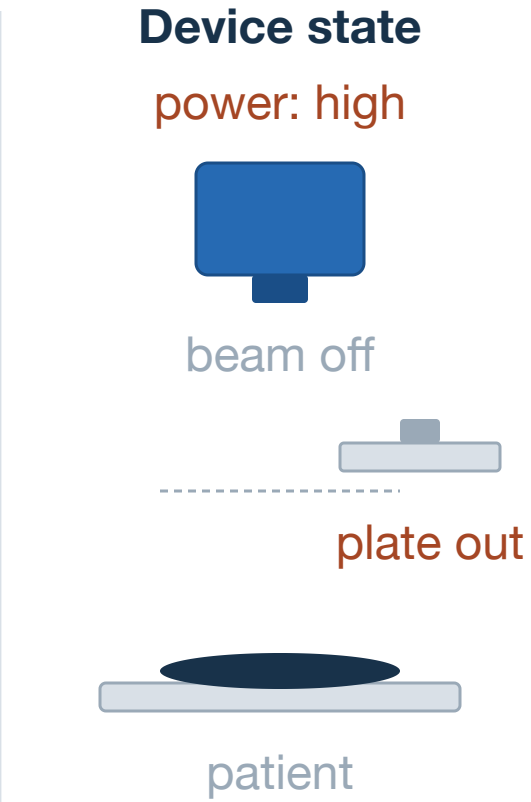




Therac-25: a simplified trace of the missed edit

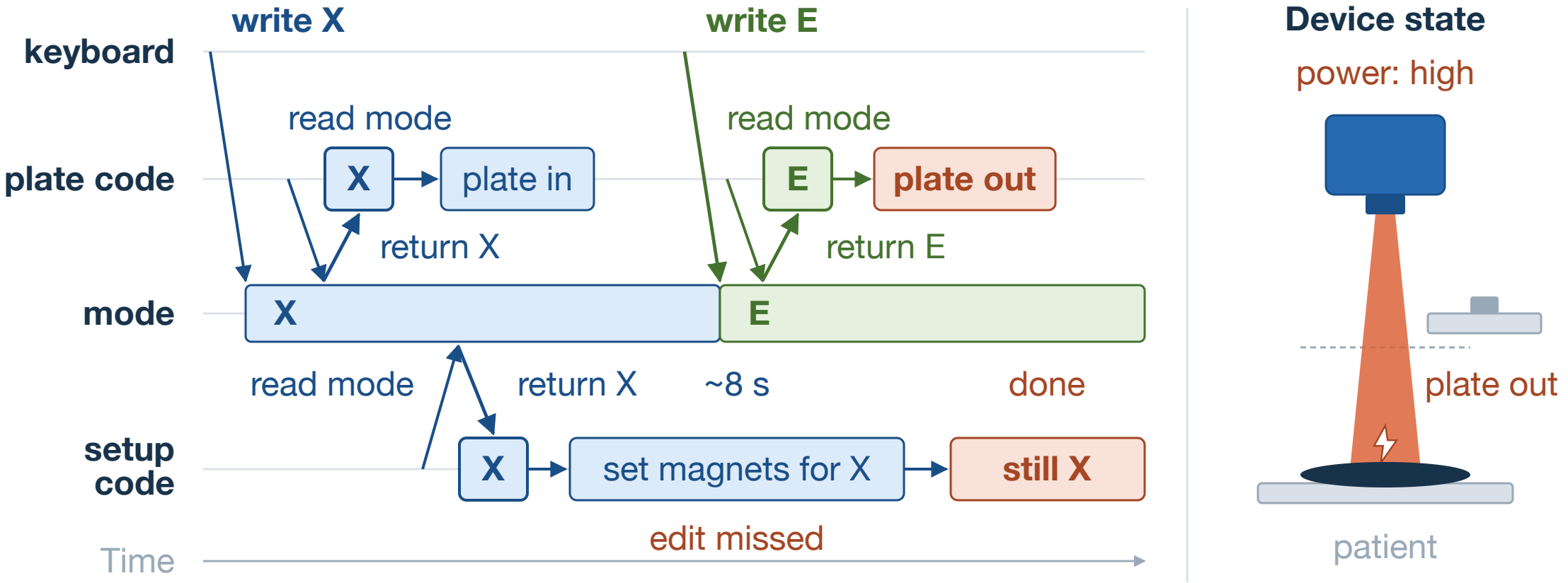
First configure X; then edit to E while magnet setup is still running





Therac-25: a simplified trace of the missed edit

First configure X; then edit to E while magnet setup is still running

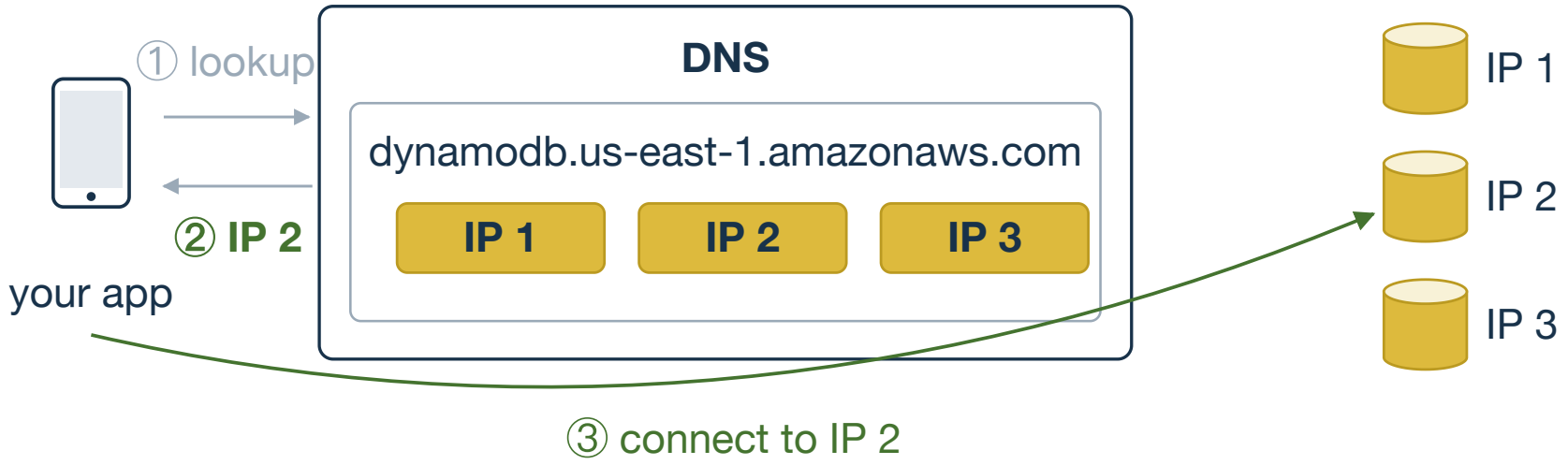


The beam turns on with X settings and the plate out: setup missed the E edit.

More case study: AWS us-east-1's outage (2025-10-20)

AWS's case: to reach a service, a program needs a DNS to find the IP of the service

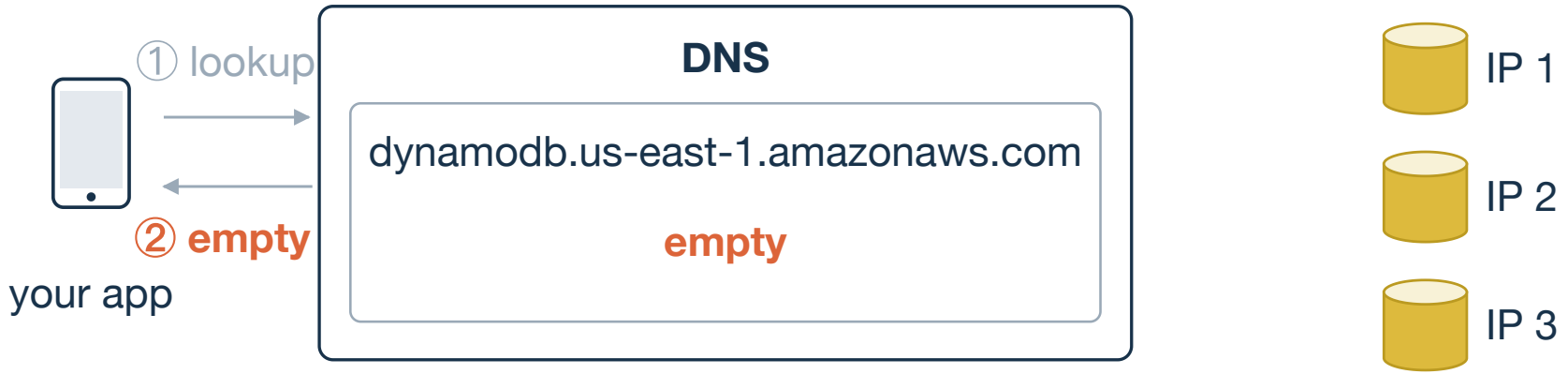
- The name → IP-list mapping is a DNS record; every connection starts with a lookup
- DynamoDB is a database service that many AWS-hosted apps build on



More case study: AWS us-east-1's outage (2025-10-20)

AWS's case: to reach a service, a program needs a DNS to find the IP of the service

- The name → IP-list mapping is a DNS record; every connection starts with a lookup
- DynamoDB is a database service that many AWS-hosted apps build on



nothing to connect to

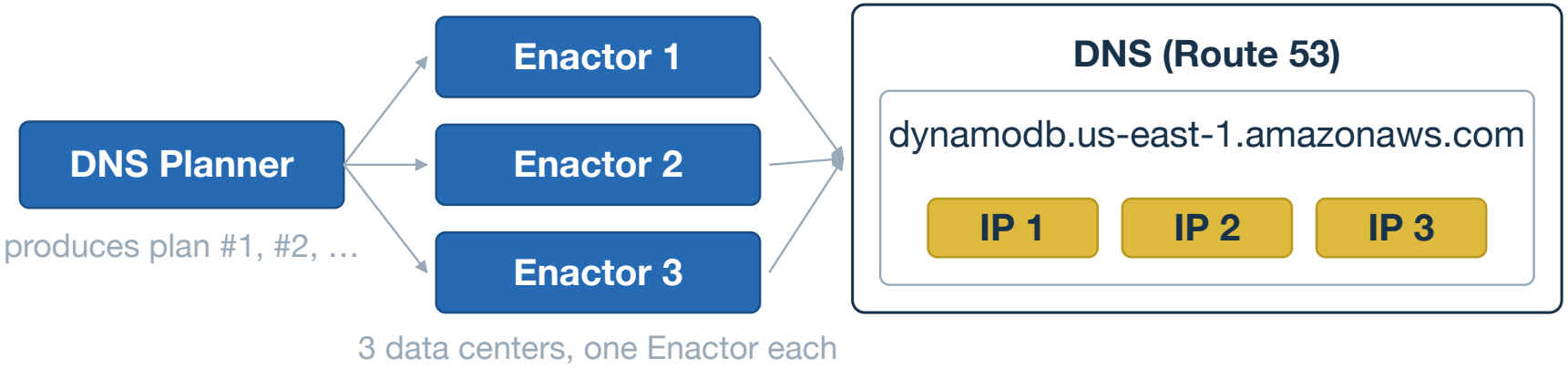
On 2025-10-20 DynamoDB's record was left empty for ~2.5 hours

- Every lookup returned nothing, so nothing could connect: the apps on top failed together

AWS hosts DNS Planner and DNS Enactor to update the record

Servers can fail/reconfigure, so the IP list changes all the time

- DNS Planner keeps producing new versions of the IP list: plan #1, #2, …
- DNS Enactor picks up the newest plan and writes it into the record



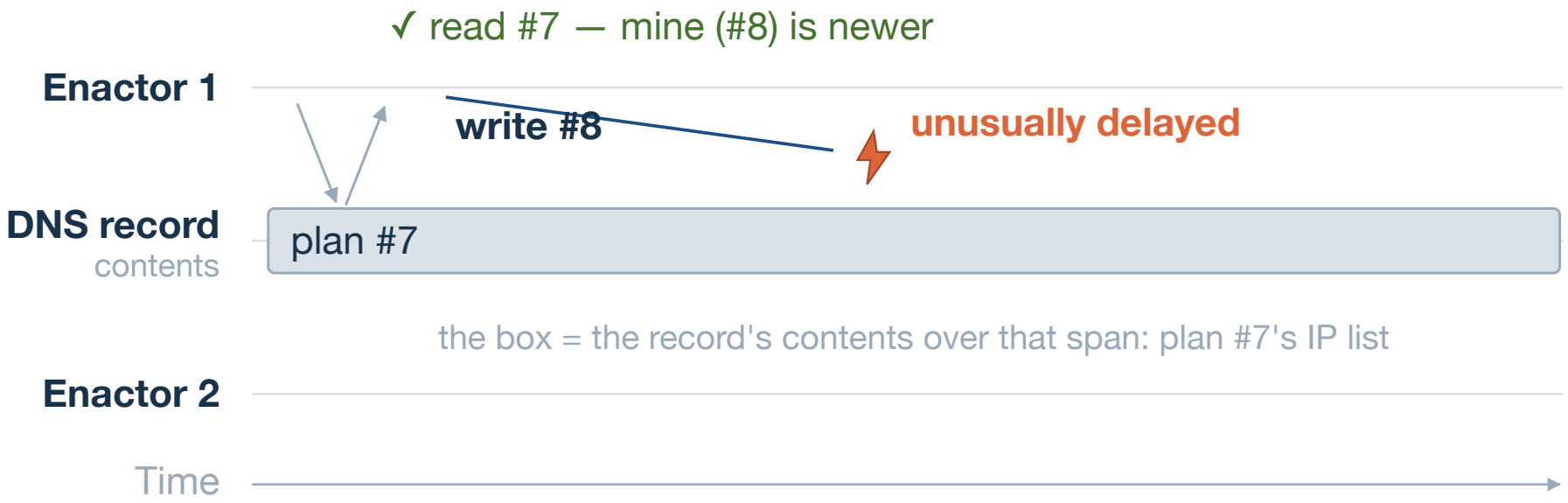
For fault tolerance, multiple Enactors run in three data centers (availability zones)

- Three writers, one shared record; sounds familiar?

Analysis: the interleaving that happened caused AWS's outage

An Enactor updates the record with RPCs: read, write, then clean-up

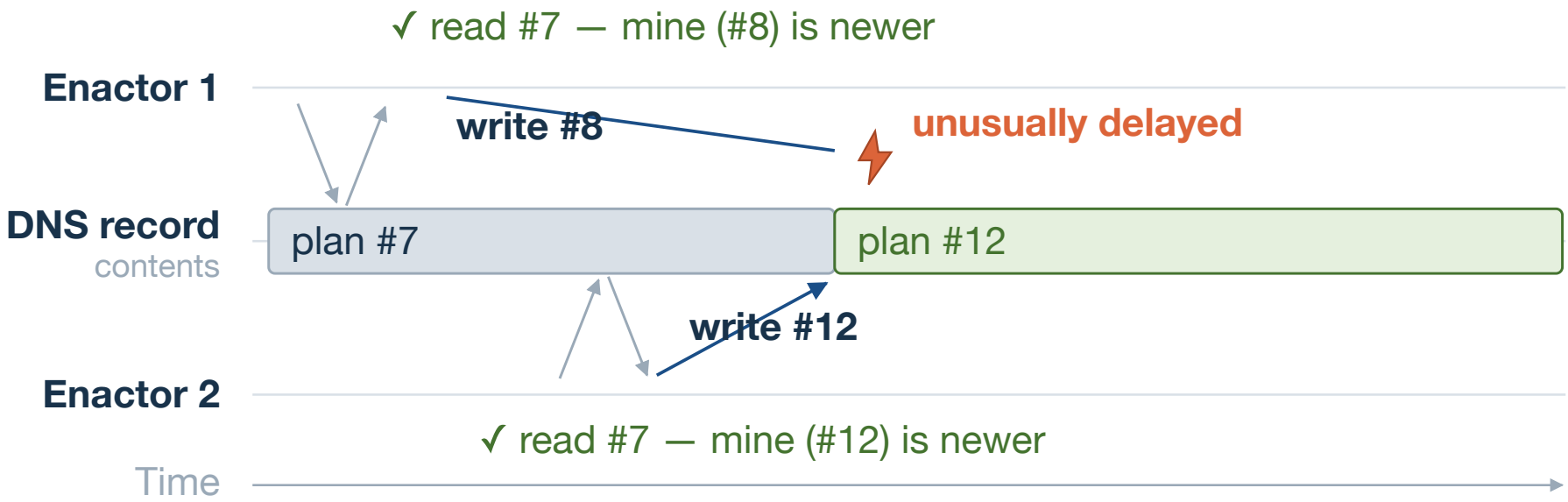
- Read: which plan does the record hold? If mine is newer, send the write
- After the write succeeds, one more RPC deletes the data of old plans



Analysis: the interleaving that happened caused AWS's outage

An Enactor updates the record with RPCs: read, write, then clean-up

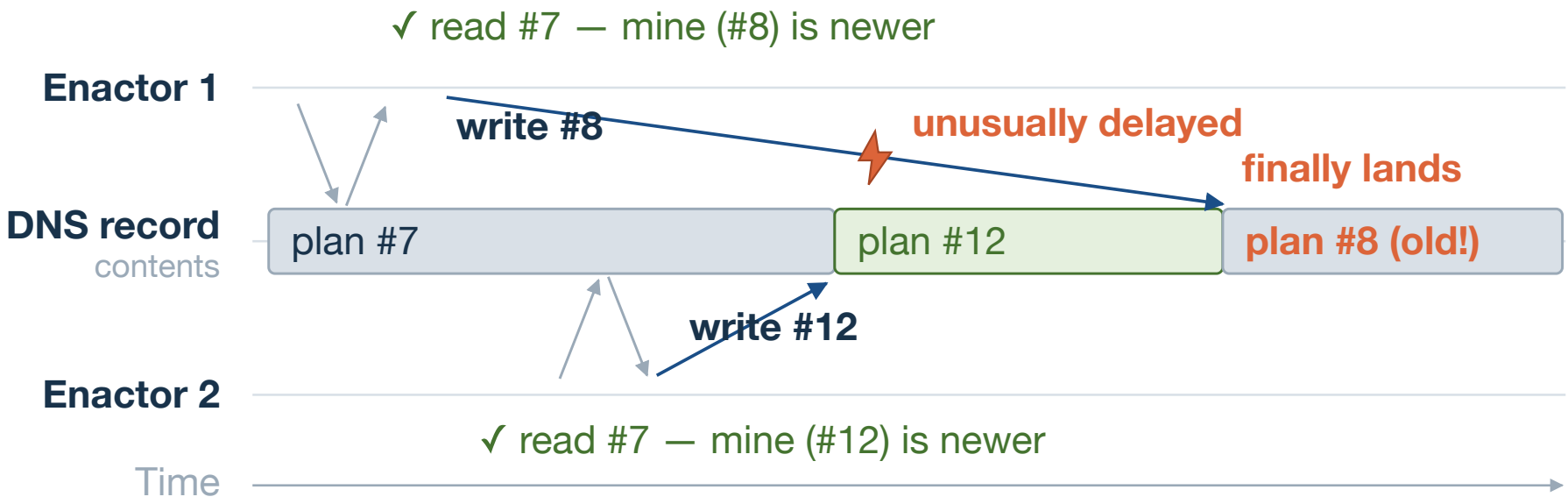
- Read: which plan does the record hold? If mine is newer, send the write
- After the write succeeds, one more RPC deletes the data of old plans



Analysis: the interleaving that happened caused AWS's outage

An Enactor updates the record with RPCs: read, write, then clean-up

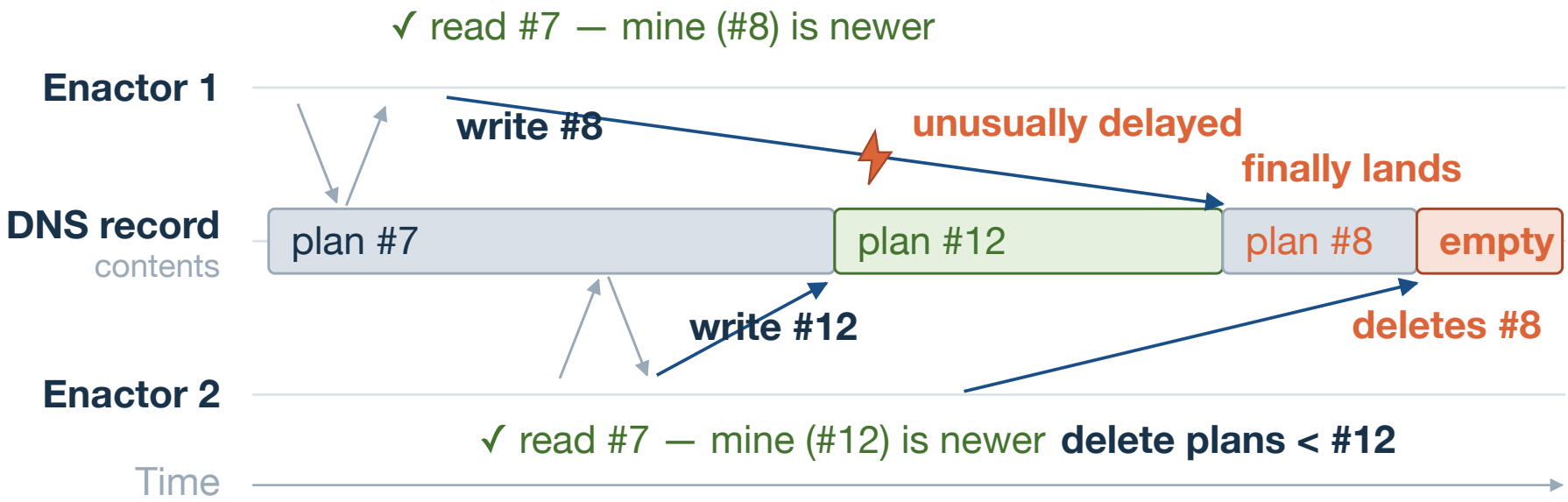
- Read: which plan does the record hold? If mine is newer, send the write
- After the write succeeds, one more RPC deletes the data of old plans



Analysis: the interleaving that happened caused AWS's outage

An Enactor updates the record with RPCs: read, write, then clean-up

- Read: which plan does the record hold? If mine is newer, send the write
- After the write succeeds, one more RPC deletes the data of old plans



Question: what happens if multiple Enactors interleave?

Summary: race conditions are harmful in practice if not handled

No data was corrupted, but the service was unavailable for ~2.5 hours

AWS's fixes:

- The Planner / Enactor automation was disabled worldwide
- Before re-enabling it: fix the race condition, and add protections against applying incorrect plans

📖 AWS post-mortem: "We have already disabled the DynamoDB DNS Planner and the DNS Enactor automation worldwide."

"In advance of re-enabling this automation, we will fix the race condition scenario and add additional protections to prevent the application of incorrect DNS plans."

Rethink: What is a strong consistency model

It is easy for users to reason about correctness assuming

- Everything has only one copy
- The overall behavior is equivalent to some serial behavior
- The operations belonging to one transaction happen completely or not at all under failure

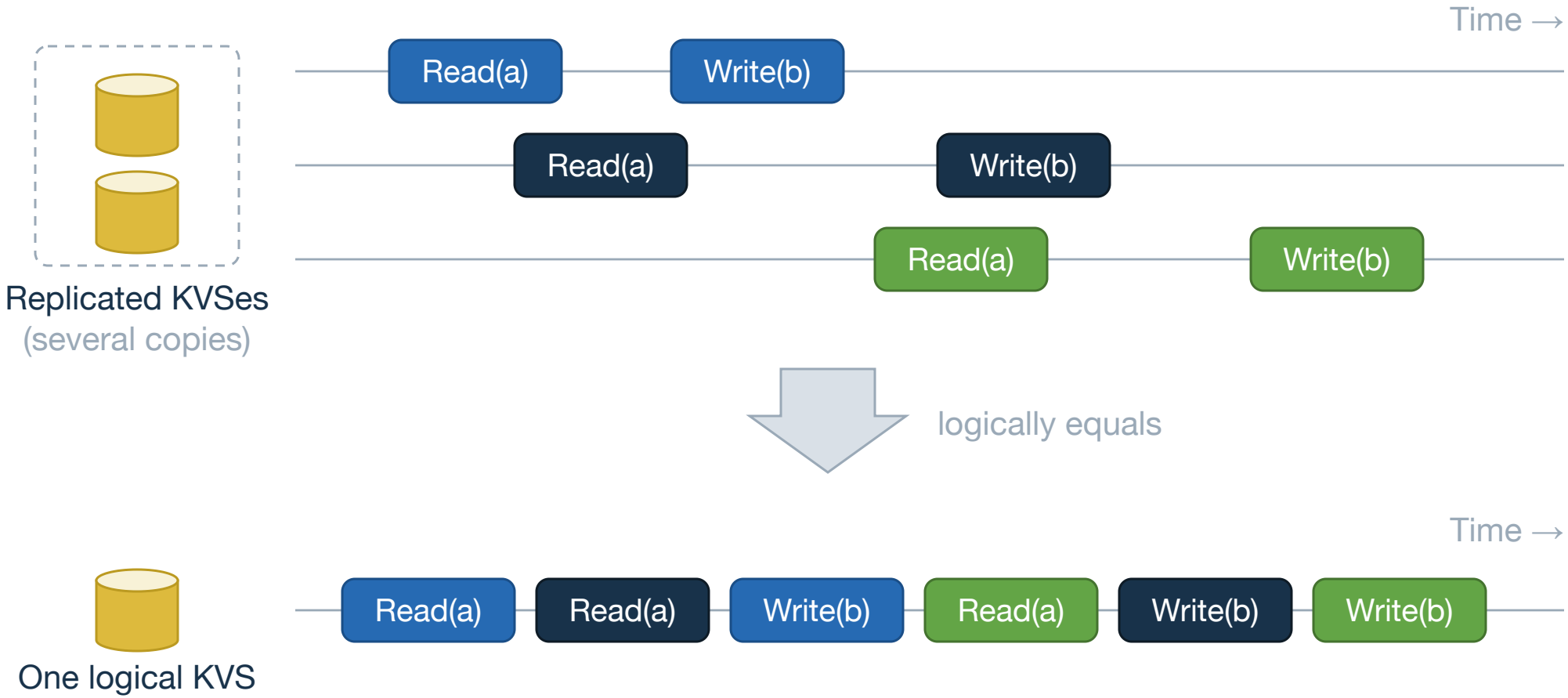
The first and the third are already covered

- The first can be done with primary-backup (covered in linearizability); more protocols will be discussed later
- The third came from the all-or-nothing atomicity lecture

Today we look at the second one more deeply

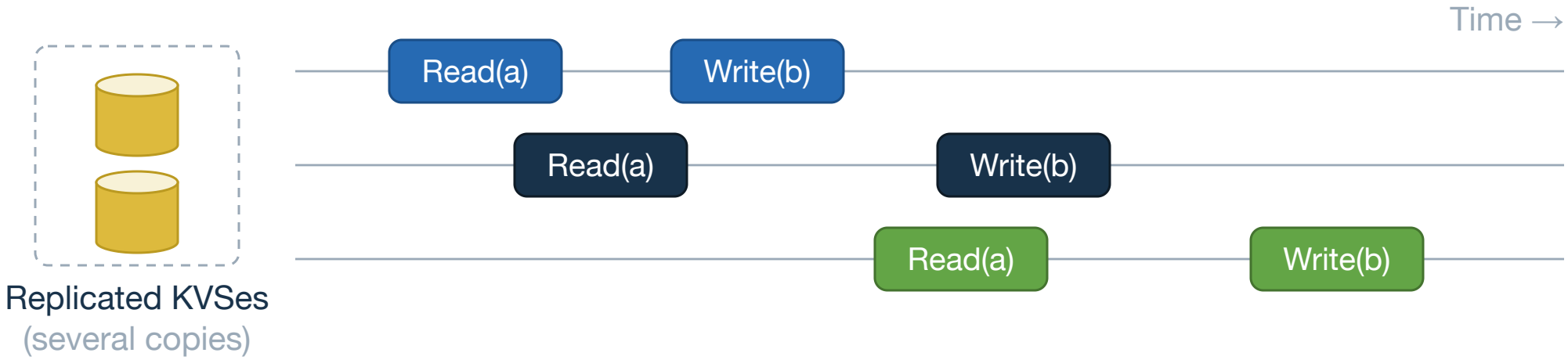
Review: serial execution of reads/writes

Linearizability tells us that the data reads and writes are serialized



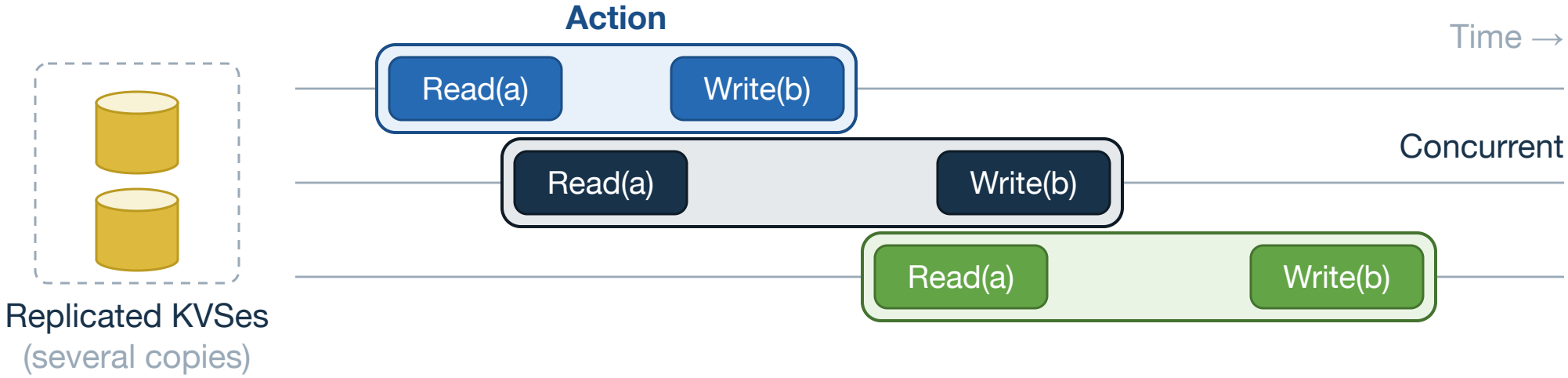
One step further: serial execution of actions

But what if multiple reads and writes must be grouped in an atomic action?



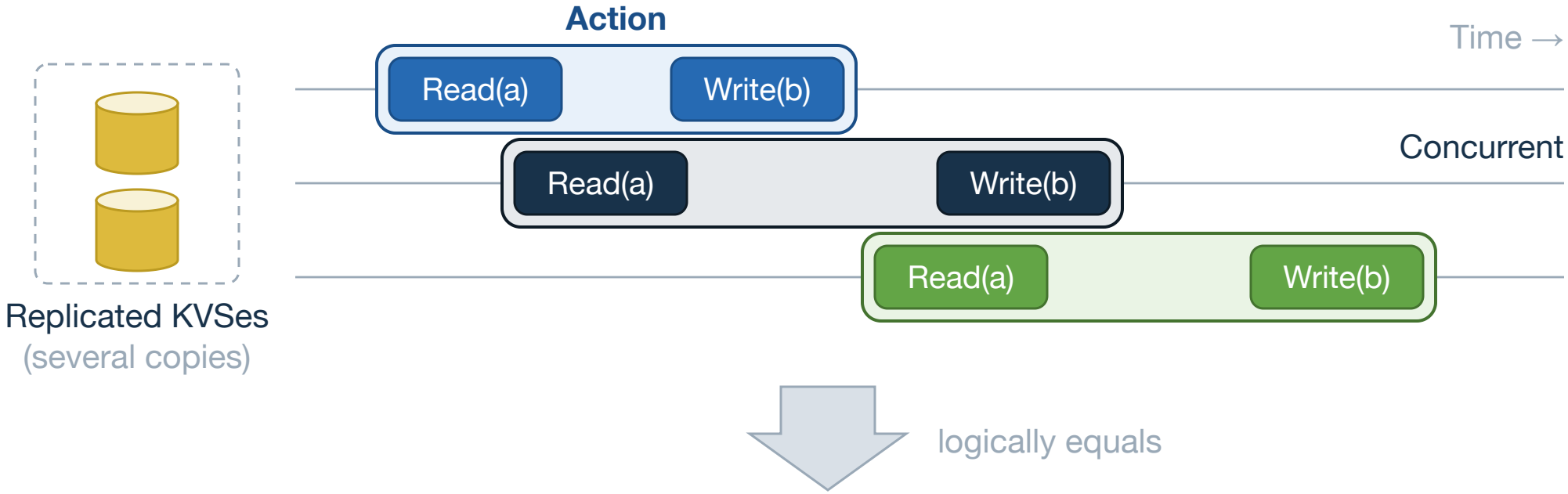
One step further: serial execution of actions

But what if multiple reads and writes must be grouped in an atomic action?



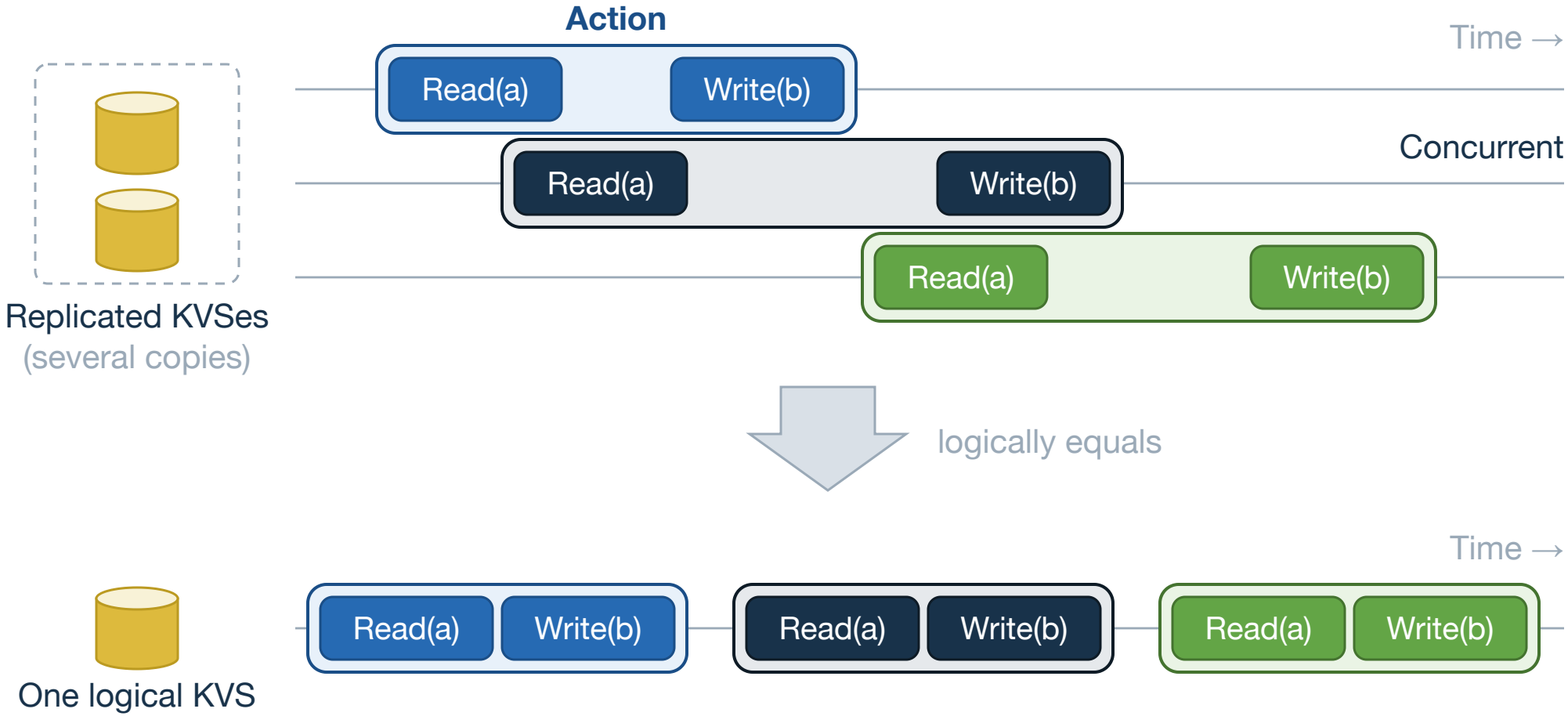
One step further: serial execution of actions

But what if multiple reads and writes must be grouped in an atomic action?



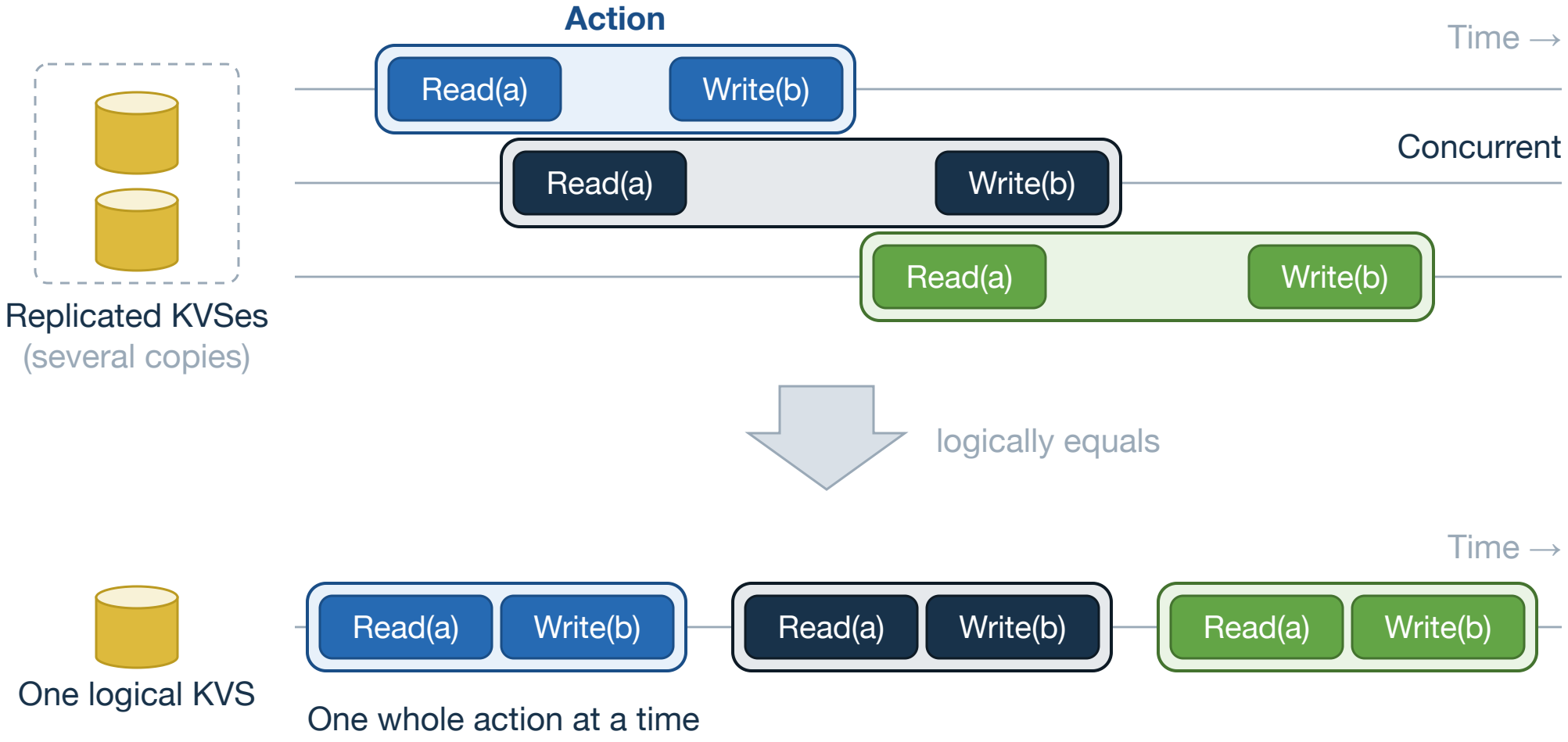
One step further: serial execution of actions

But what if multiple reads and writes must be grouped in an atomic action?



One step further: serial execution of actions

But what if multiple reads and writes must be grouped in an atomic action?



Goal: before-or-after atomicity (a.k.a. isolation)

To prevent the hazard produced by race conditions, we need a group of reads/writes to be atomic

- Nobody can see or overwrite the intermediate state of a concurrent action
- One action may contain multiple individually linearizable reads/writes

> **before-or-after atomicity**
>
> **From the point of view of their invokers, two concurrent actions occur either completely before or completely after one another** [1]

In other fields of computer science, before-or-after is also called isolation

[1] Definition from Saltzer & Kaashoek, *Principles of Computer System Design* §9.1.5

Realizing before-or-after: Locks

A lock is a data structure that only one entity (e.g., thread, or node) can acquire at a given time

- `acquire(lock)`: if the acquisition fails, the CPU waits until it succeeds
- `release(lock)`: hand it back
- Example: `pthread_mutex`

Implementing locks efficiently is a broad topic, out of the scope of this lecture

- The OS courses will discuss the various design choices deeper

Use locking to achieve before-or-after: global lock

Global lock: one single lock for the whole action

- The action must acquire the global lock before executing, and release it after it finishes

```
Deposit(bank, acct, amt):
    bank[acct] += amt
```

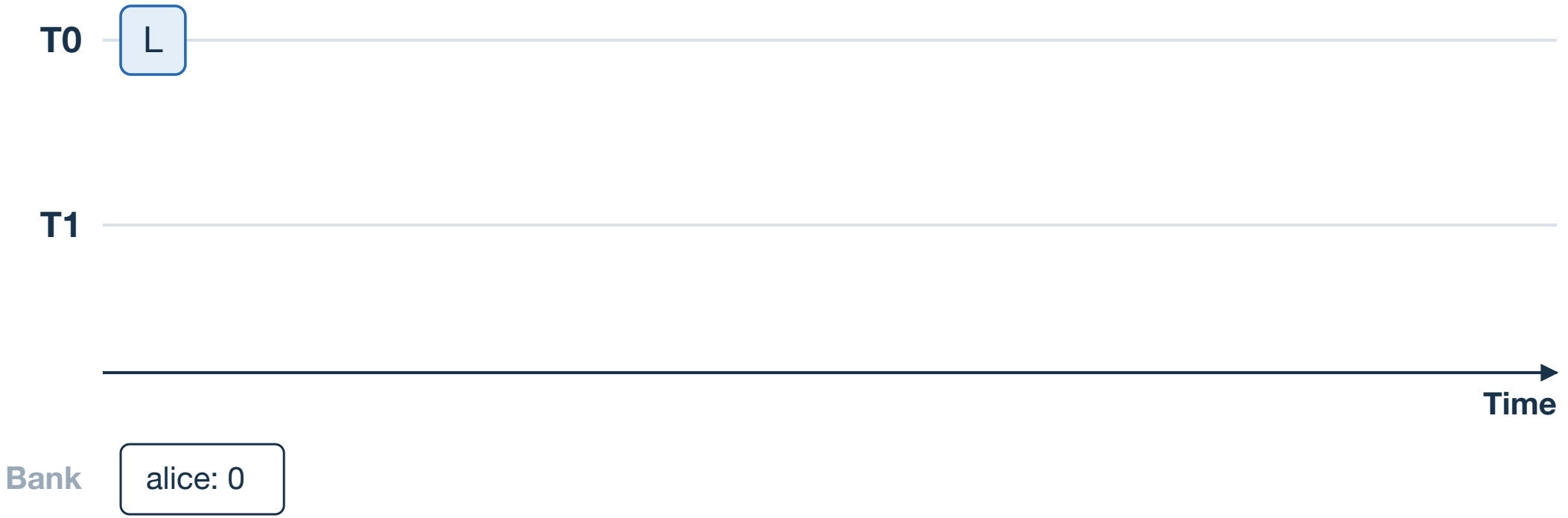
The code now looks something like this:

```
Deposit(bank, lock, acct, amt):
    acquire(lock)
    bank[acct] += amt
    release(lock)
```

Analysis: the same two deposits under a global lock

The global lock serializes the actions (the section in the lock is called the critical section)

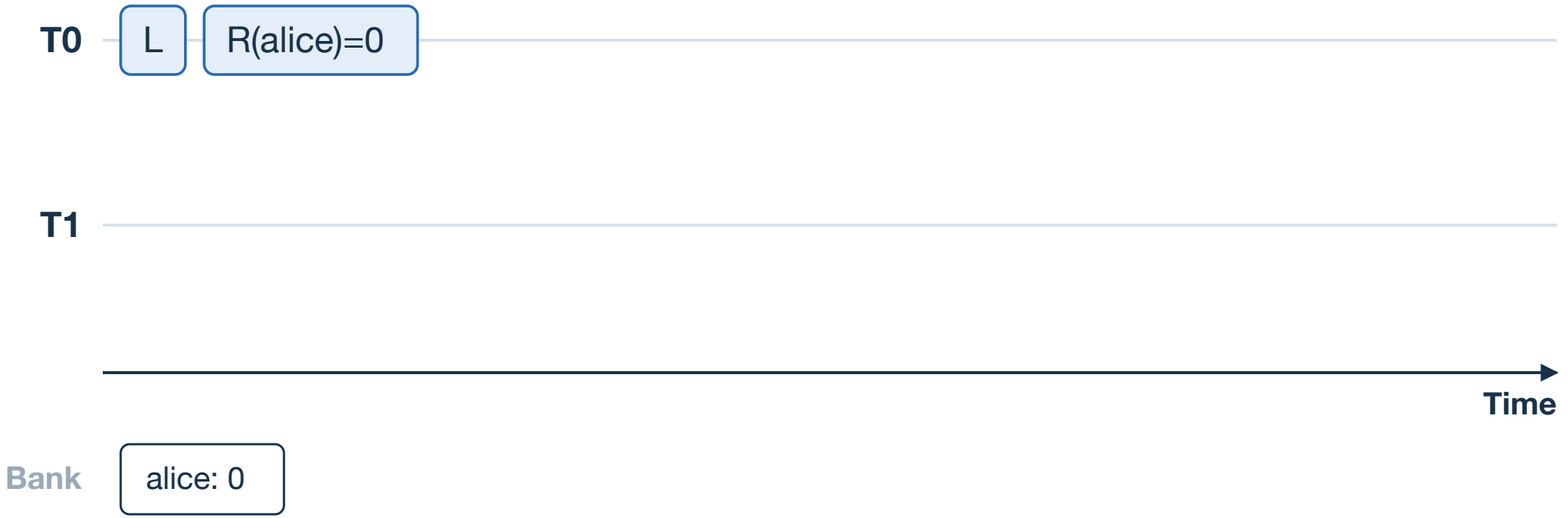
- T1 cannot start until T0 releases, so it reads what T0 wrote
- The result is the same as running T0 entirely before T1



Analysis: the same two deposits under a global lock

The global lock serializes the actions (the section in the lock is called the critical section)

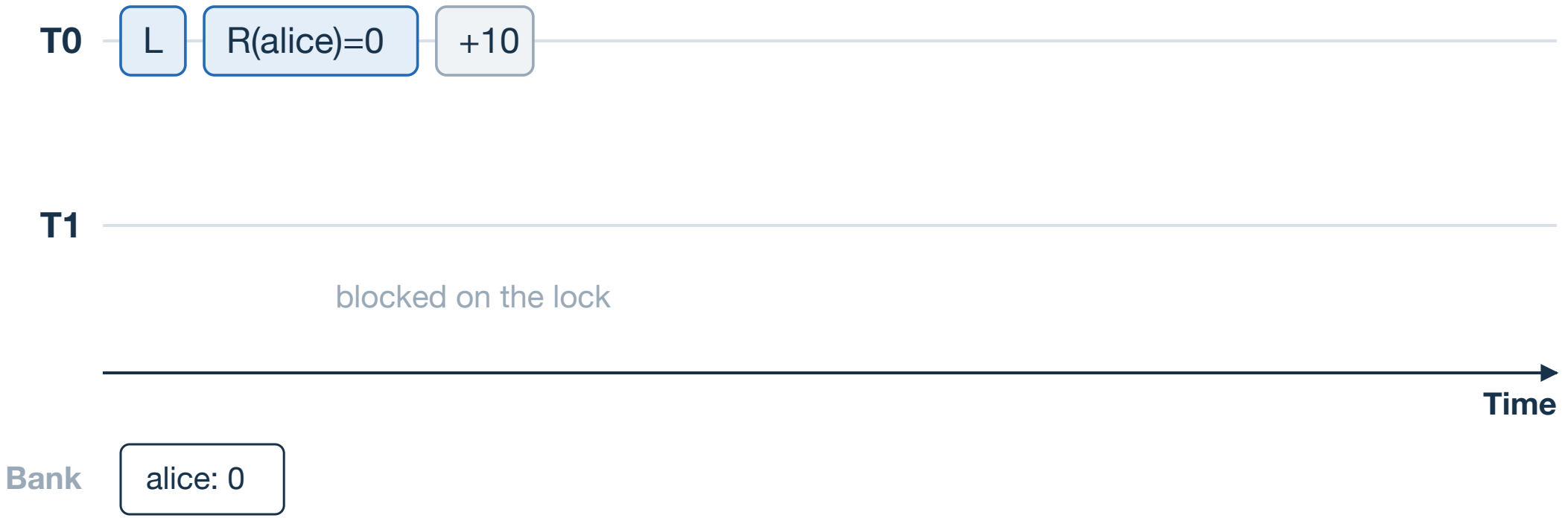
- T1 cannot start until T0 releases, so it reads what T0 wrote
- The result is the same as running T0 entirely before T1



Analysis: the same two deposits under a global lock

The global lock serializes the actions (the section in the lock is called the critical section)

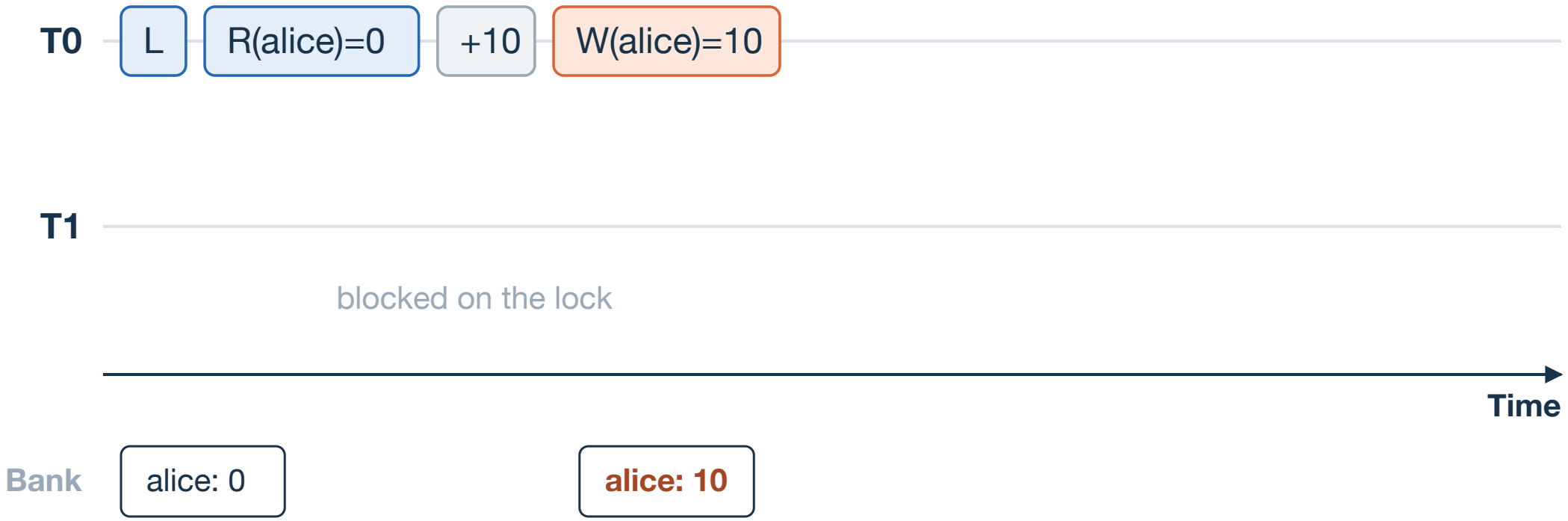
- T1 cannot start until T0 releases, so it reads what T0 wrote
- The result is the same as running T0 entirely before T1



Analysis: the same two deposits under a global lock

The global lock serializes the actions (the section in the lock is called the critical section)

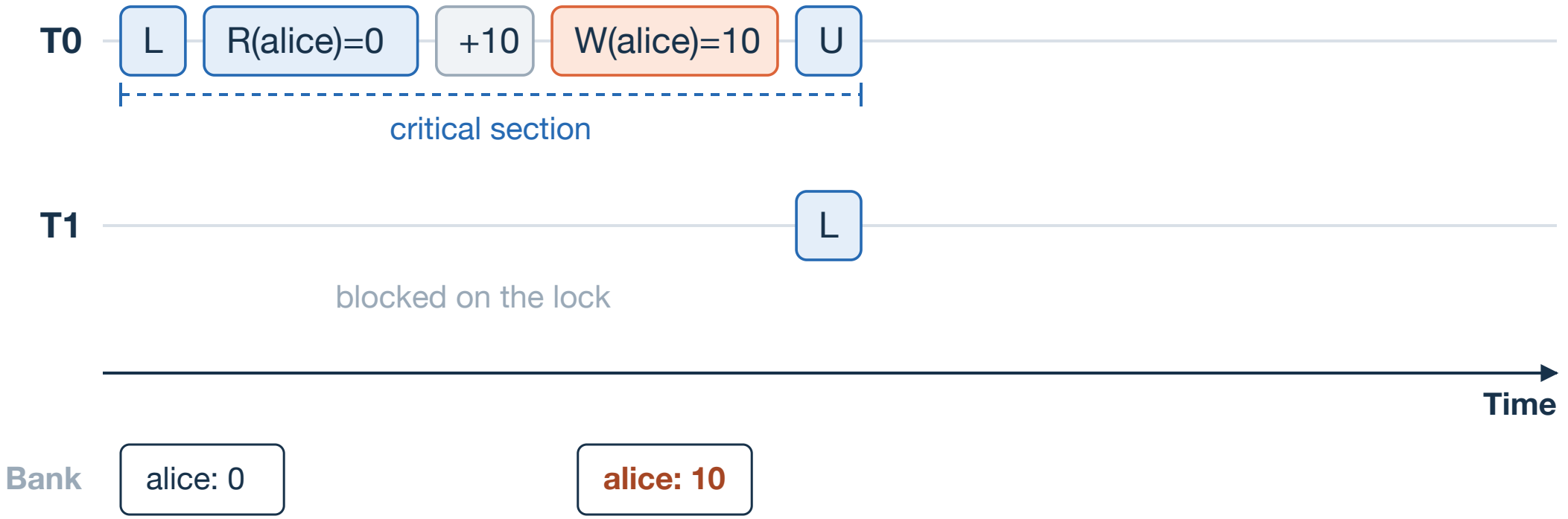
- T1 cannot start until T0 releases, so it reads what T0 wrote
- The result is the same as running T0 entirely before T1



Analysis: the same two deposits under a global lock

The global lock serializes the actions (the section in the lock is called the critical section)

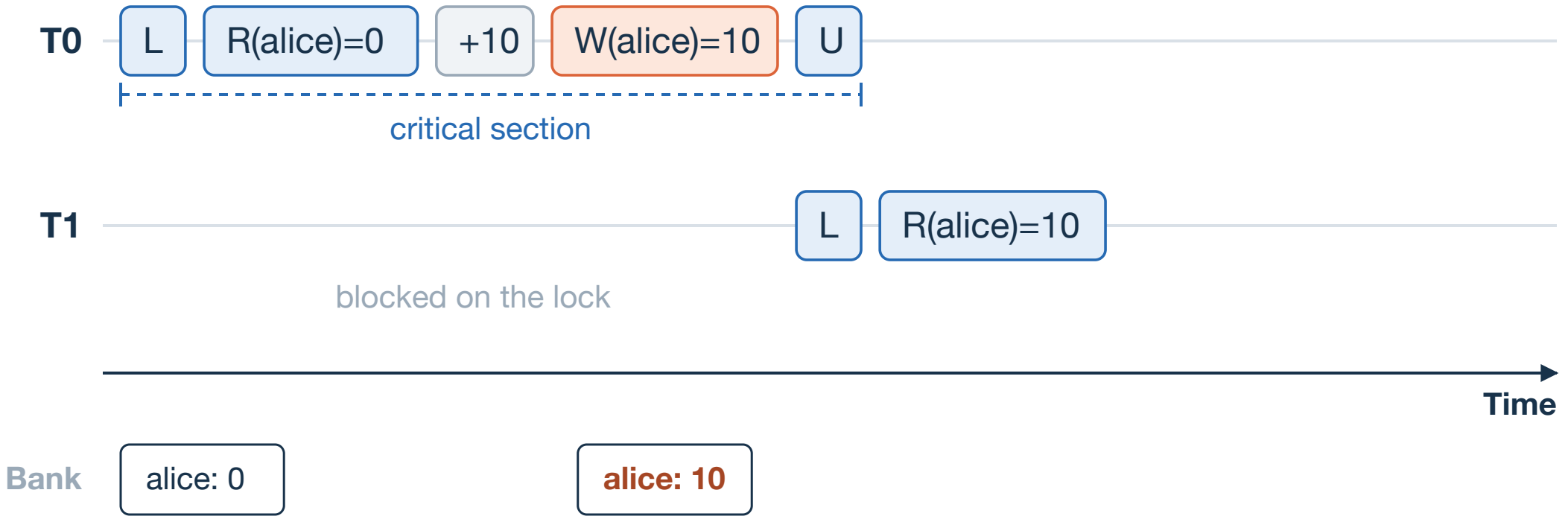
- T1 cannot start until T0 releases, so it reads what T0 wrote
- The result is the same as running T0 entirely before T1



Analysis: the same two deposits under a global lock

The global lock serializes the actions (the section in the lock is called the critical section)

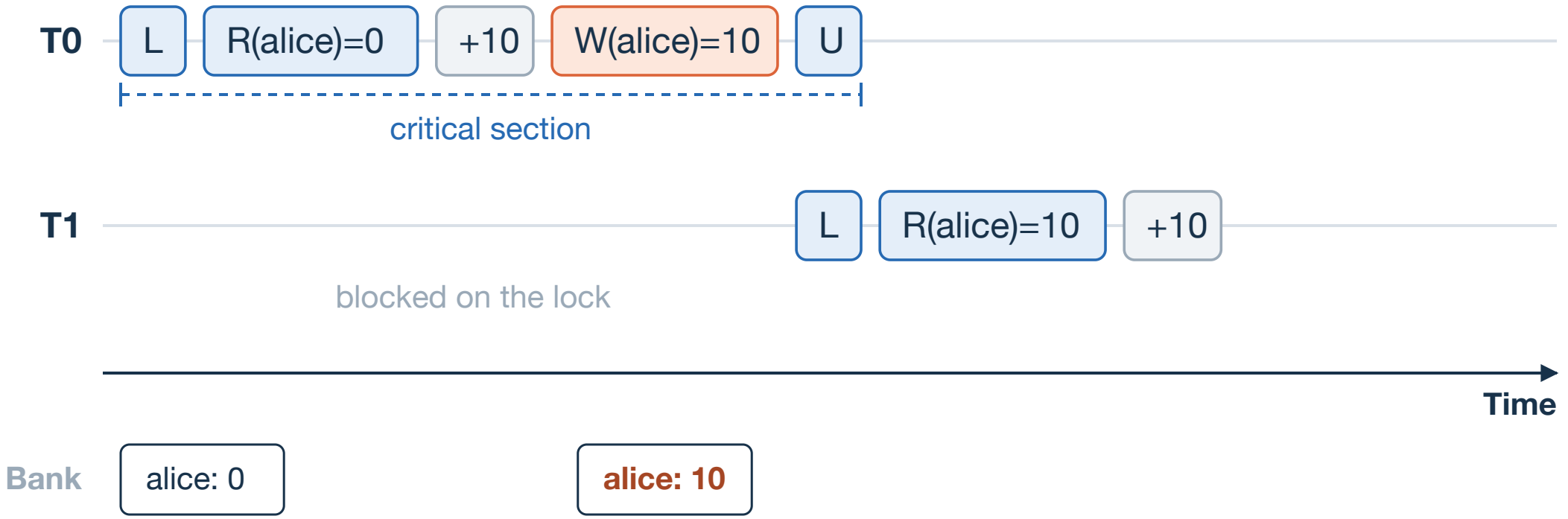
- T1 cannot start until T0 releases, so it reads what T0 wrote
- The result is the same as running T0 entirely before T1



Analysis: the same two deposits under a global lock

The global lock serializes the actions (the section in the lock is called the critical section)

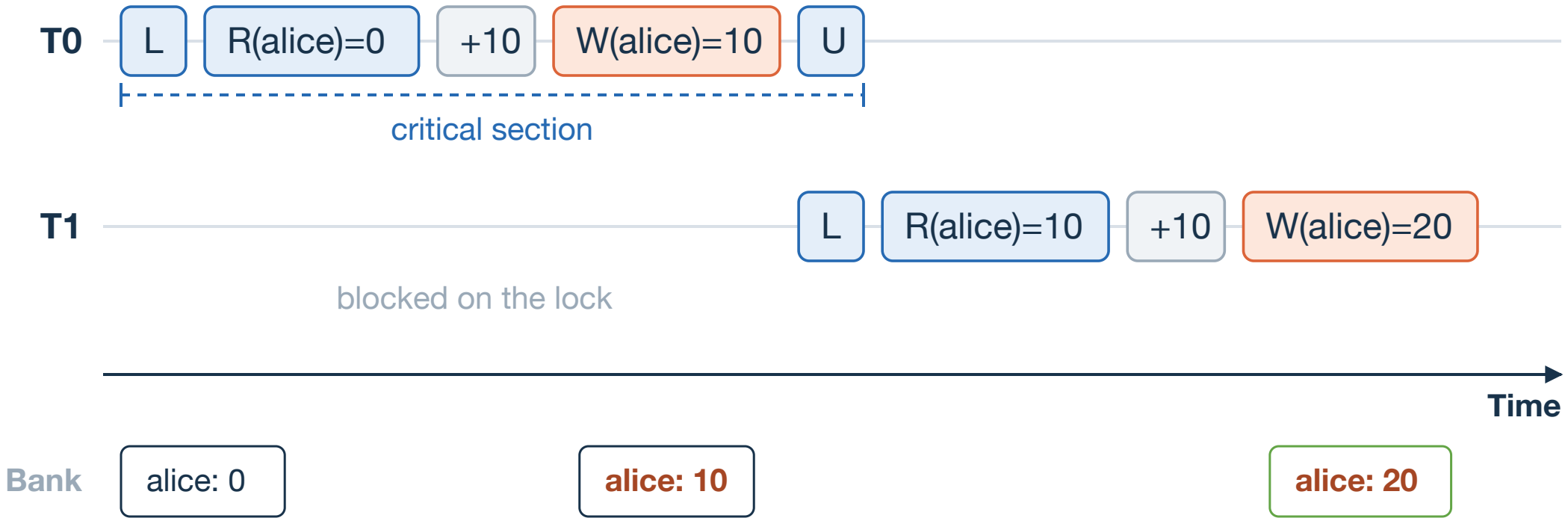
- T1 cannot start until T0 releases, so it reads what T0 wrote
- The result is the same as running T0 entirely before T1



Analysis: the same two deposits under a global lock

The global lock serializes the actions (the section in the lock is called the critical section)

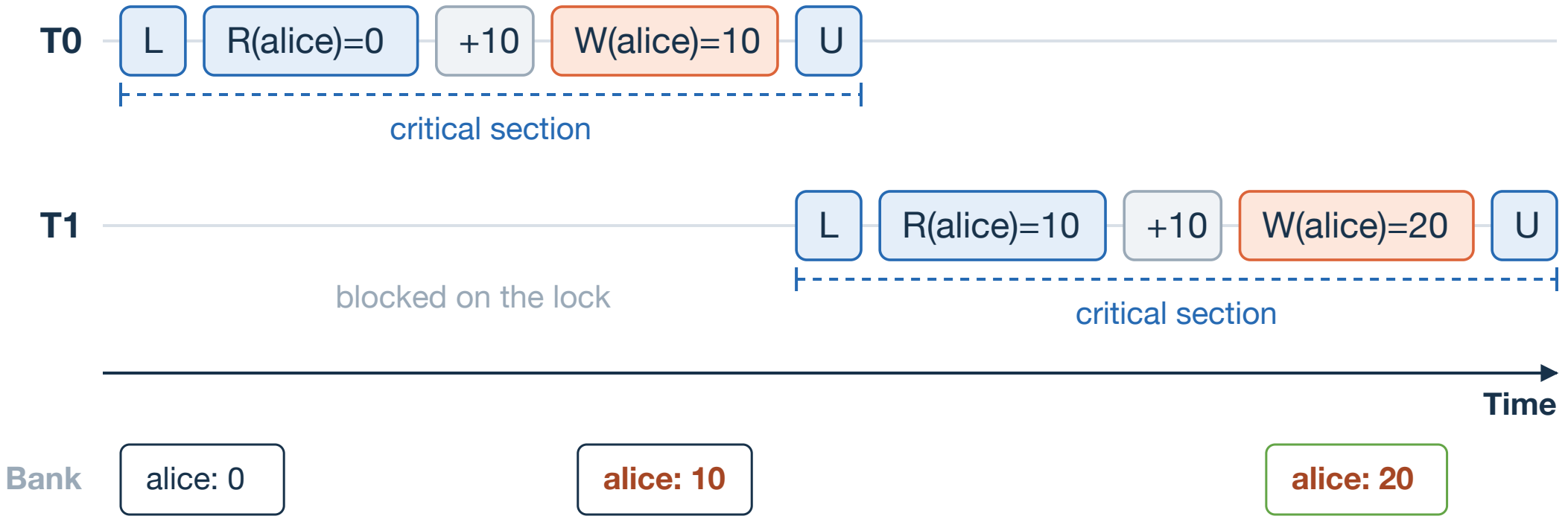
- T1 cannot start until T0 releases, so it reads what T0 wrote
- The result is the same as running T0 entirely before T1



Analysis: the same two deposits under a global lock

The global lock serializes the actions (the section in the lock is called the critical section)

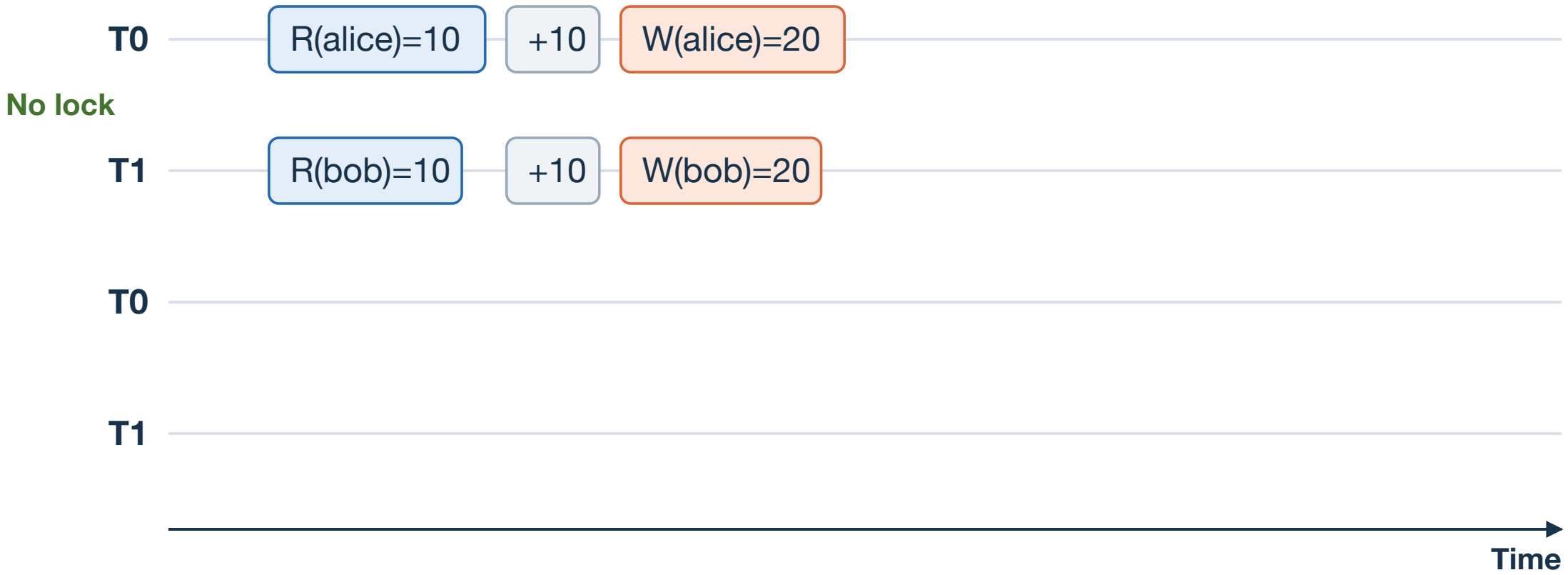
- T1 cannot start until T0 releases, so it reads what T0 wrote
- The result is the same as running T0 entirely before T1



Global lock: trading performance for correctness

Only one action runs at a time

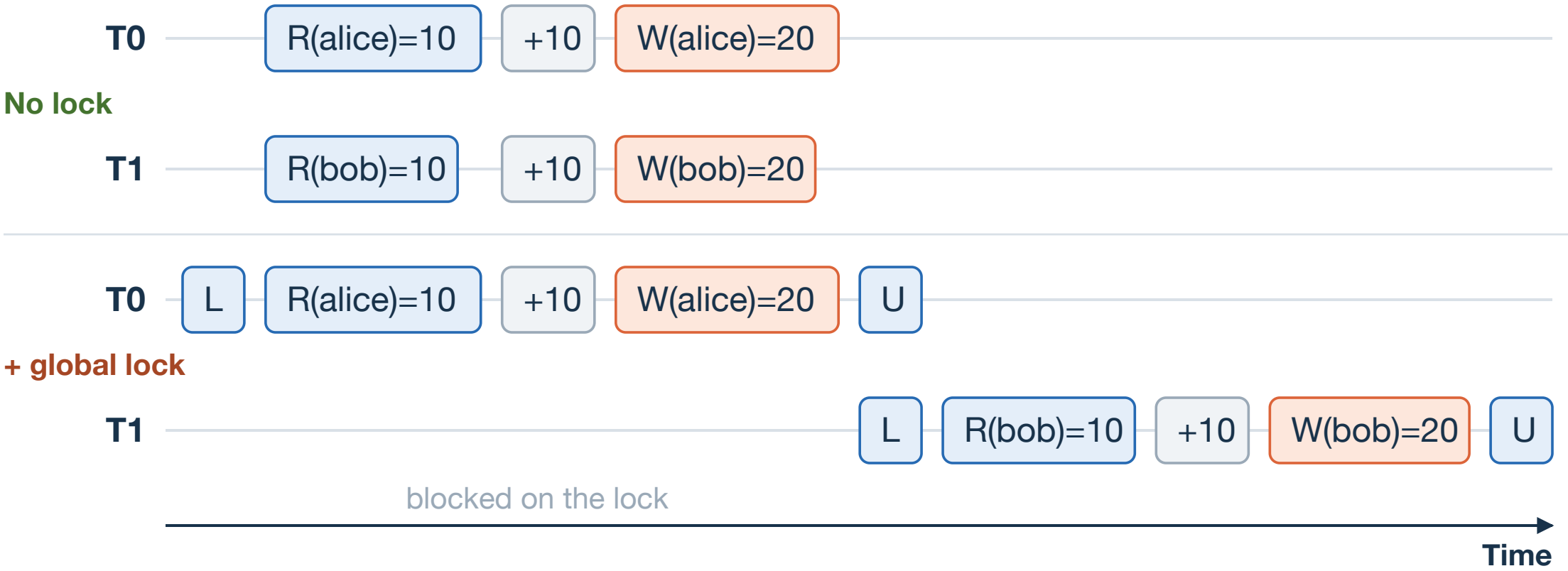
- Two actions are serialized even though they could run concurrently
- May be slower than the single-threaded setup we started from (no throughput gain!)



Global lock: trading performance for correctness

Only one action runs at a time

- Two actions are serialized even though they could run concurrently
- May be slower than the single-threaded setup we started from (no throughput gain!)



Question: how to optimize?

Method #1: Reduce the locked scope (size of the critical section): not all code needs to be blocked

- i.e., only use a lock to protect a portion of the code
- Question: how to select the portion?

Method #2: Make acquiring and releasing the lock faster

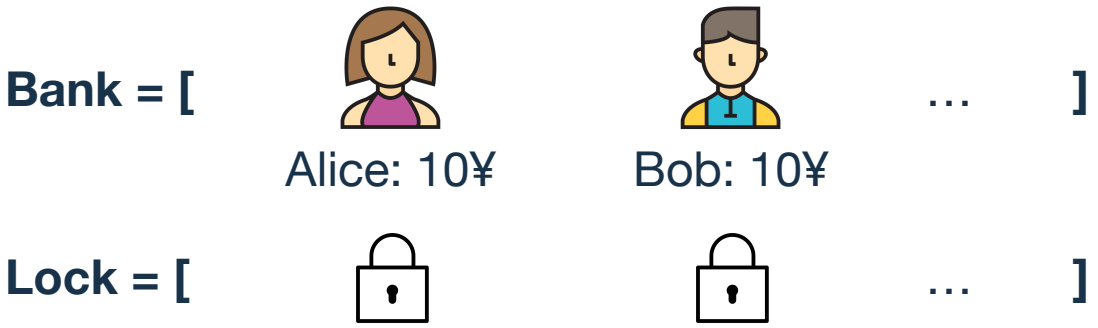
- Does not help in the case of global locking
- Is orthogonal to Method #1

Use locking to achieve before-or-after: fine-grained locking

Observation: only shared data need to be protected, others don't

Fine-grained locking: each data record is protected by a dedicated lock

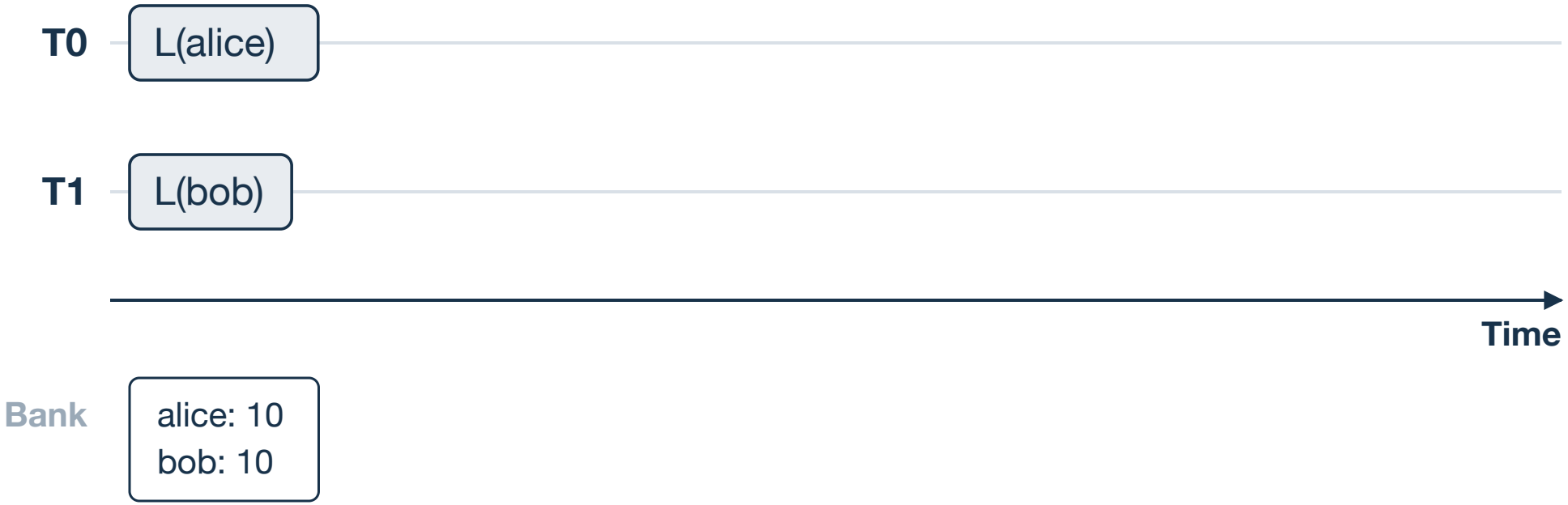
- A lock for Alice and a lock for Bob
- Lock rule: acquire a record's lock before accessing it, release it once the access finishes




```
Deposit(bank, locks, acct, amt):
    acquire(locks[acct])
    bank[acct] += amt
    release(locks[acct])
```

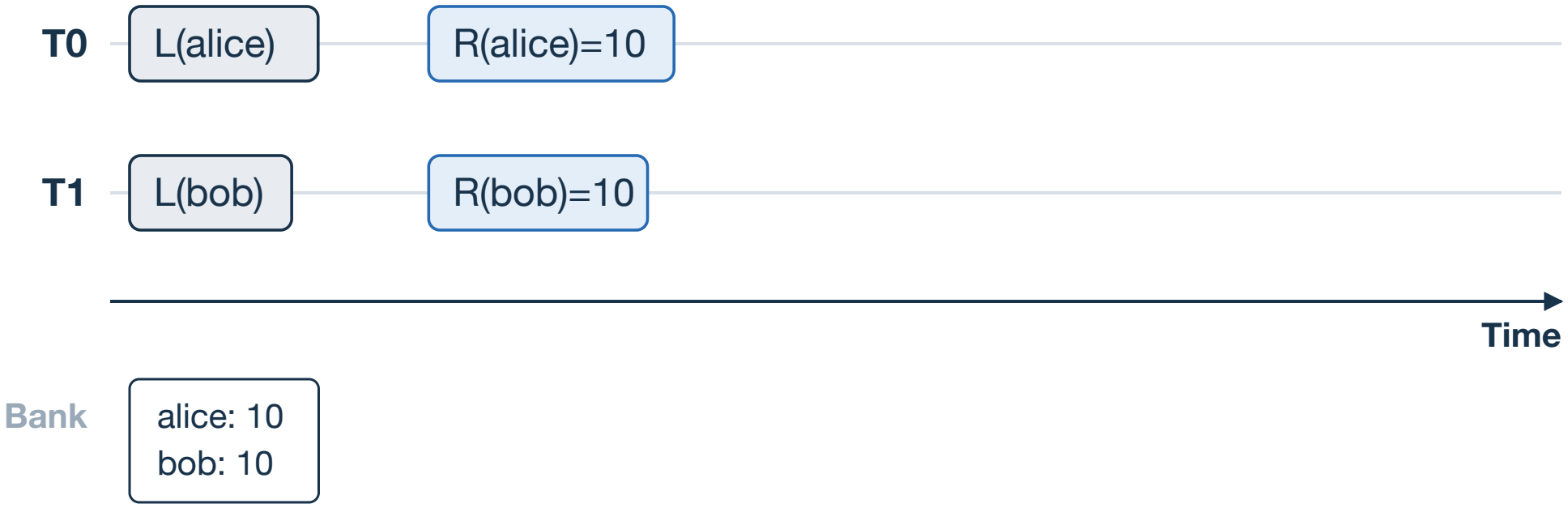
Fine-grained locking allows more concurrency

Two deposits on different accounts can really run concurrently



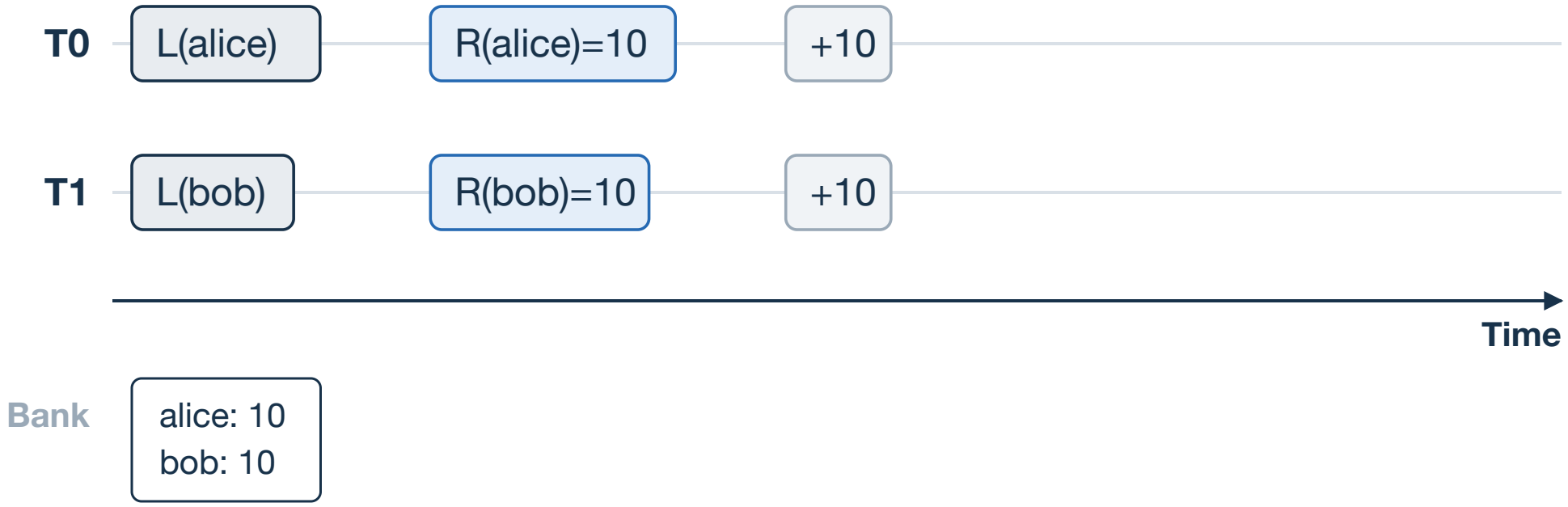
Fine-grained locking allows more concurrency

Two deposits on different accounts can really run concurrently



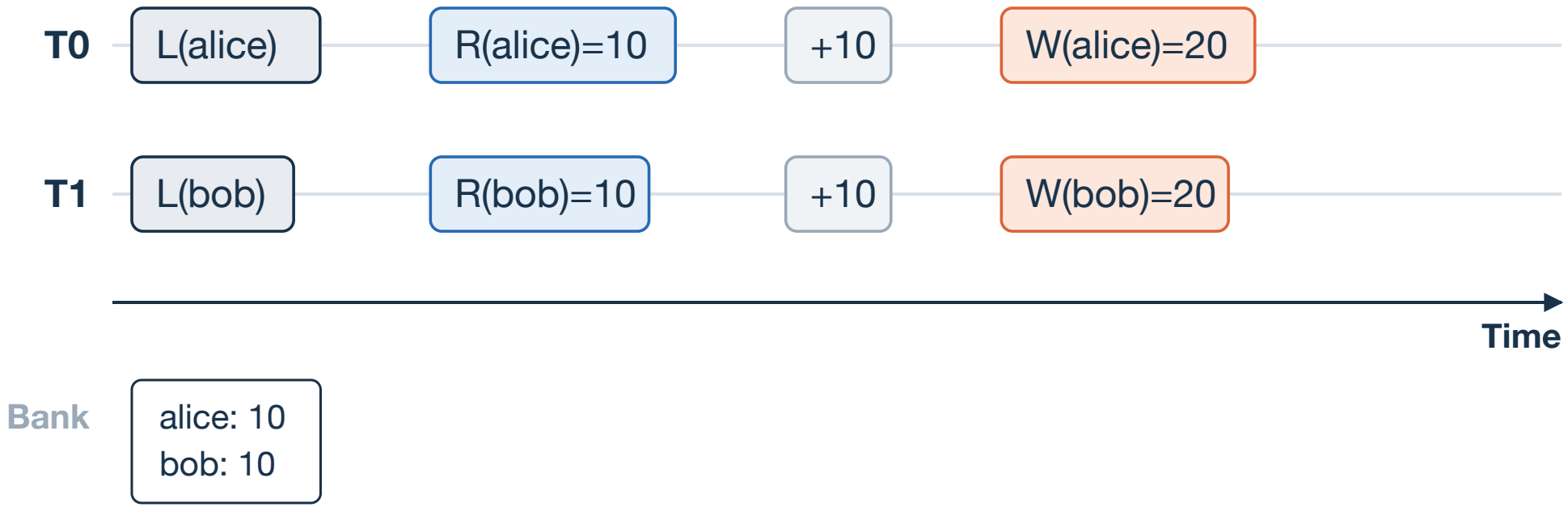
Fine-grained locking allows more concurrency

Two deposits on different accounts can really run concurrently



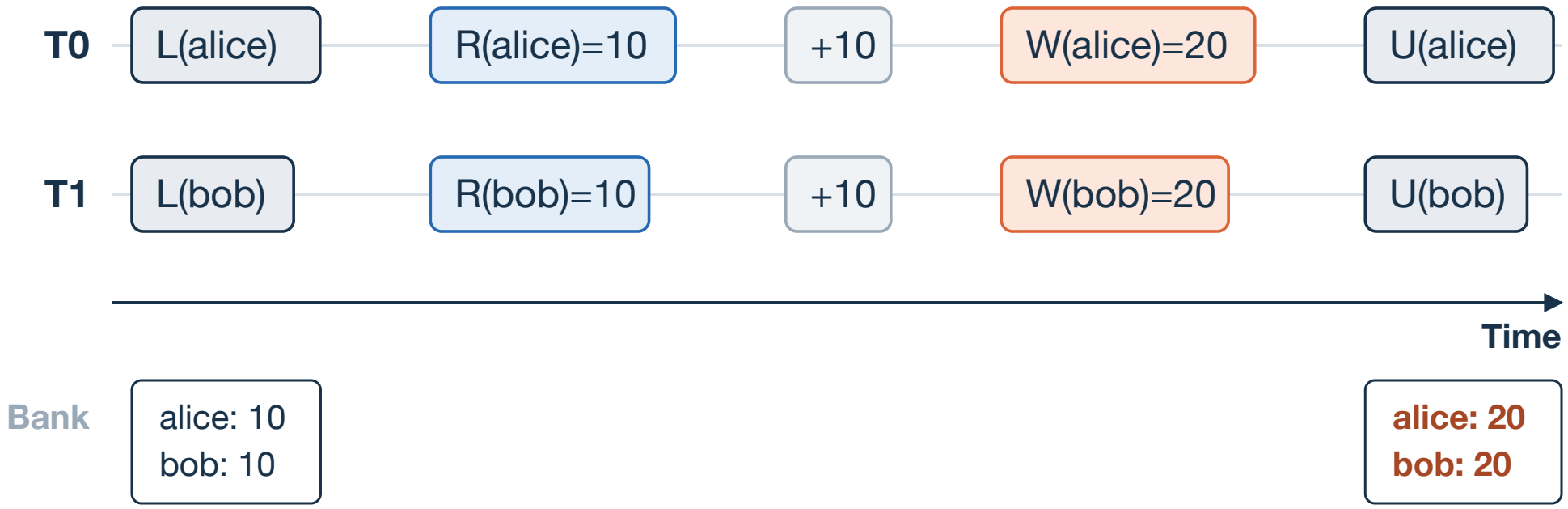
Fine-grained locking allows more concurrency

Two deposits on different accounts can really run concurrently



Fine-grained locking allows more concurrency

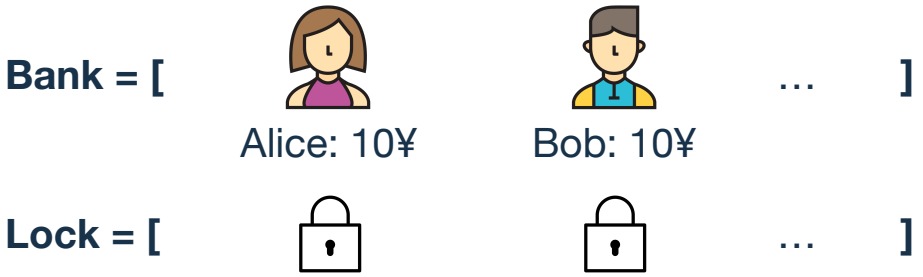
Two deposits on different accounts can really run concurrently



Can fine-grained locking avoid any race condition?

A more complex example: Transfer + Audit

Suppose our bank needs to support more actions

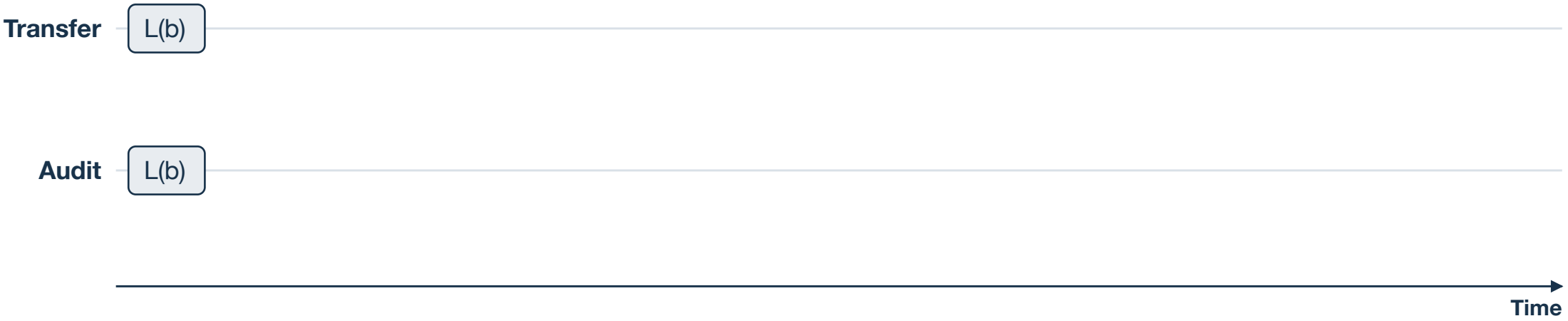



```
Transfer(bank, locks, a, b, amt):
    acquire(locks[b]); bank[b] += amt; release(locks[b])
    acquire(locks[a]); bank[a] -= amt; release(locks[a])
```

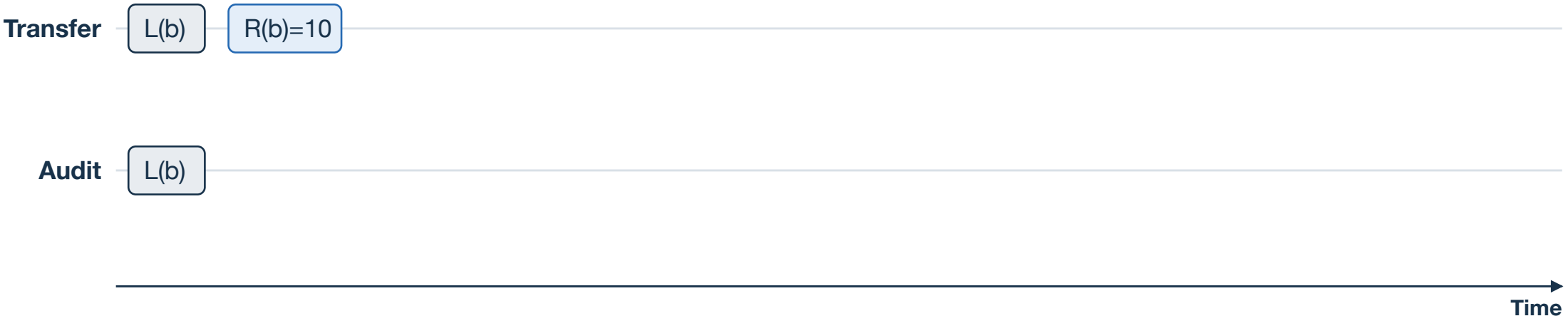
```
Audit(bank, locks):
    sum = 0
    for acct in bank:
        acquire(locks[acct]); sum += bank[acct]; release(locks[acct])
    print(sum)
```

Question: what is the ideal output of Audit?

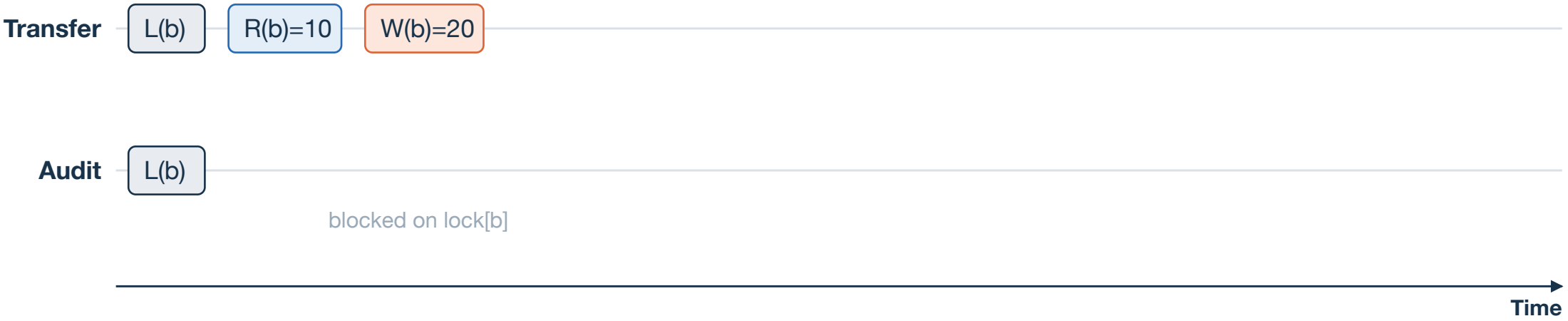
Question: think about the following execution?



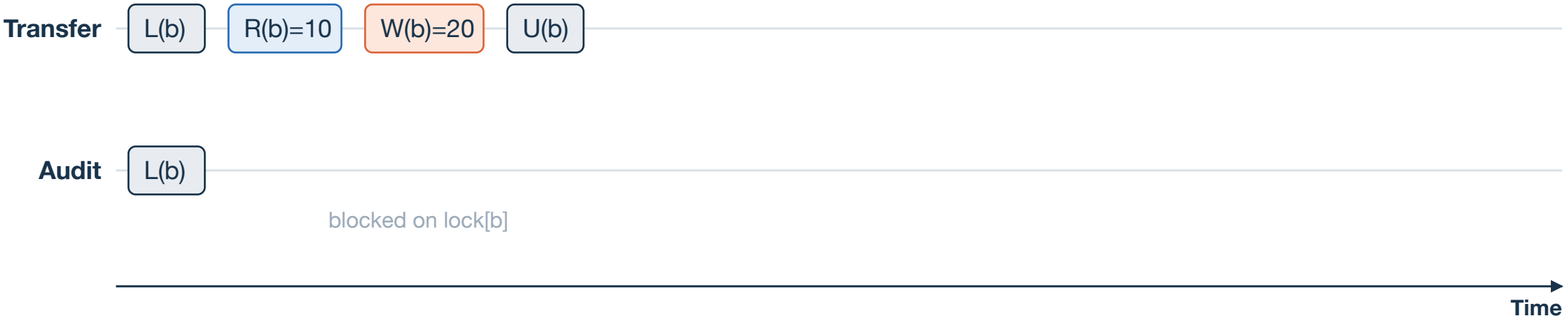
Question: think about the following execution?



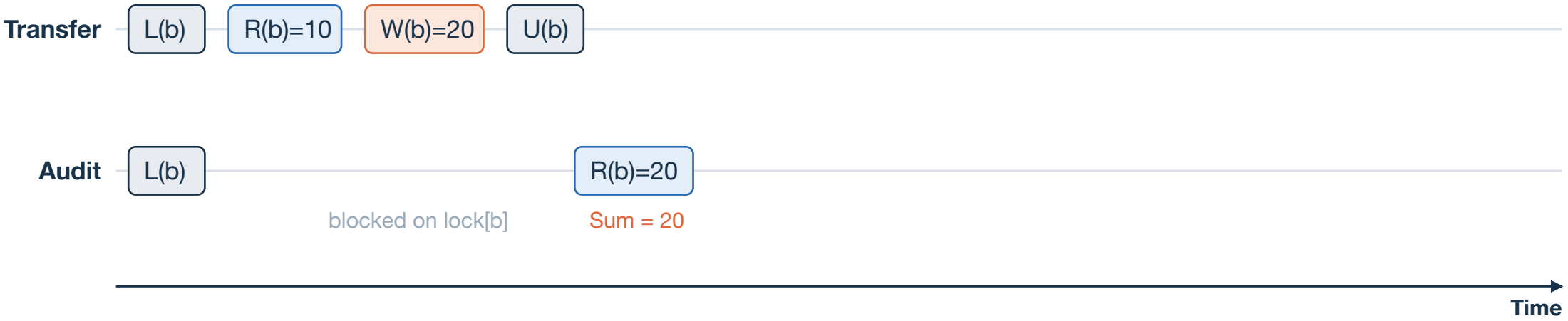
Question: think about the following execution?



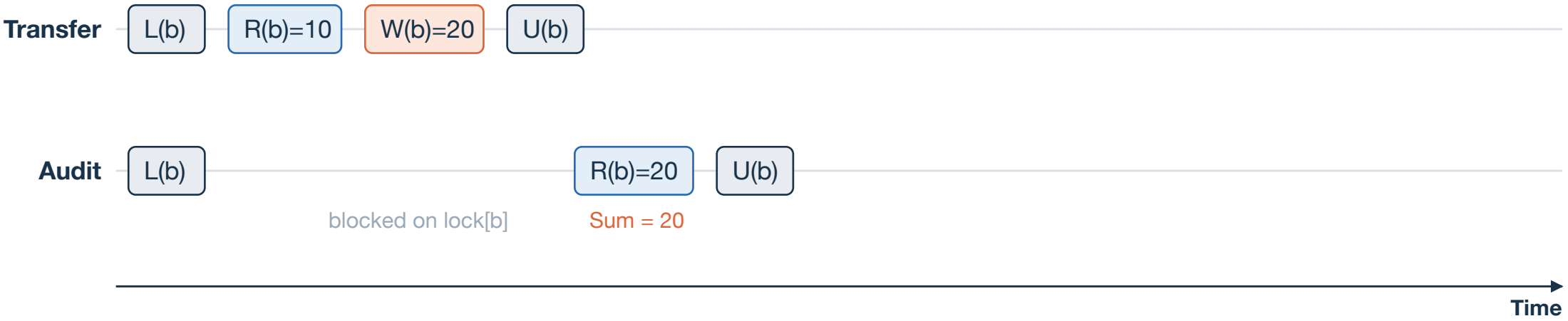
Question: think about the following execution?



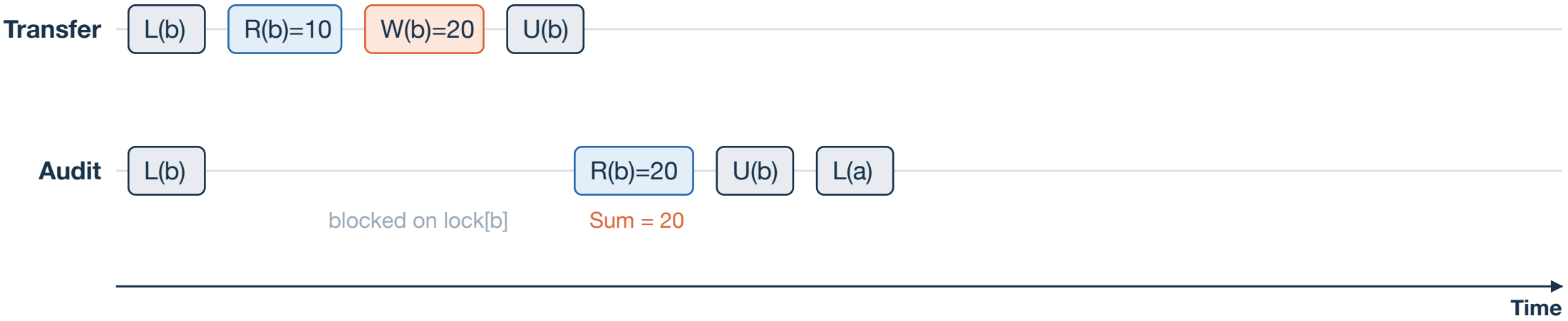
Question: think about the following execution?



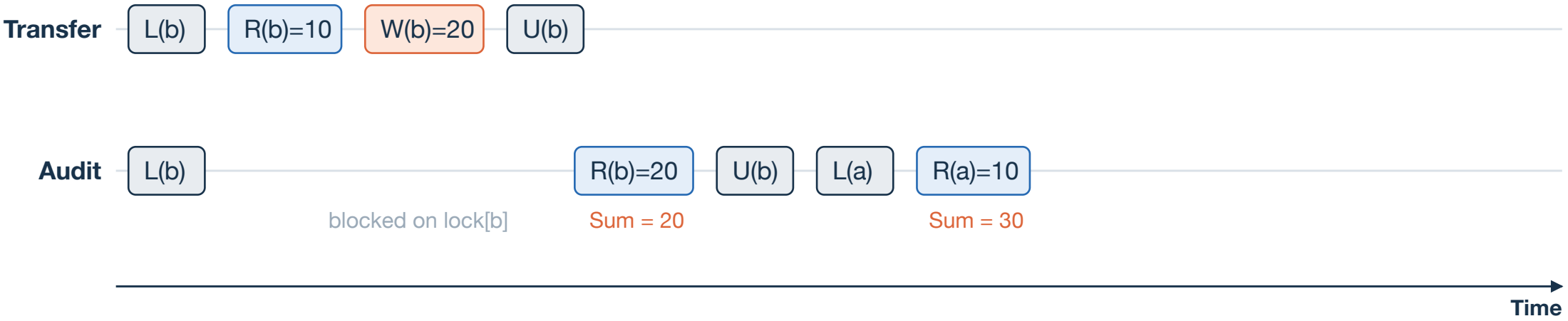
Question: think about the following execution?



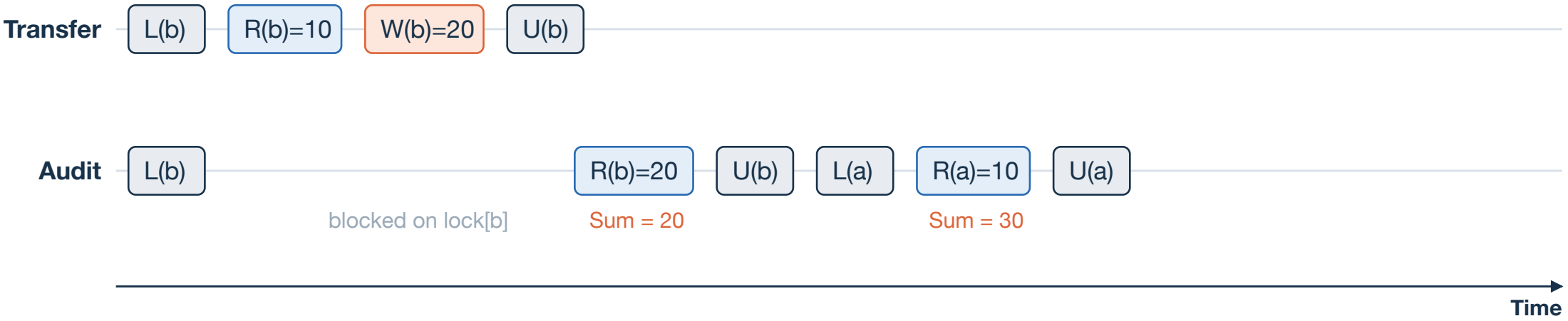
Question: think about the following execution?



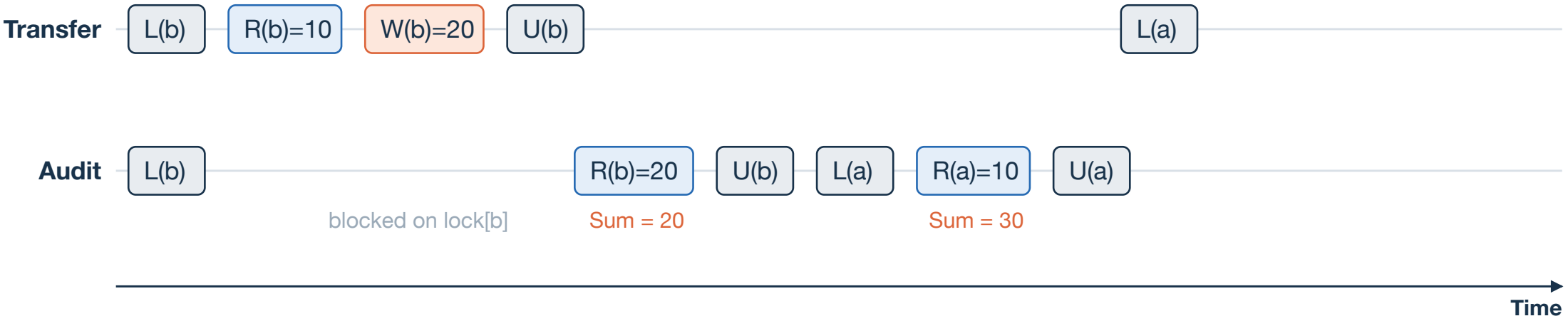
Question: think about the following execution?



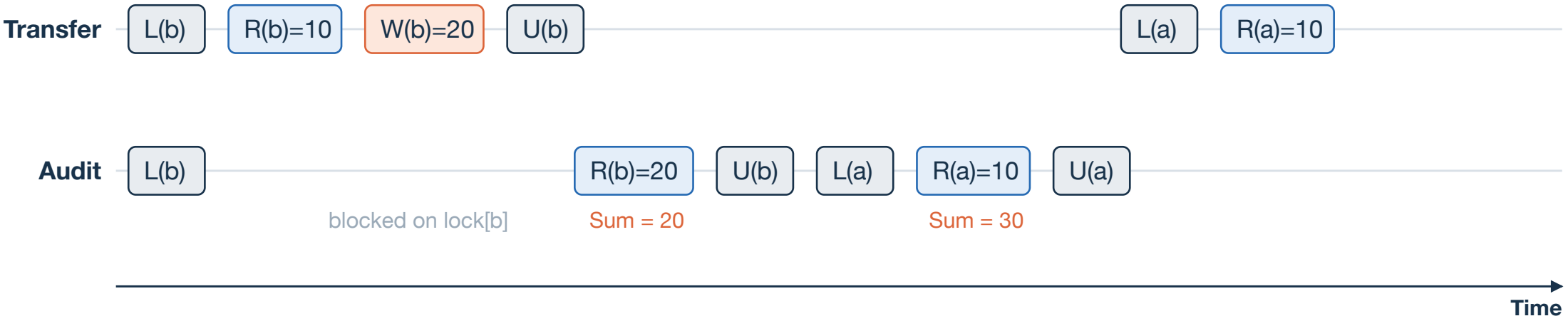
Question: think about the following execution?



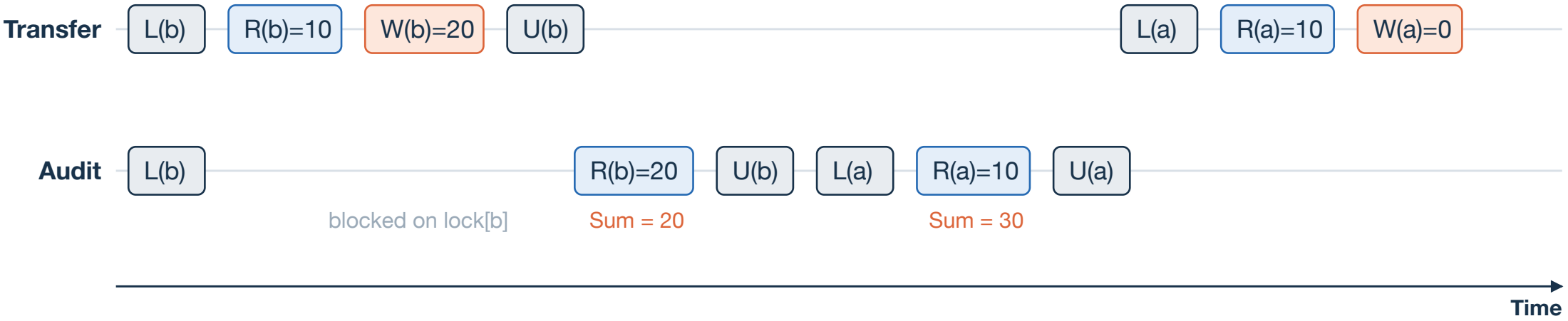
Question: think about the following execution?



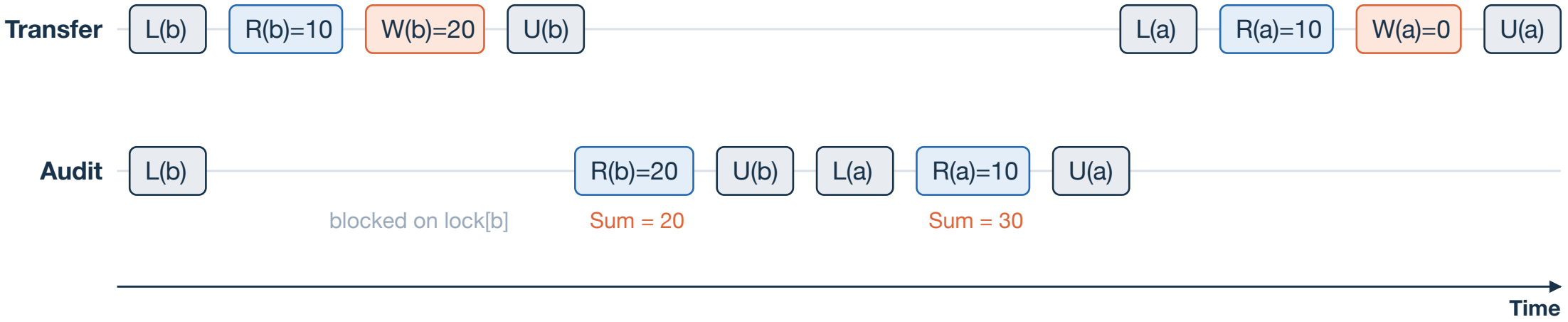
Question: think about the following execution?



Question: think about the following execution?



Question: think about the following execution?



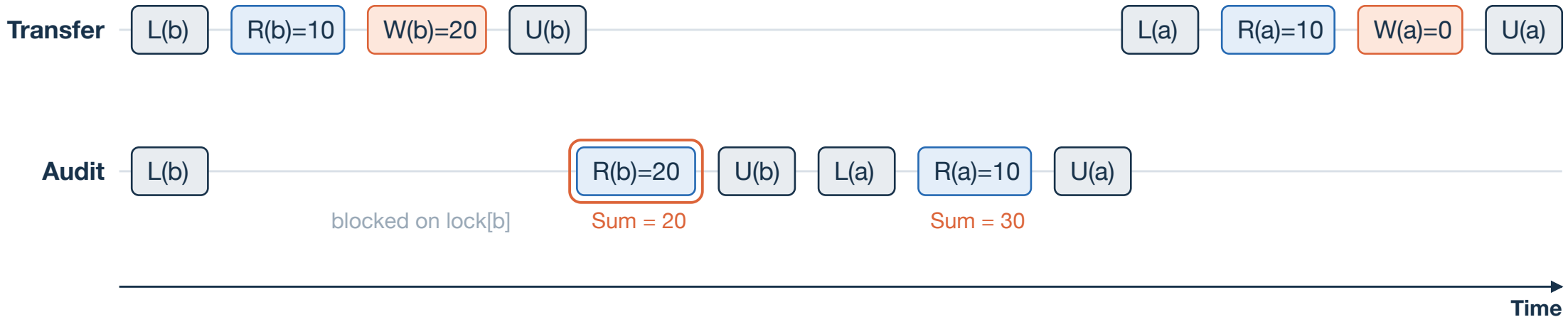
Question #1: is the final state of the program (i.e., the bank accounts) correct?

Question #2: is the sum printed by Audit correct?

Problem: the intermediate state is exposed with improper lock scope

Simple fine-grained locking does not prevent the exposure of an intermediate result

- e.g., The moment Transfer releases `lock[b]`



Insight: the lock needs to be released once no intermediate state can be exposed

- e.g., released at the end of the execution

Use locking to achieve before-or-after: static locking

Static locking: take every lock before the action starts, give them all back after it ends

- The granularity does not change: still one lock per record
- What changes is how long a lock is held — for the whole action, not for one access

```
Transfer(bank, locks, a, b, amt):
    acquire_all(locks[a], locks[b])     # every lock, before any access
    bank[b] += amt
    bank[a] -= amt
    release_all(locks[a], locks[b])
```

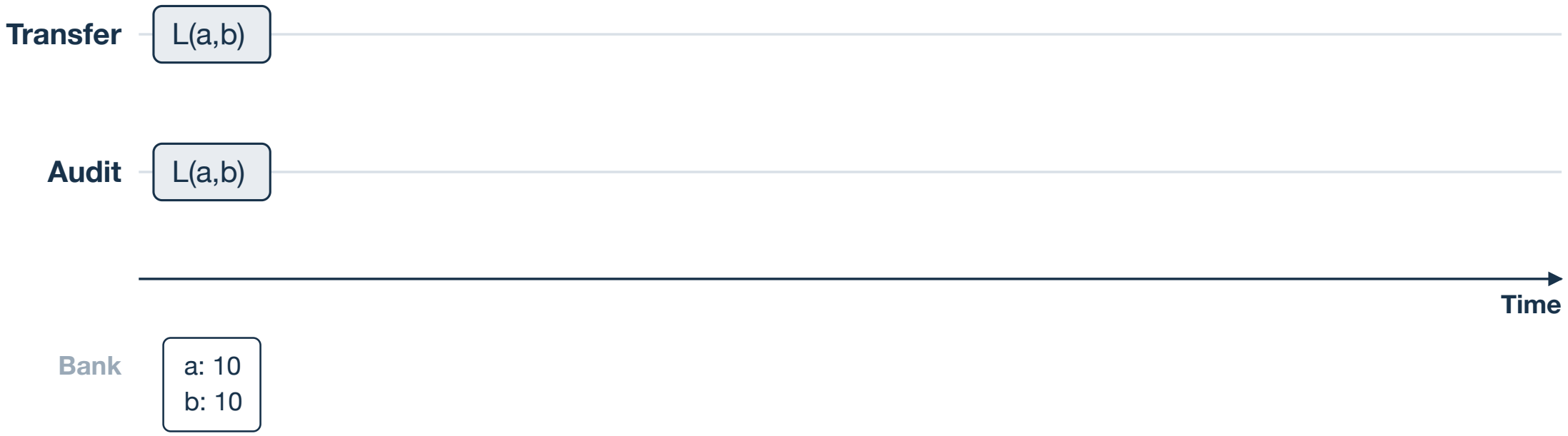
What static locking assumes

- The action knows its whole read/write set before it runs — `Audit` does not, so it locks the whole bank

An example execution of static locking

Transfer holds `lock[a]` and `lock[b]` from before its first read until after its last write

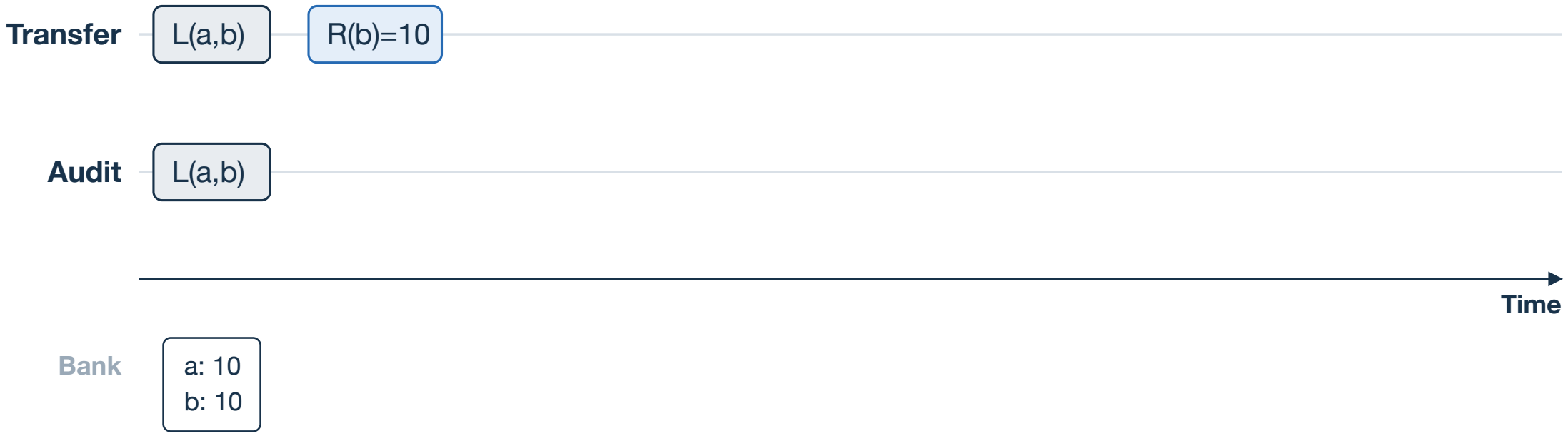
- If Transfer successfully holds the locks, the execution of Audit is deferred until the state is correct



An example execution of static locking

Transfer holds `lock[a]` and `lock[b]` from before its first read until after its last write

- If Transfer successfully holds the locks, the execution of Audit is deferred until the state is correct



An example execution of static locking

Transfer holds `lock[a]` and `lock[b]` from before its first read until after its last write

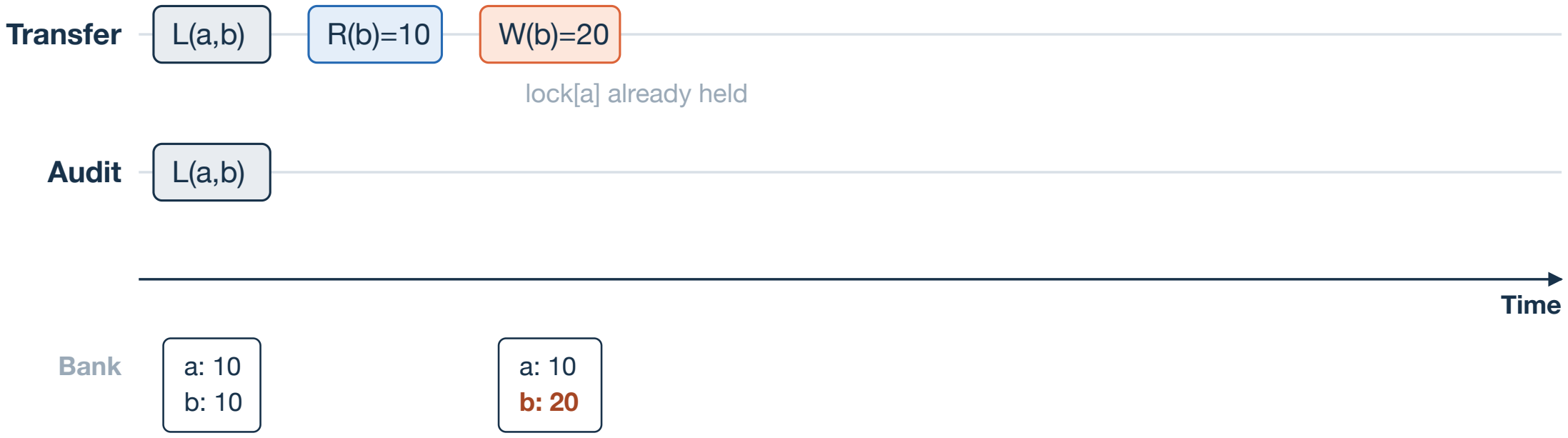
- If Transfer successfully holds the locks, the execution of Audit is deferred until the state is correct



An example execution of static locking

Transfer holds `lock[a]` and `lock[b]` from before its first read until after its last write

- If Transfer successfully holds the locks, the execution of Audit is deferred until the state is correct



An example execution of static locking

Transfer holds `lock[a]` and `lock[b]` from before its first read until after its last write

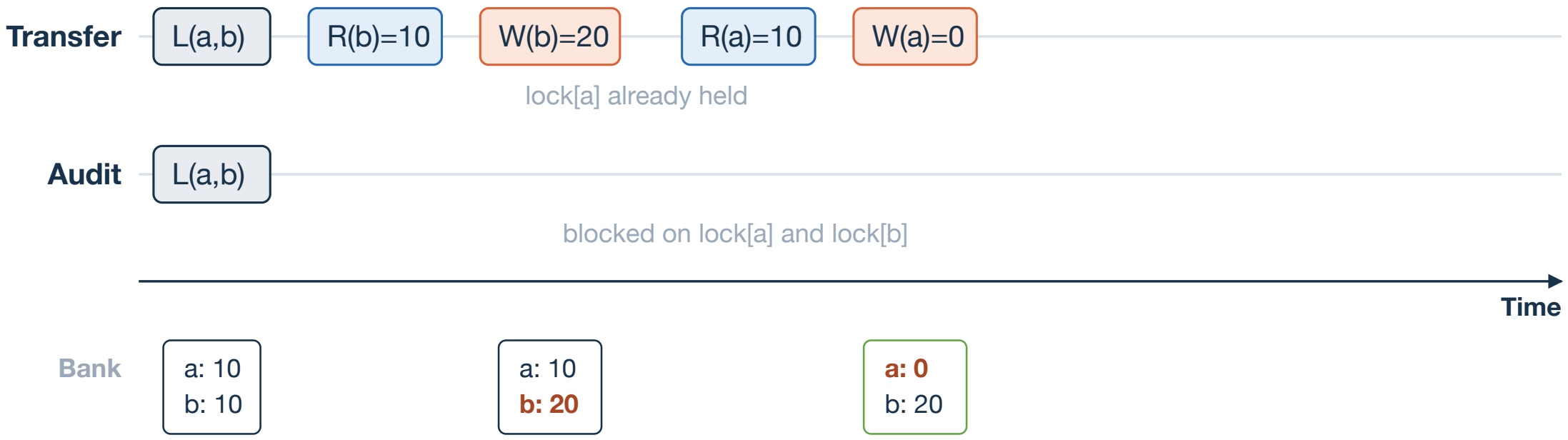
- If Transfer successfully holds the locks, the execution of Audit is deferred until the state is correct



An example execution of static locking

Transfer holds `lock[a]` and `lock[b]` from before its first read until after its last write

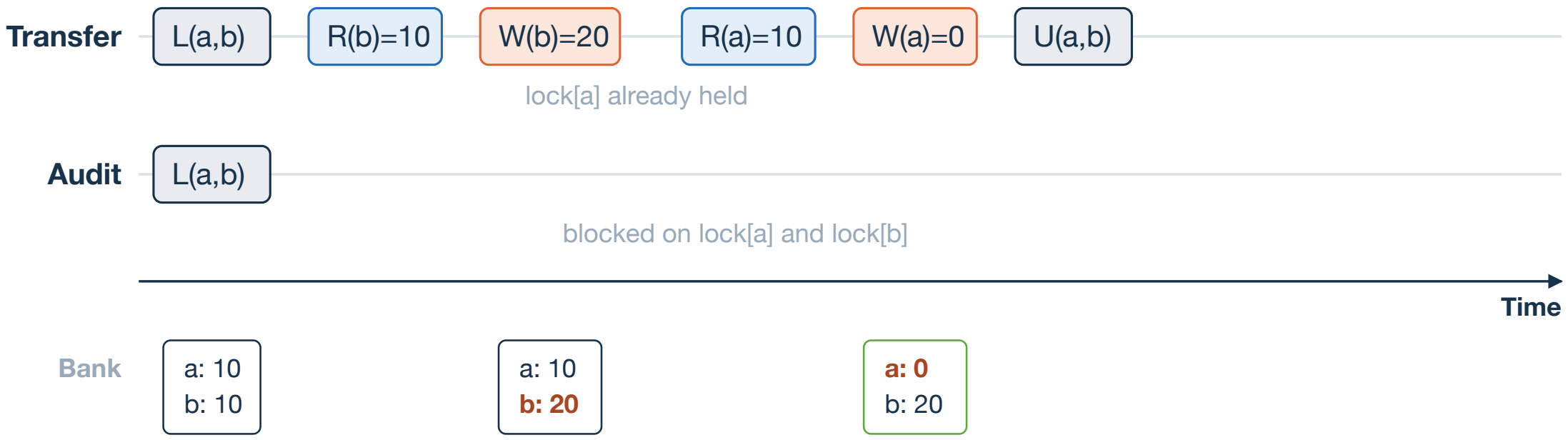
- If Transfer successfully holds the locks, the execution of Audit is deferred until the state is correct



An example execution of static locking

Transfer holds `lock[a]` and `lock[b]` from before its first read until after its last write

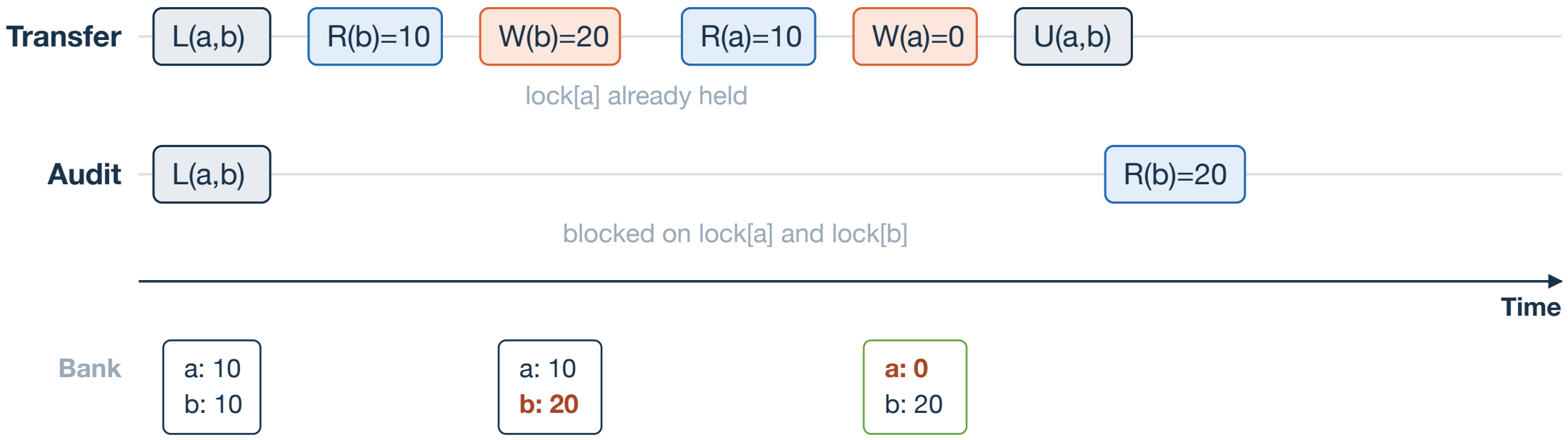
- If Transfer successfully holds the locks, the execution of Audit is deferred until the state is correct



An example execution of static locking

Transfer holds `lock[a]` and `lock[b]` from before its first read until after its last write

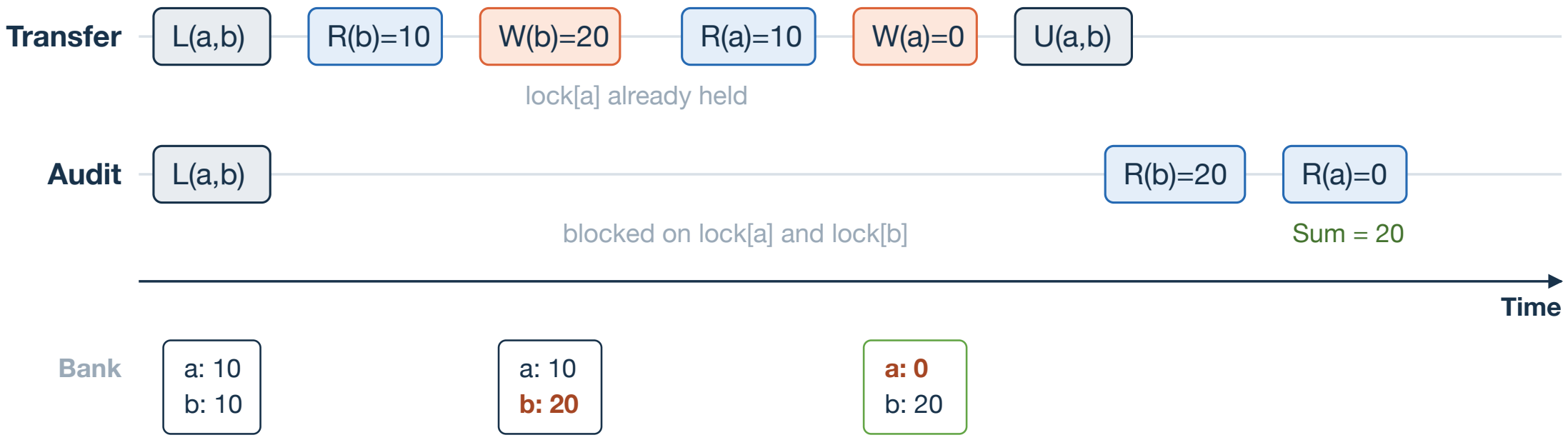
- If Transfer successfully holds the locks, the execution of Audit is deferred until the state is correct



An example execution of static locking

Transfer holds `lock[a]` and `lock[b]` from before its first read until after its last write

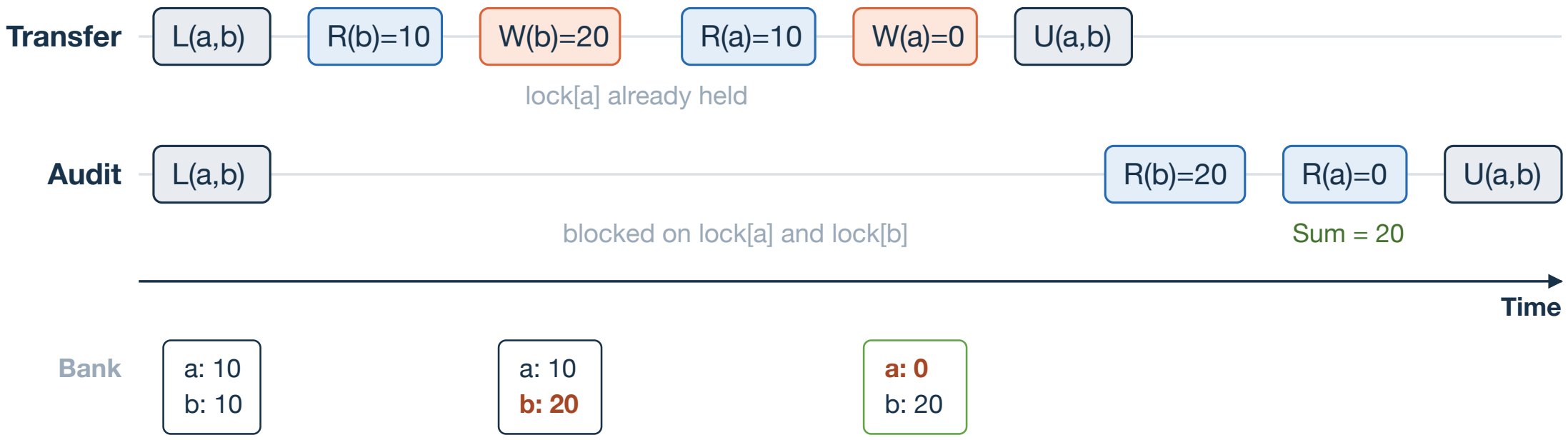
- If Transfer successfully holds the locks, the execution of Audit is deferred until the state is correct



An example execution of static locking

Transfer holds `lock[a]` and `lock[b]` from before its first read until after its last write

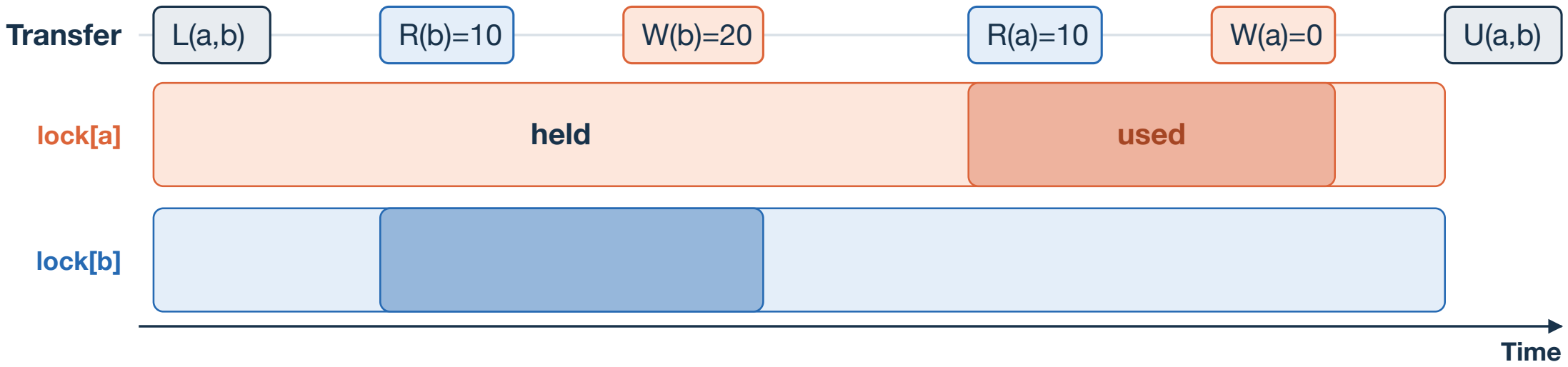
- If Transfer successfully holds the locks, the execution of Audit is deferred until the state is correct



The drawback of static locking: the lock is held far longer than it is used

A lock is held for the whole action, even over the steps that never touch its record

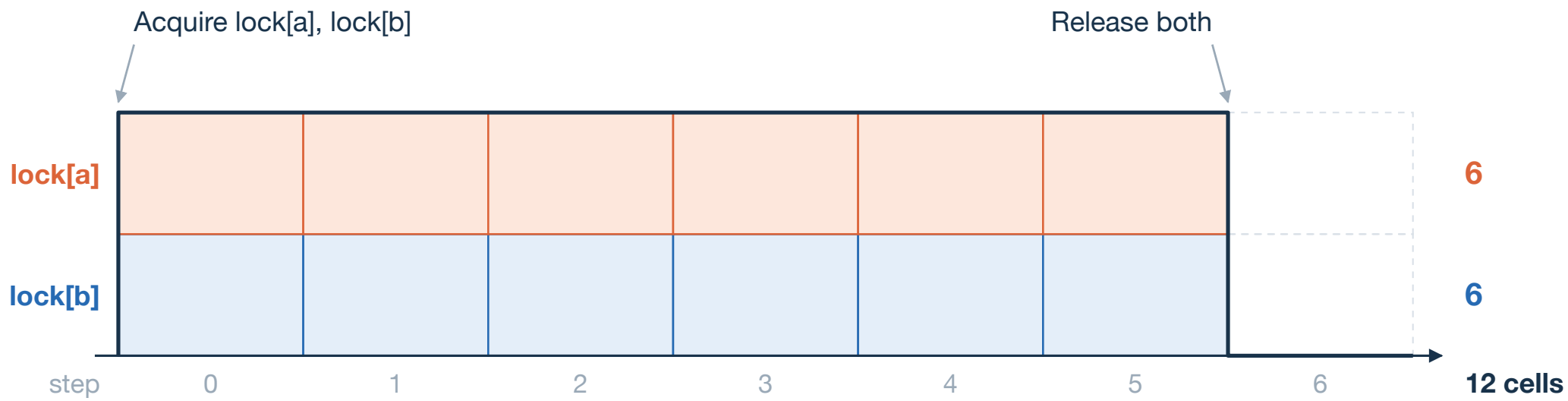
- `lock[a]` is held from the first step, but `a` is not touched until the last step of the action



The drawback of static locking: the lock is held far longer than it is used

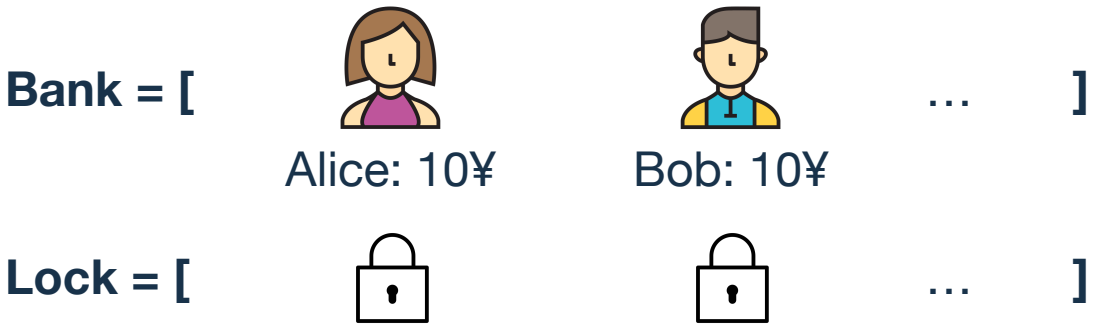
A lock is held for the whole action, even over the steps that never touch its record

- `lock[a]` is held from the first step, but `a` is not touched until the last step of the action



Use locking to achieve before-or-after: two-phase locking (2PL)

The lock granularity does not change: one lock per shared data record



The 2PL locking rule

- The action must acquire a shared data's lock before accessing it
- Once any lock has been released, no new lock can be acquired

Static locking vs 2PL

The difference is when the locks are acquired and released



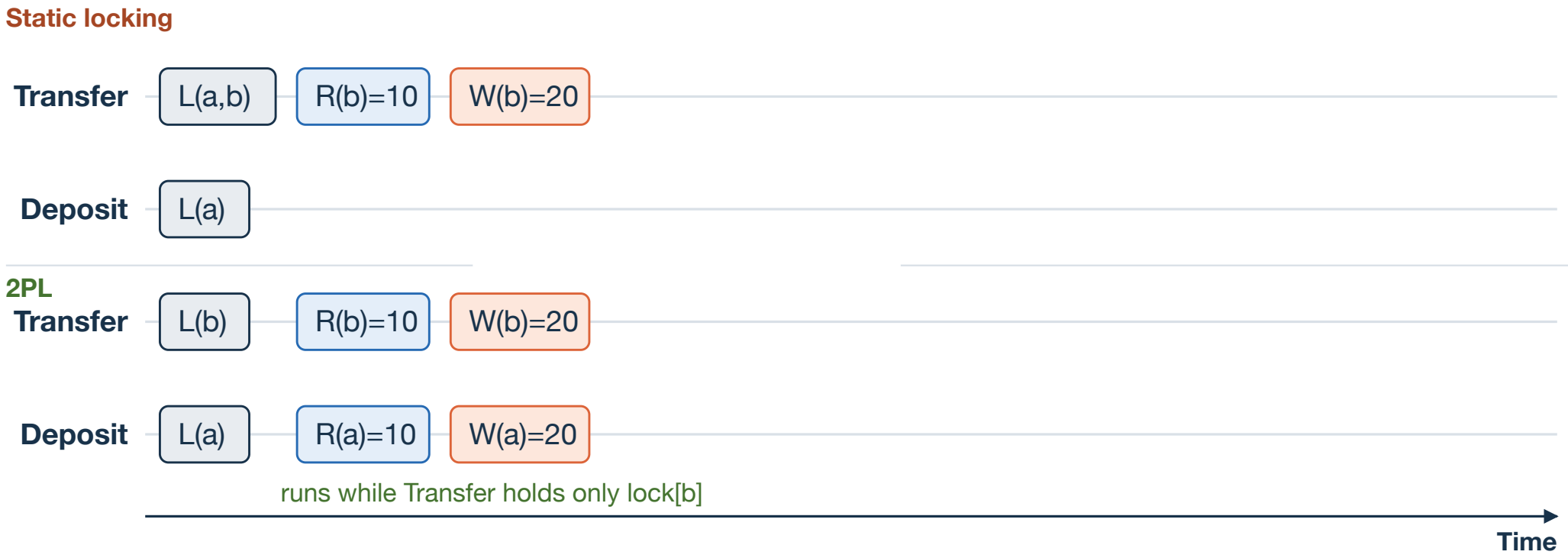
Static locking vs 2PL

The difference is when the locks are acquired and released



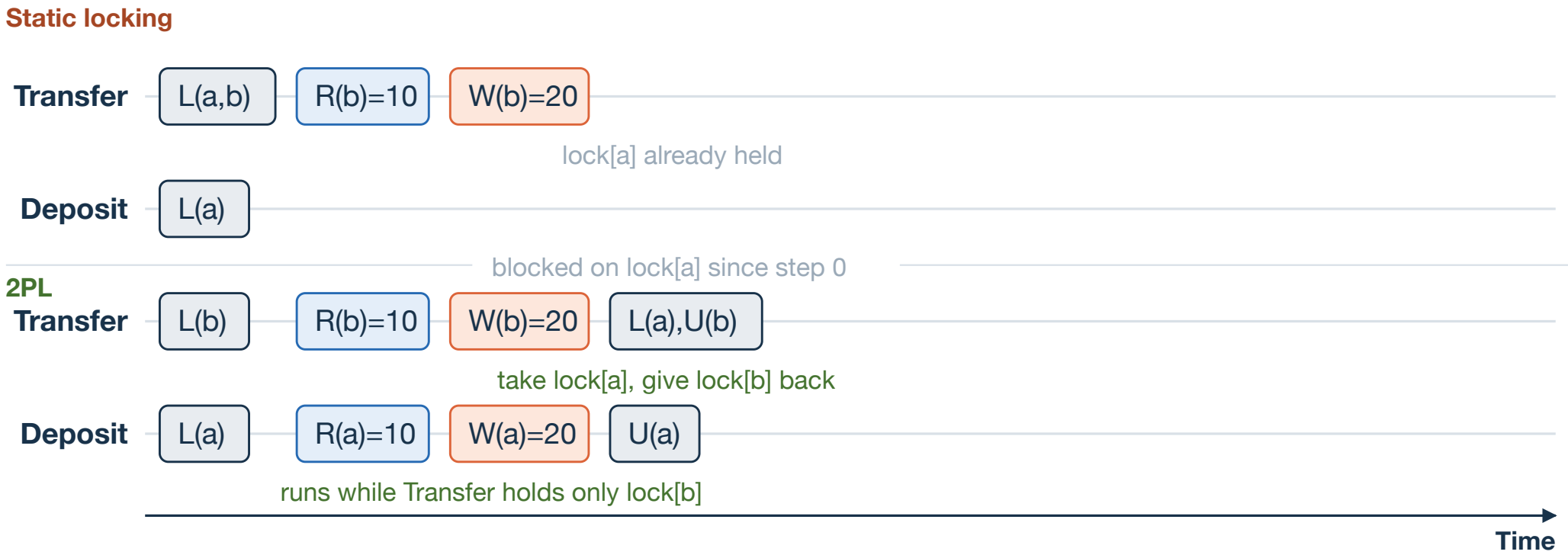
Static locking vs 2PL

The difference is when the locks are acquired and released



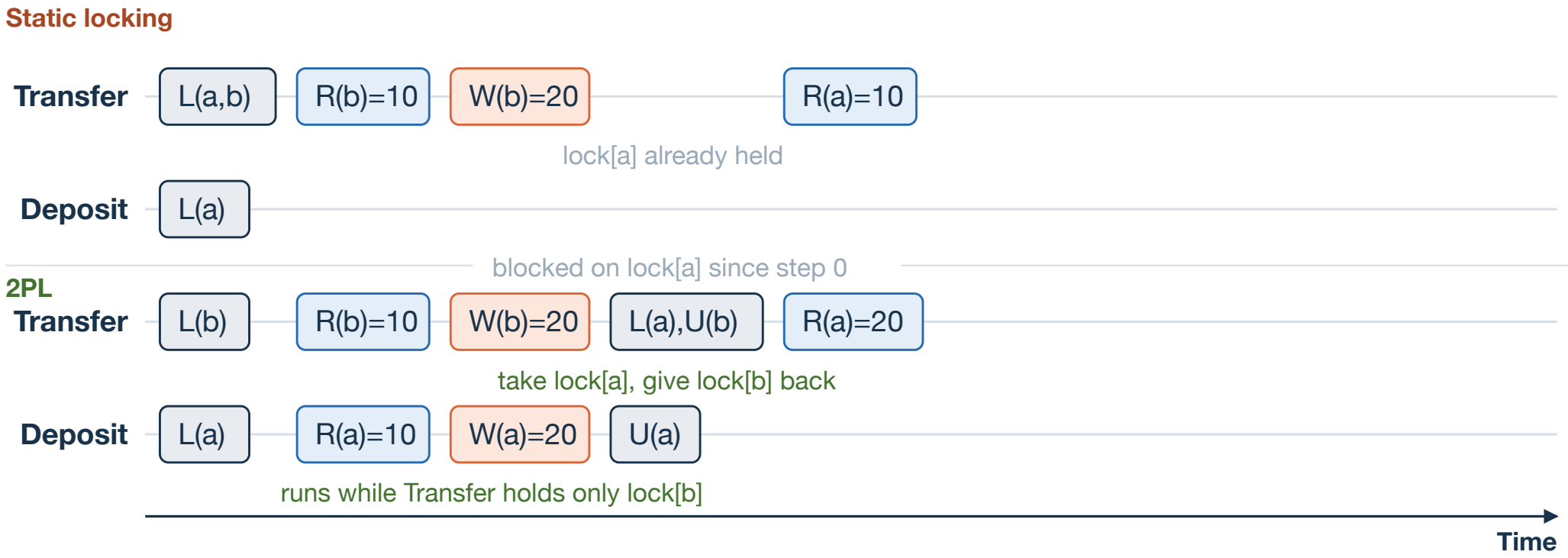
Static locking vs 2PL

The difference is when the locks are acquired and released



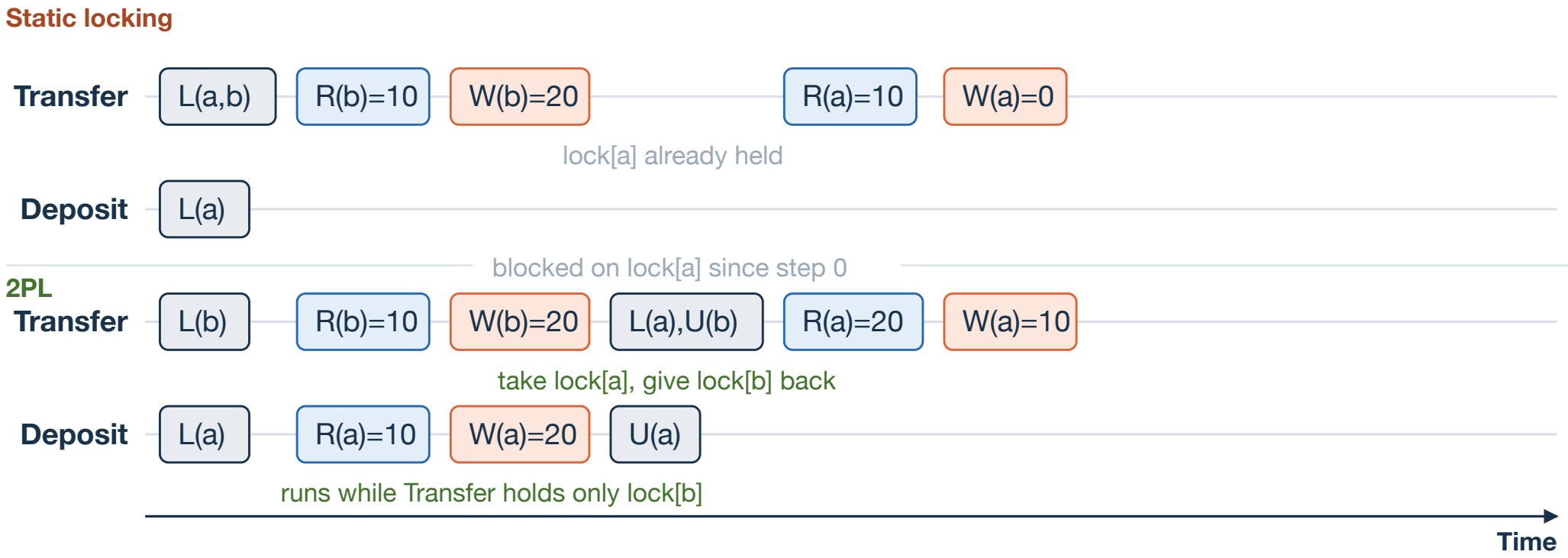
Static locking vs 2PL

The difference is when the locks are acquired and released



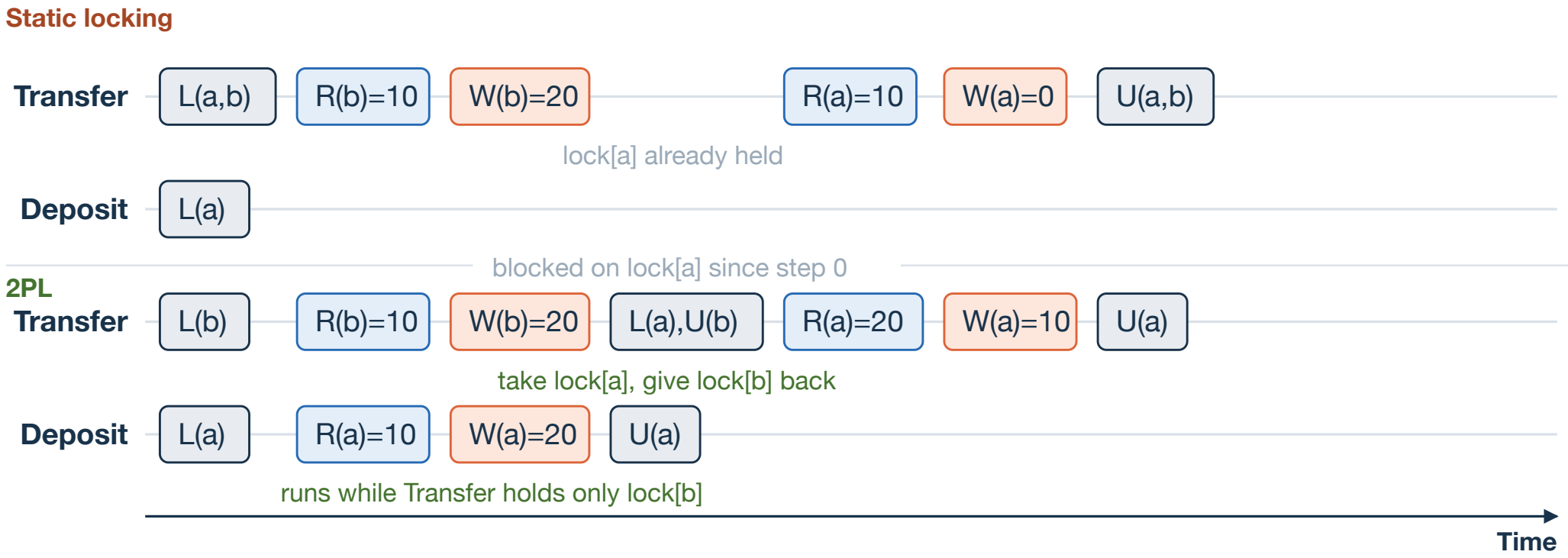
Static locking vs 2PL

The difference is when the locks are acquired and released



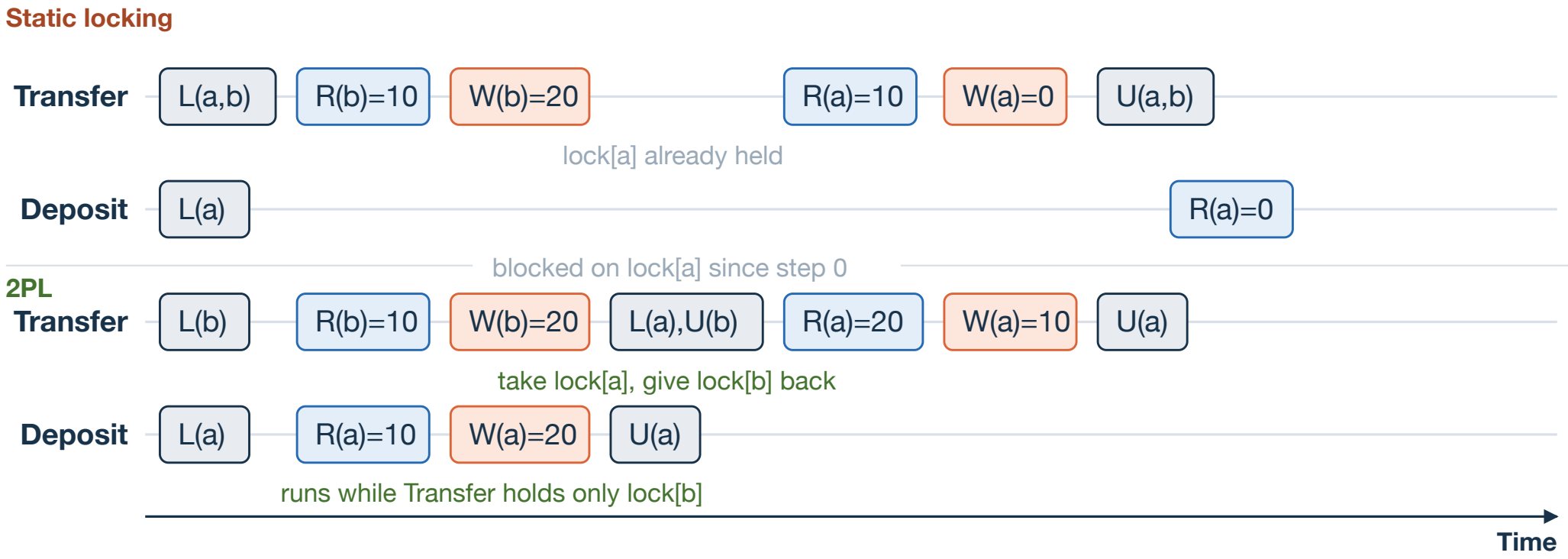
Static locking vs 2PL

The difference is when the locks are acquired and released



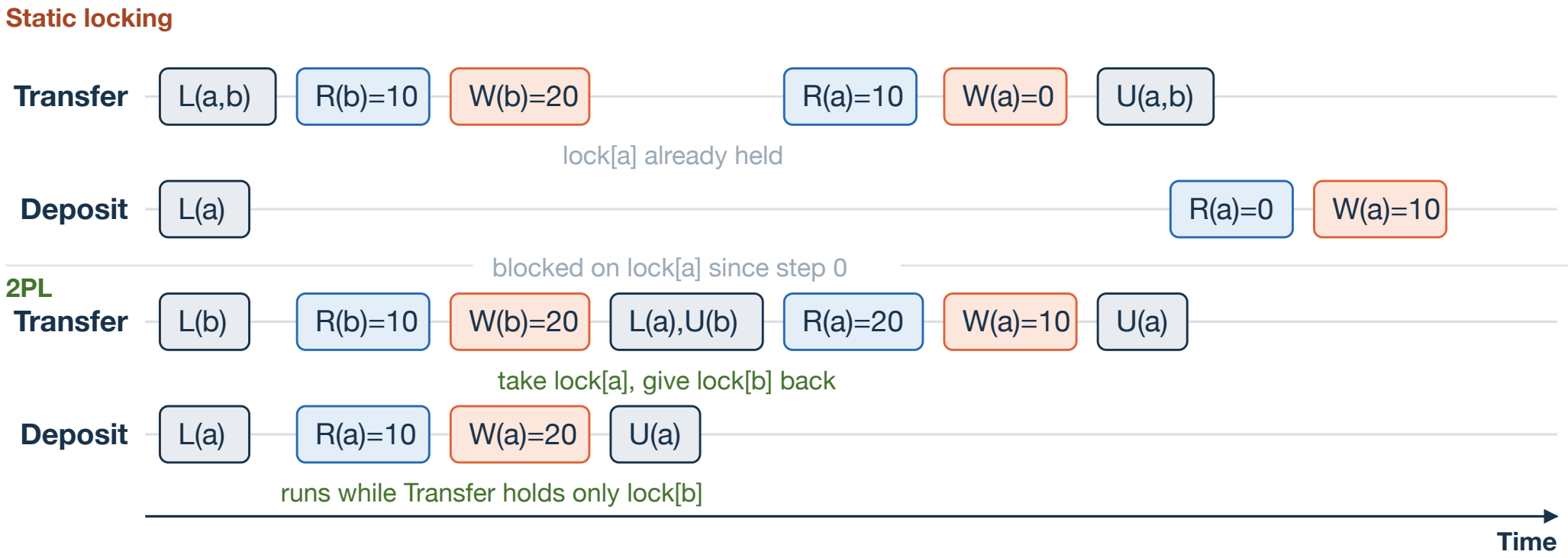
Static locking vs 2PL

The difference is when the locks are acquired and released



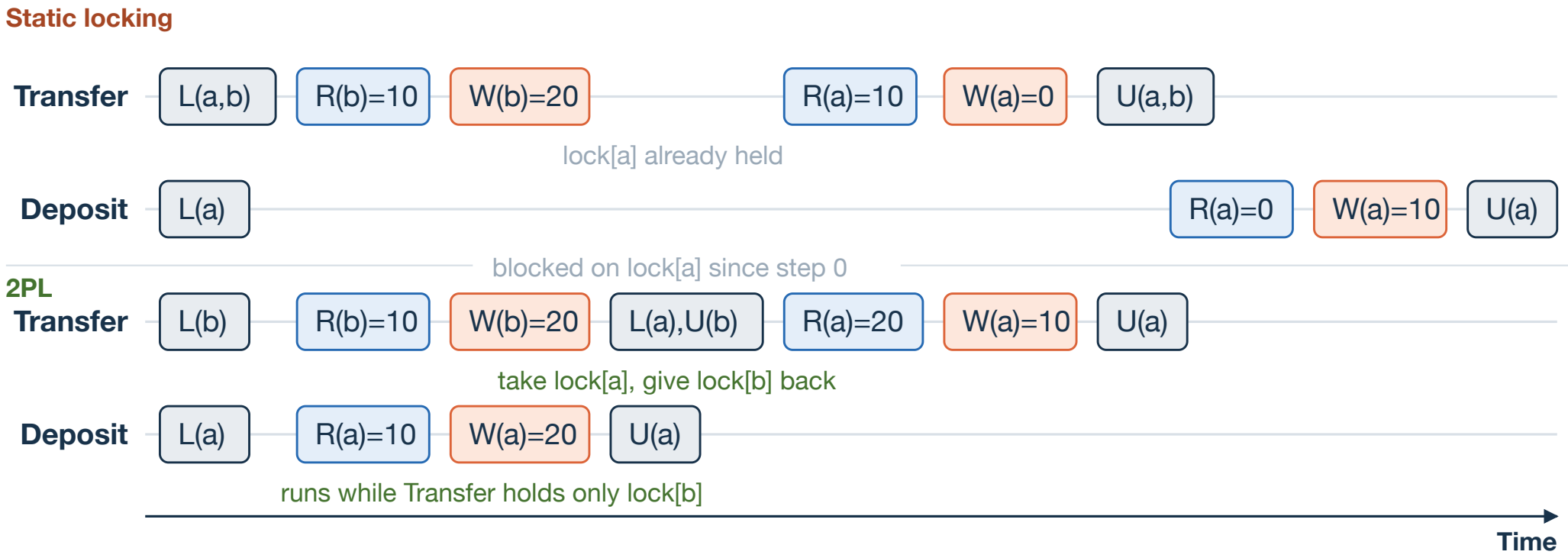
Static locking vs 2PL

The difference is when the locks are acquired and released



Static locking vs 2PL

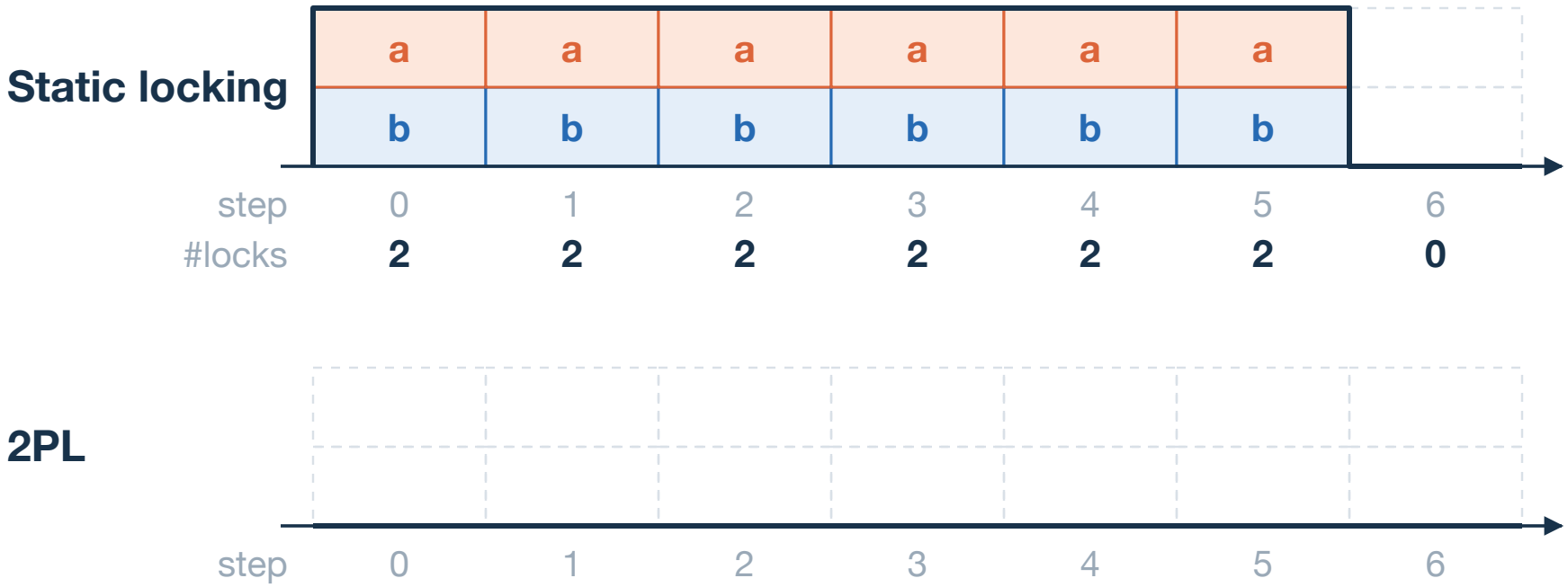
The difference is when the locks are acquired and released



Two phases of 2PL: lock acquisition & release

2PL significantly reduces the lock held area:

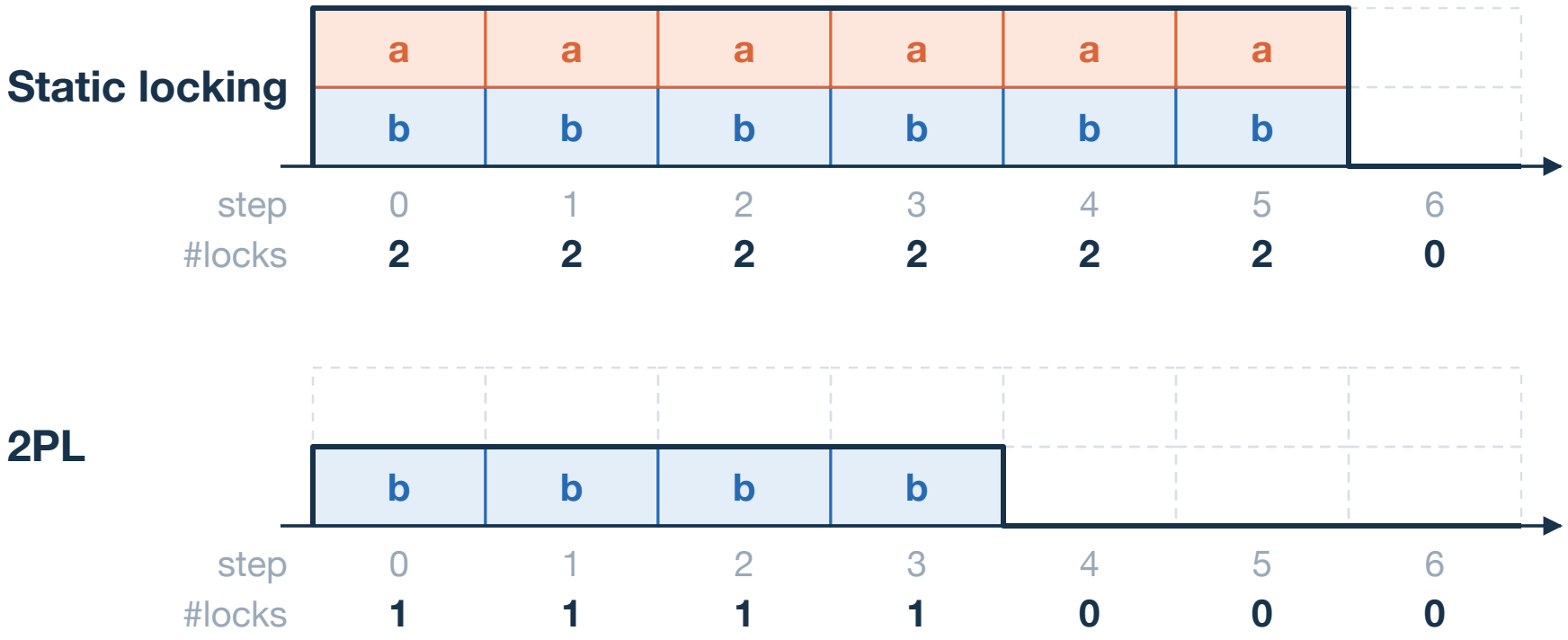
- While an action still acquires, the #locks it holds increases
- Once it releases the first lock, the #locks it holds only decreases



Two phases of 2PL: lock acquisition & release

2PL significantly reduces the lock held area:

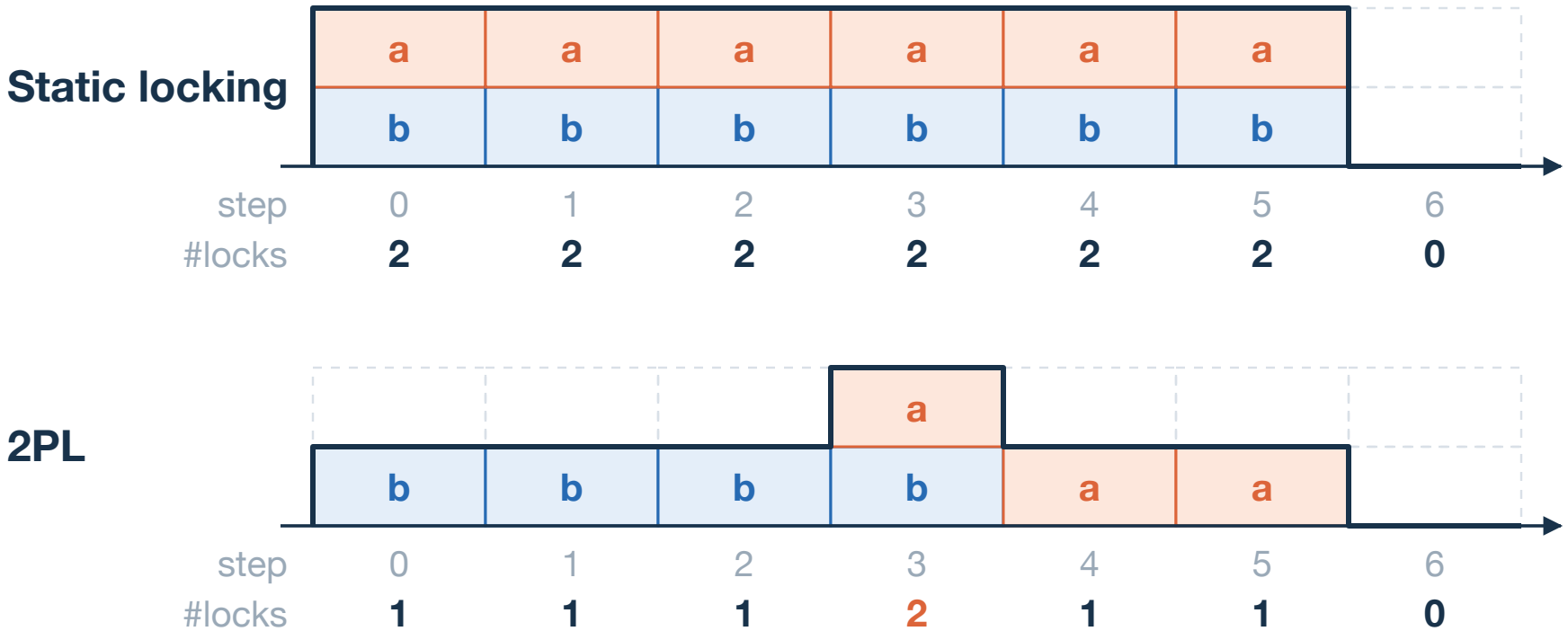
- While an action still acquires, the #locks it holds increases
- Once it releases the first lock, the #locks it holds only decreases



Two phases of 2PL: lock acquisition & release

2PL significantly reduces the lock held area:

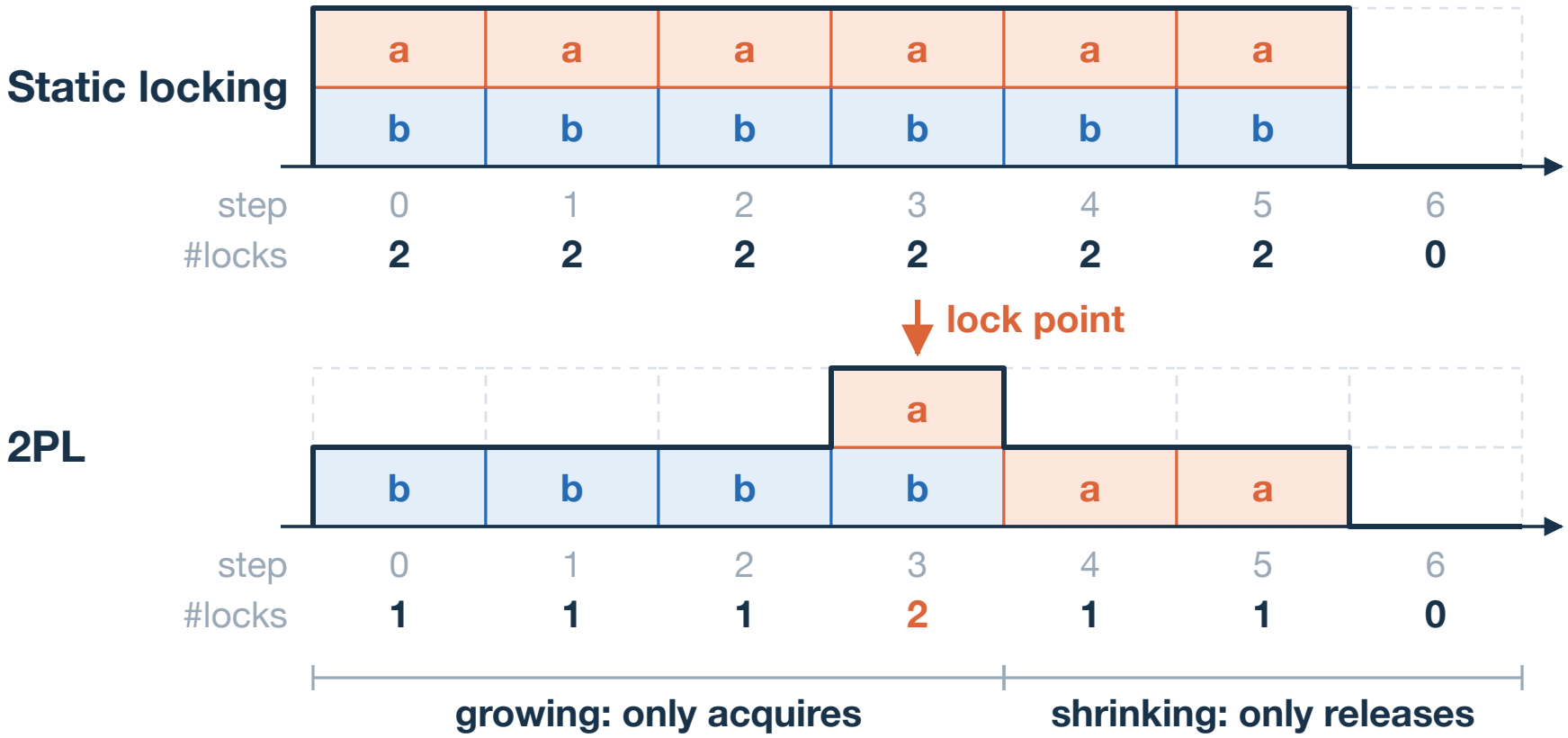
- While an action still acquires, the #locks it holds increases
- Once it releases the first lock, the #locks it holds only decreases



Two phases of 2PL: lock acquisition & release

2PL significantly reduces the lock held area:

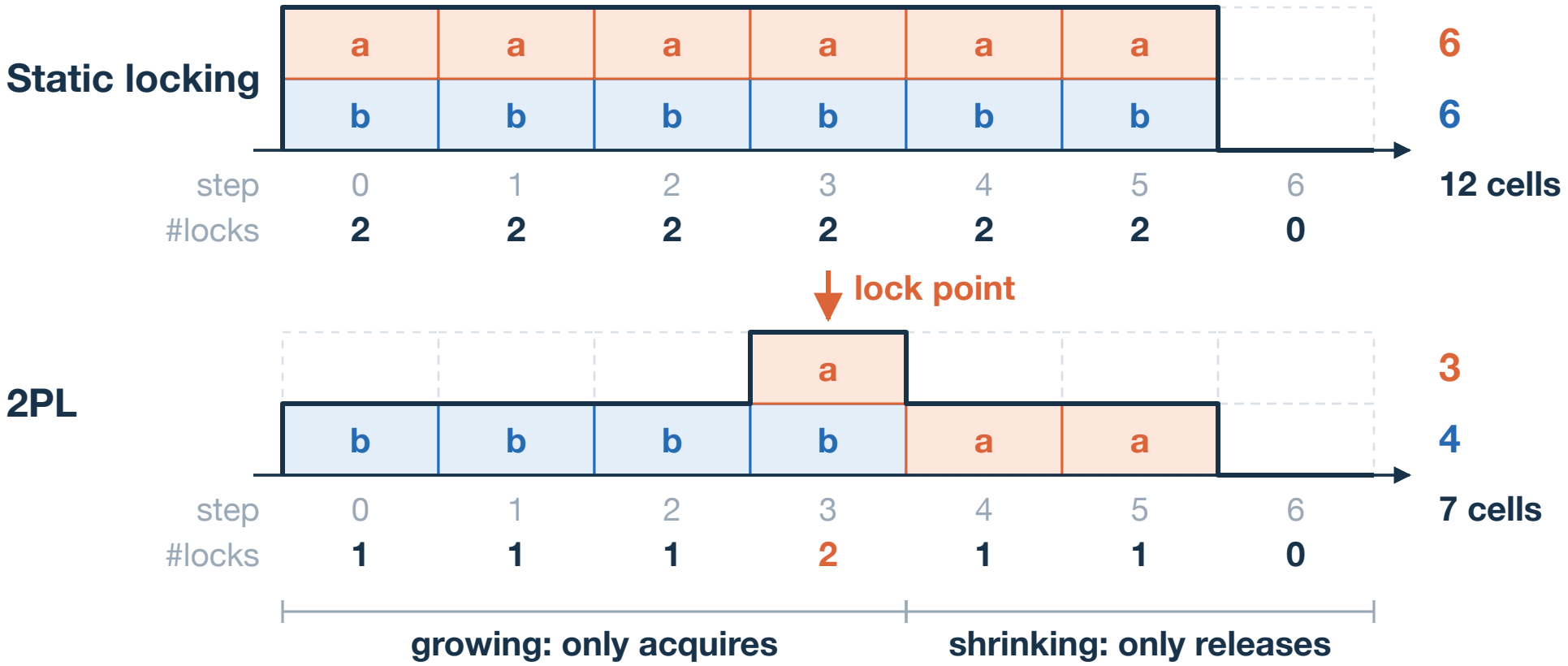
- While an action still acquires, the #locks it holds increases
- Once it releases the first lock, the #locks it holds only decreases



Two phases of 2PL: lock acquisition & release

2PL significantly reduces the lock held area:

- While an action still acquires, the #locks it holds increases
- Once it releases the first lock, the #locks it holds only decreases



Why 2PL is correct? It gives us serializability

2PL guarantees before-or-after atomicity

- Run actions T1, T2, ..., TN concurrently, and have it "appear" as if they ran sequentially

This property is often used interchangeably with serializability

- But these are actually different things

To understand why, we need to first understand what "appear" to run sequentially means

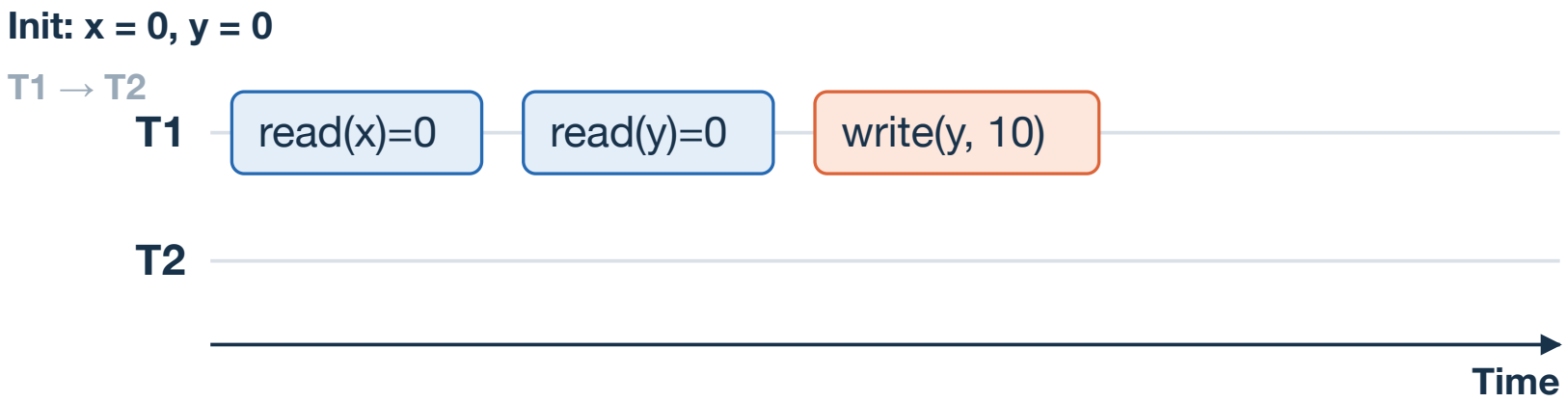
The "appear" typically means the schedule is similar

A schedule is the order in which the operations of the transactions actually occur

```
T1                          T2
begin                       begin
read(x)                     write(x, 20)
tmp = read(y)               write(y, 30)
write(y, tmp+10)            commit
commit
```

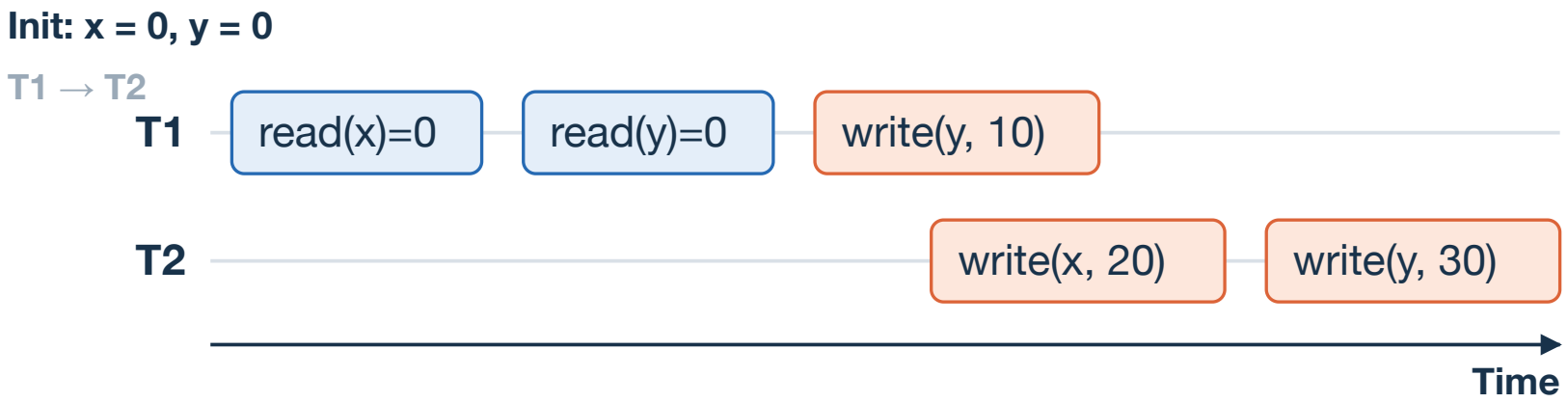
Serial schedule: run the actions sequentially

In our previous example, there are only two sequential schedules



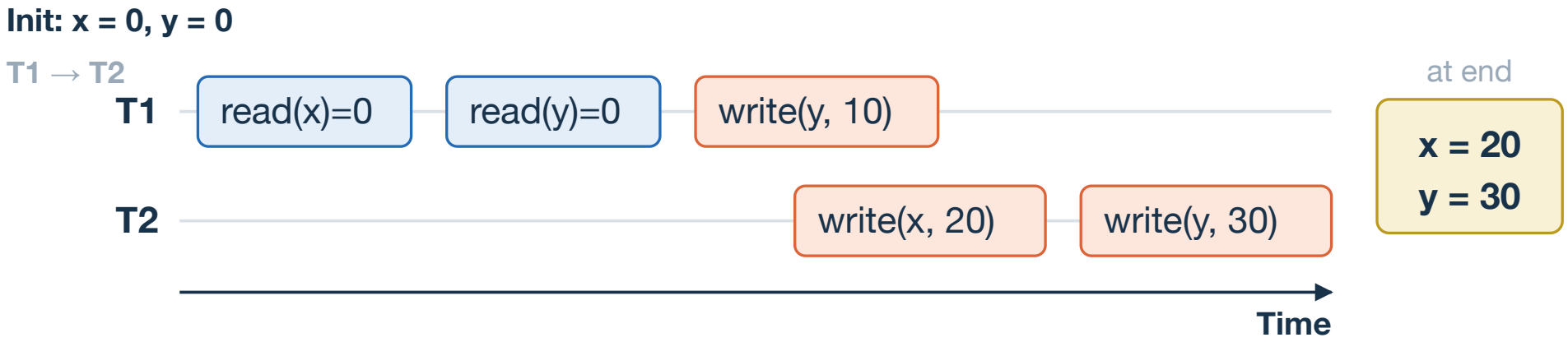
Serial schedule: run the actions sequentially

In our previous example, there are only two sequential schedules



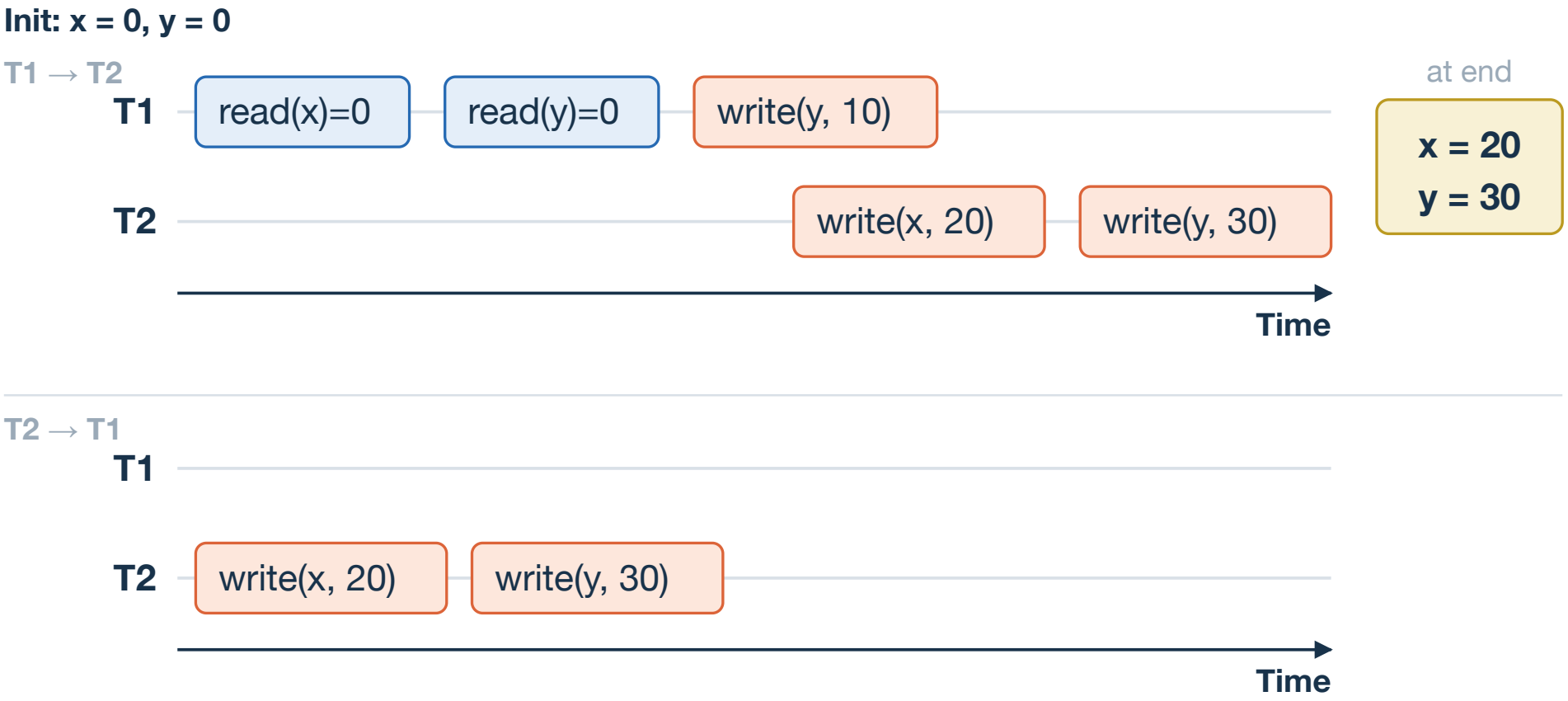
Serial schedule: run the actions sequentially

In our previous example, there are only two sequential schedules



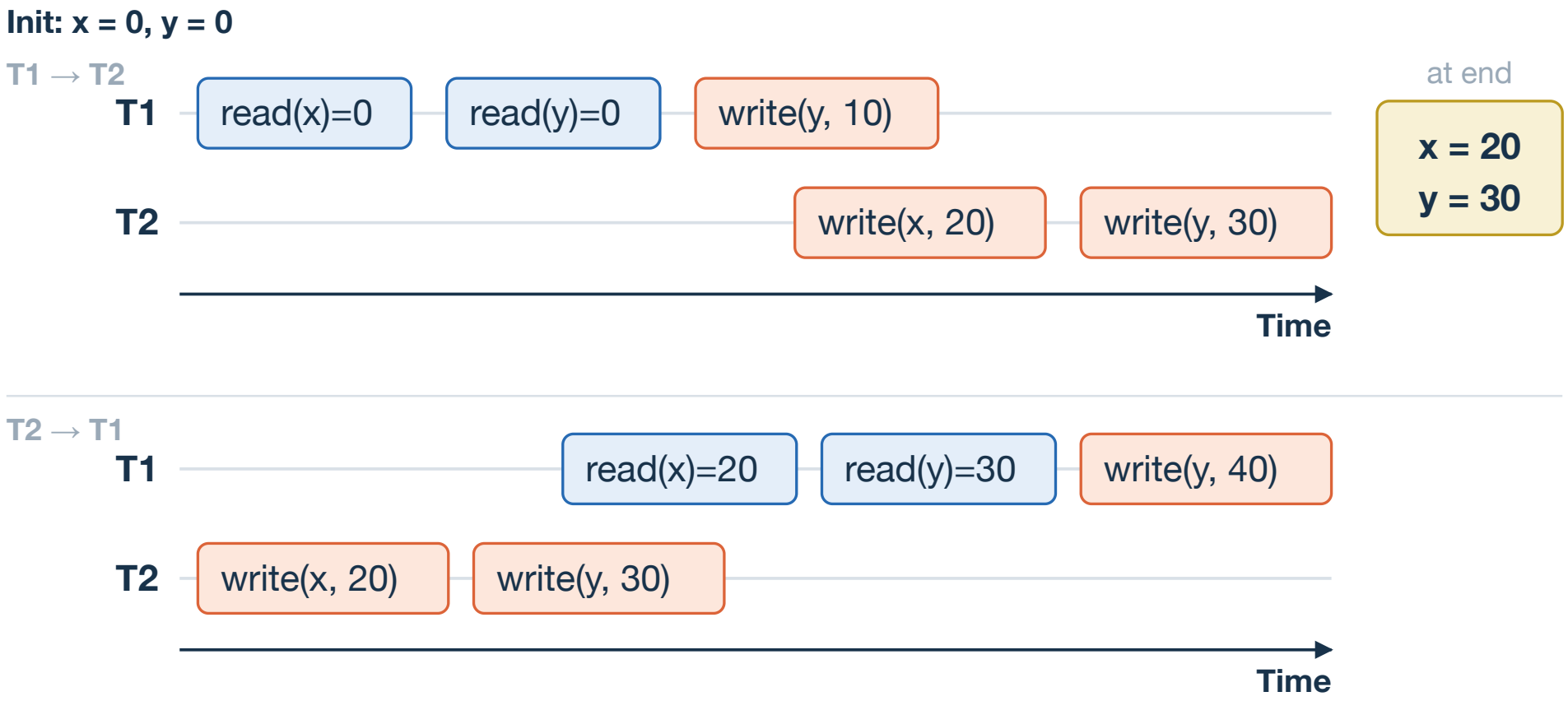
Serial schedule: run the actions sequentially

In our previous example, there are only two sequential schedules



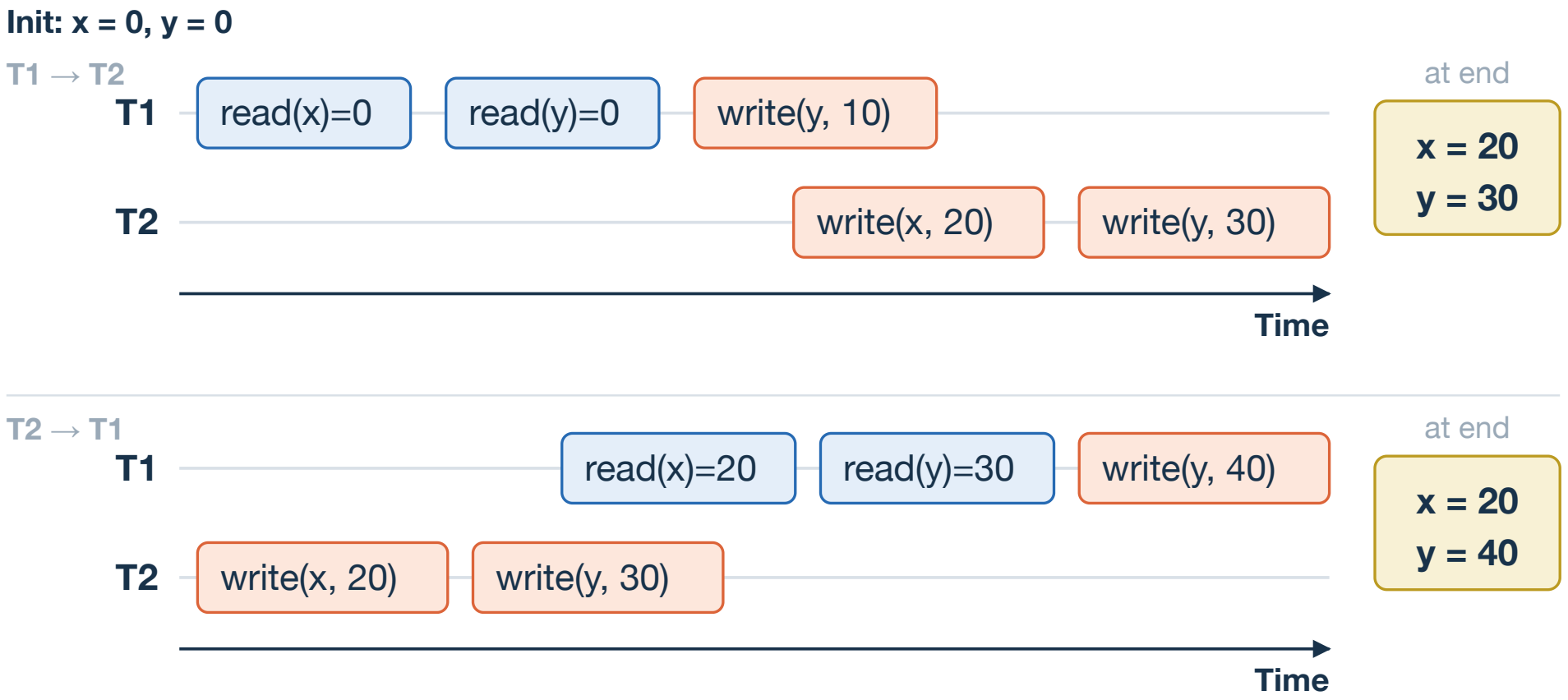
Serial schedule: run the actions sequentially

In our previous example, there are only two sequential schedules



Serial schedule: run the actions sequentially

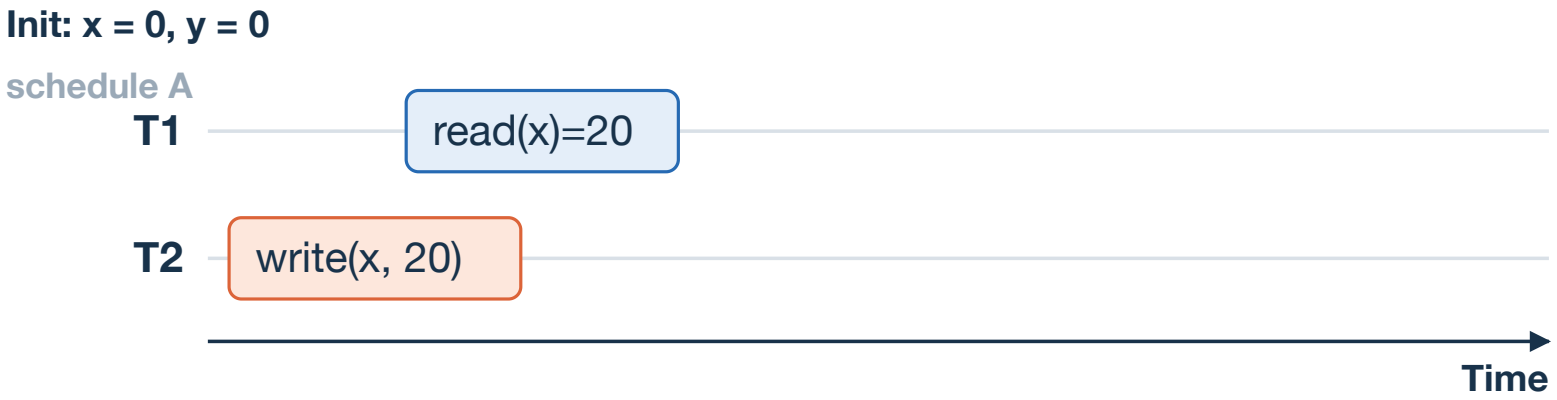
In our previous example, there are only two sequential schedules



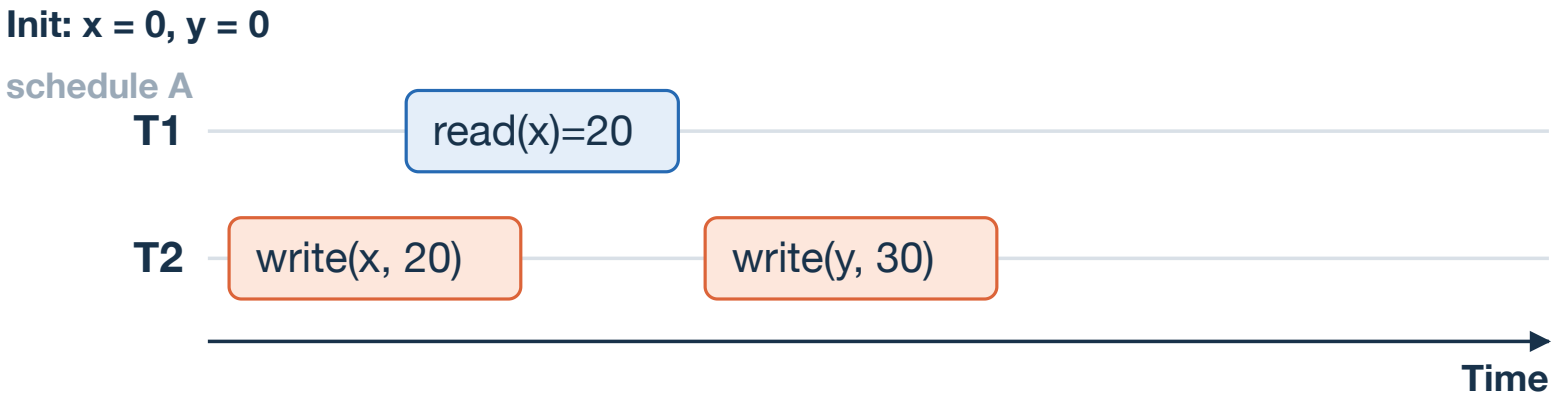
Question: what if run concurrently (concurrent schedules) appears to be the same as a serial one?



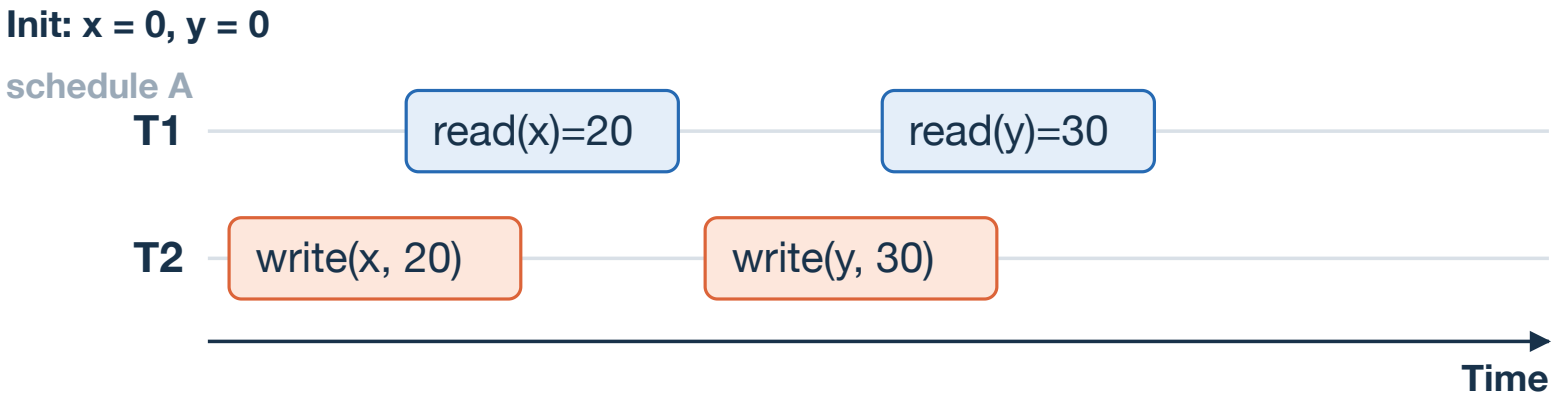
Question: what if run concurrently (concurrent schedules) appears to be the same as a serial one?



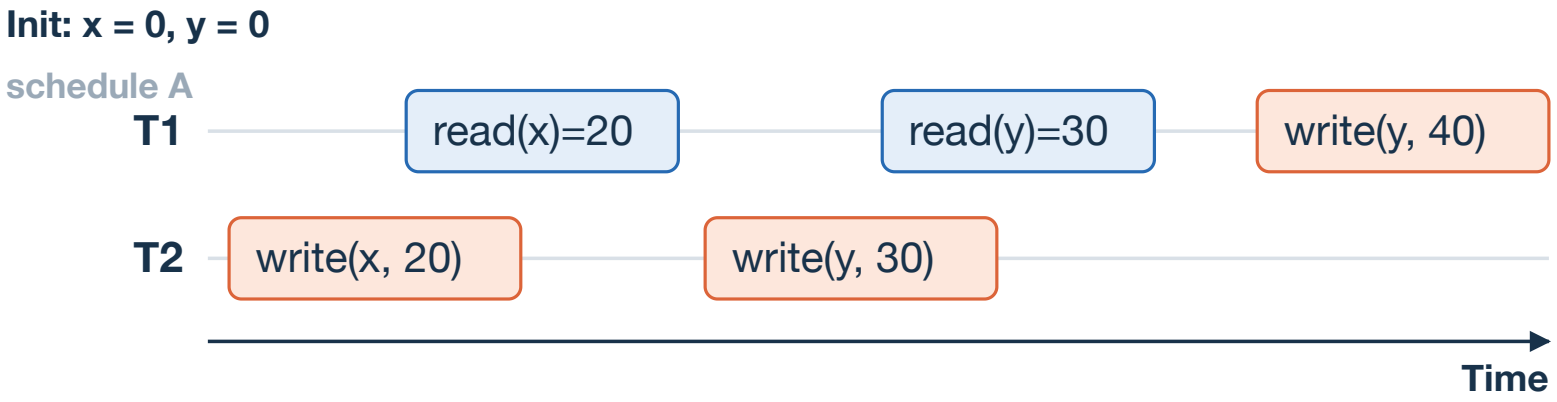
Question: what if run concurrently (concurrent schedules) appears to be the same as a serial one?



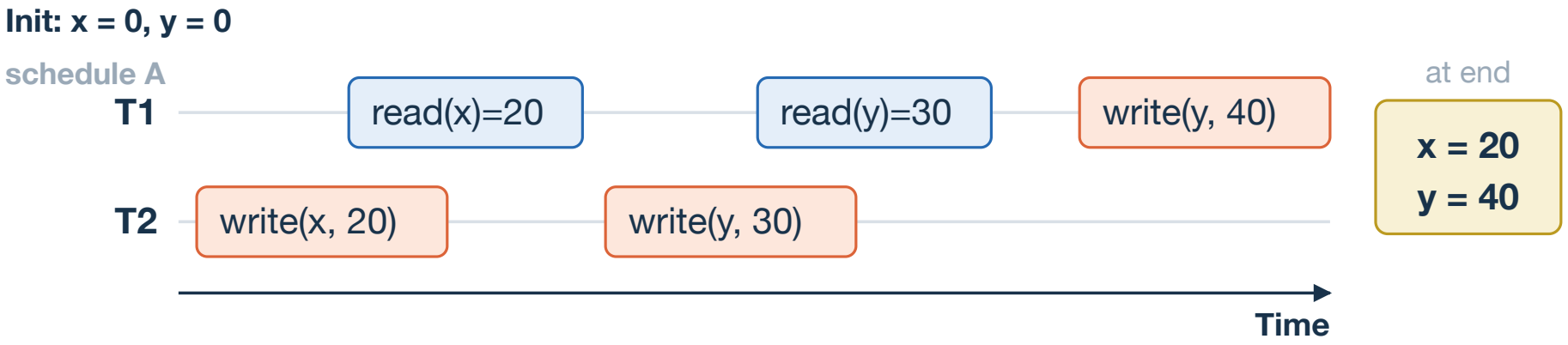
Question: what if run concurrently (concurrent schedules) appears to be the same as a serial one?



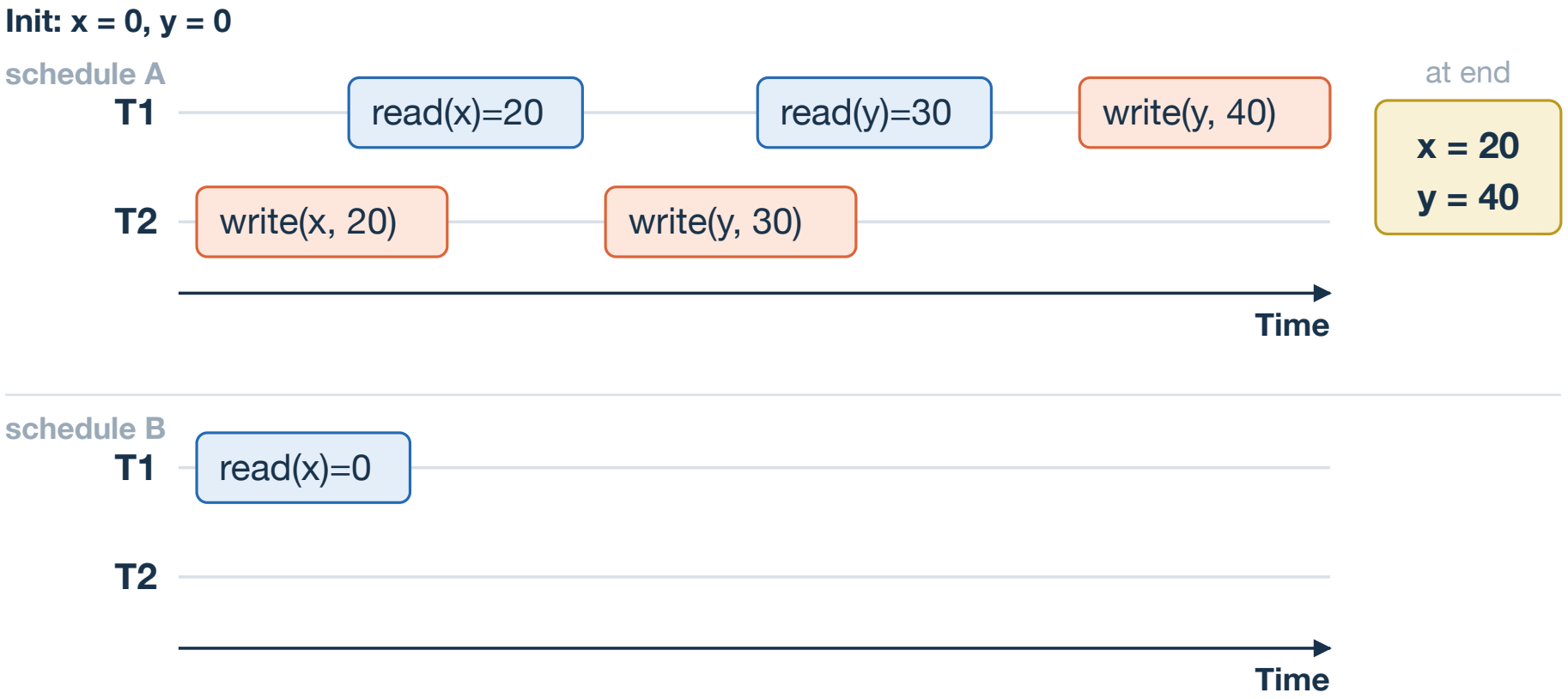
Question: what if run concurrently (concurrent schedules) appears to be the same as a serial one?



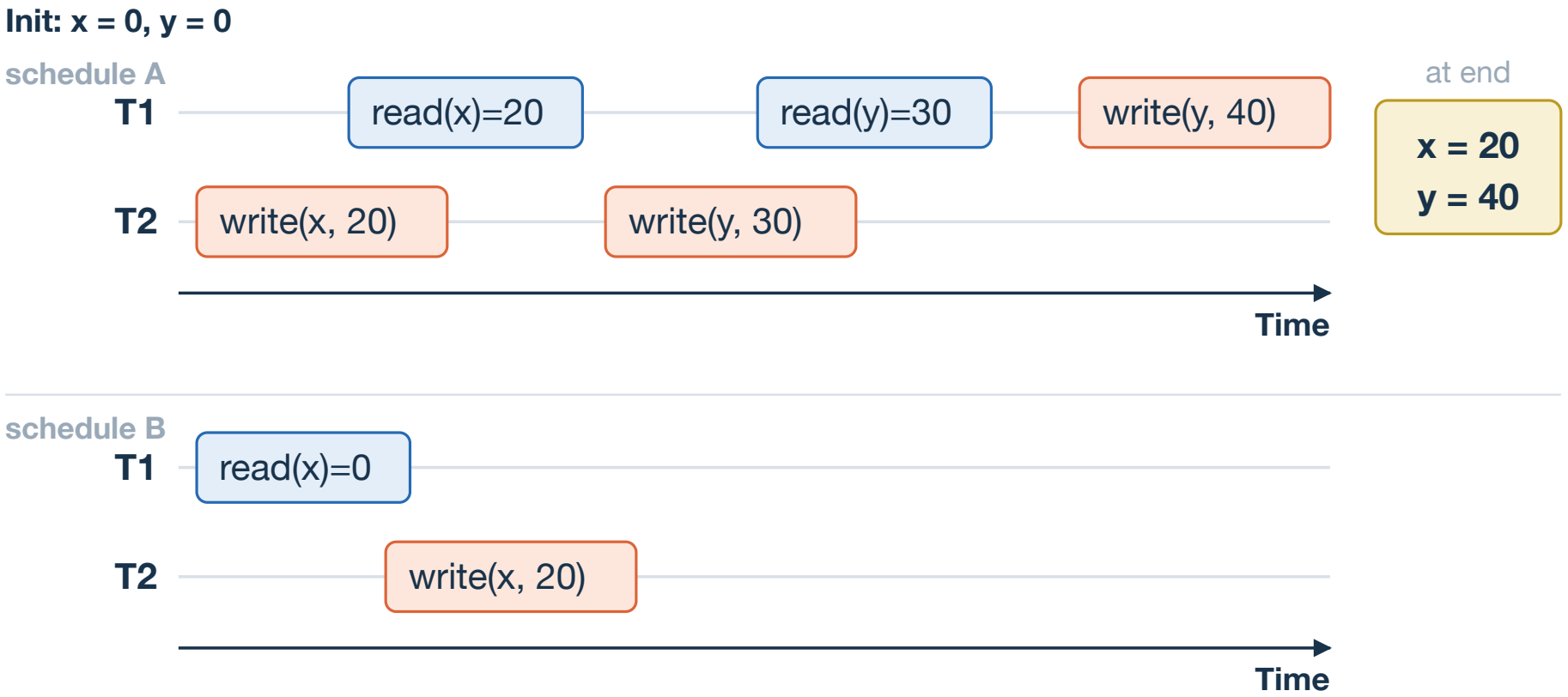
Question: what if run concurrently (concurrent schedules) appears to be the same as a serial one?



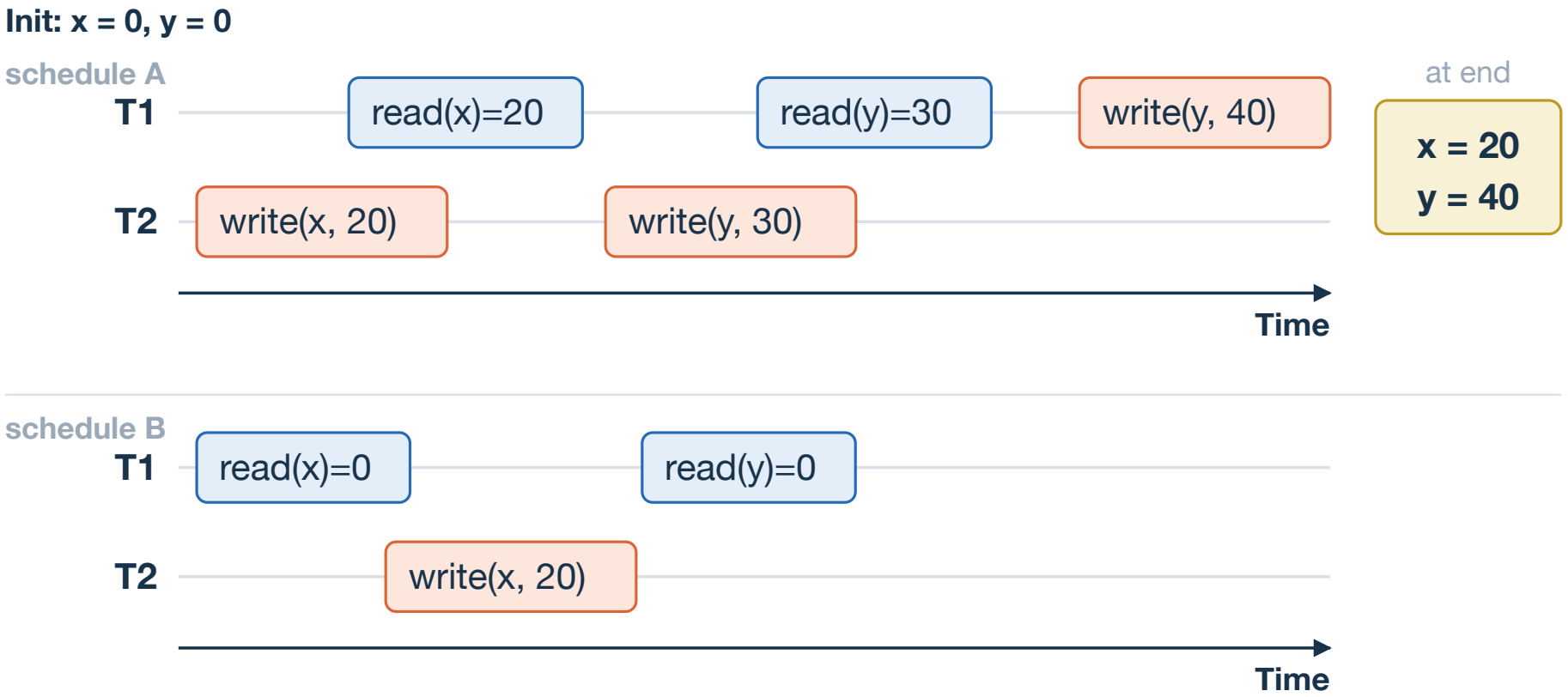
Question: what if run concurrently (concurrent schedules) appears to be the same as a serial one?



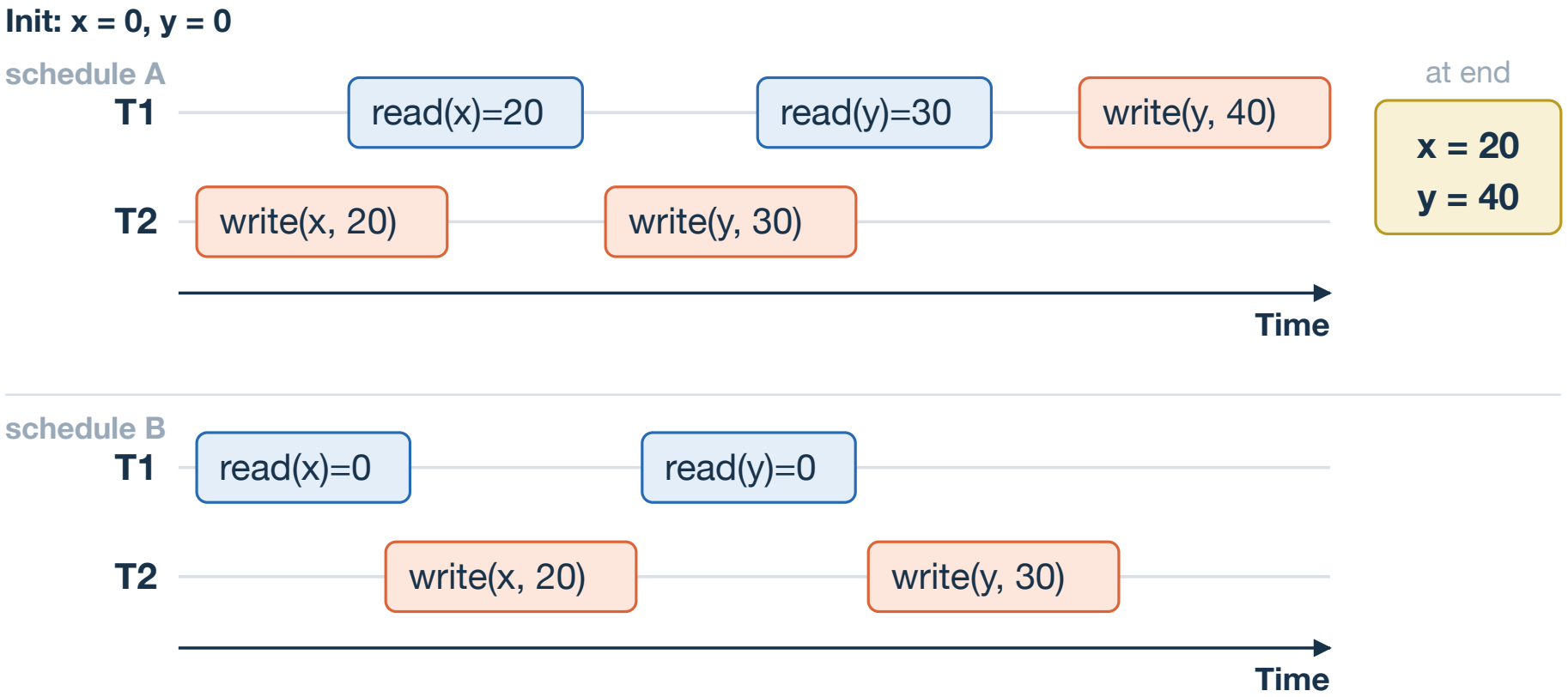
Question: what if run concurrently (concurrent schedules) appears to be the same as a serial one?



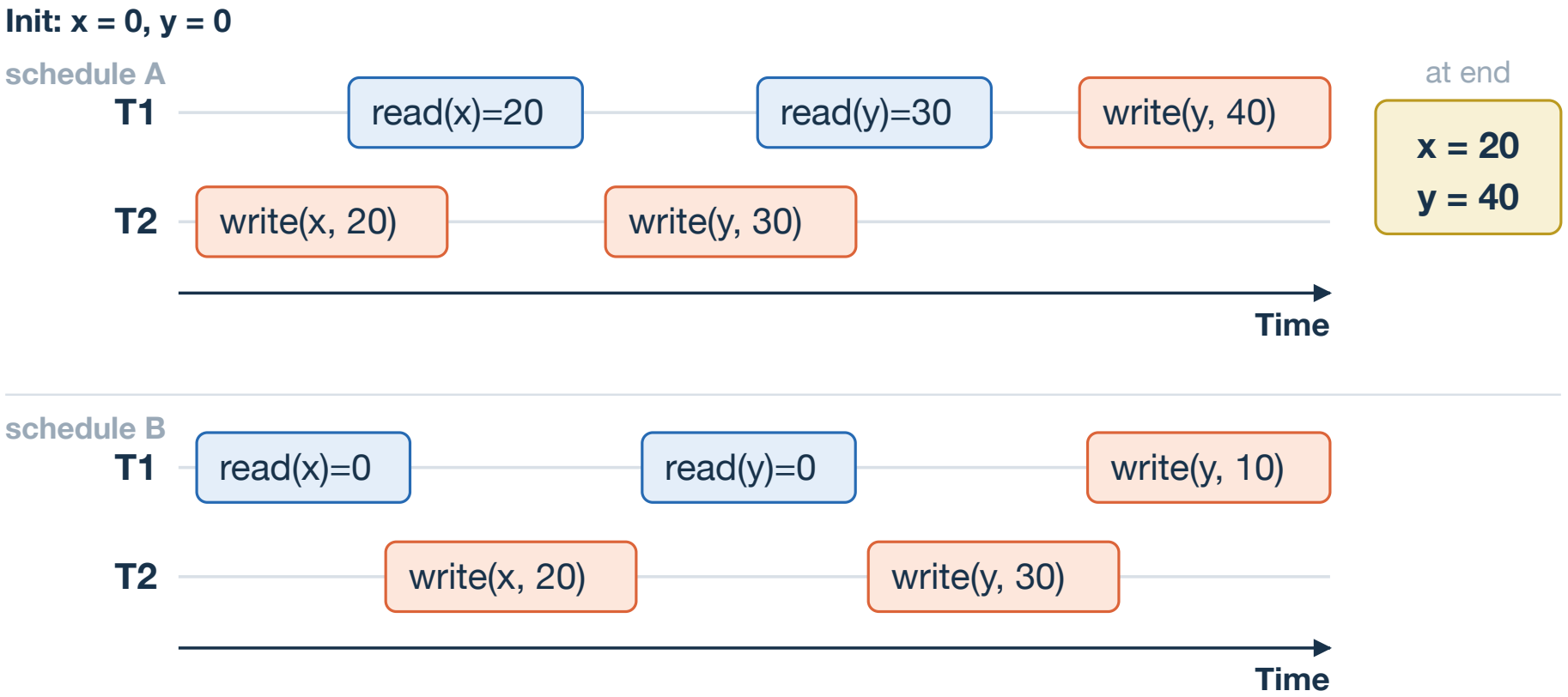
Question: what if run concurrently (concurrent schedules) appears to be the same as a serial one?



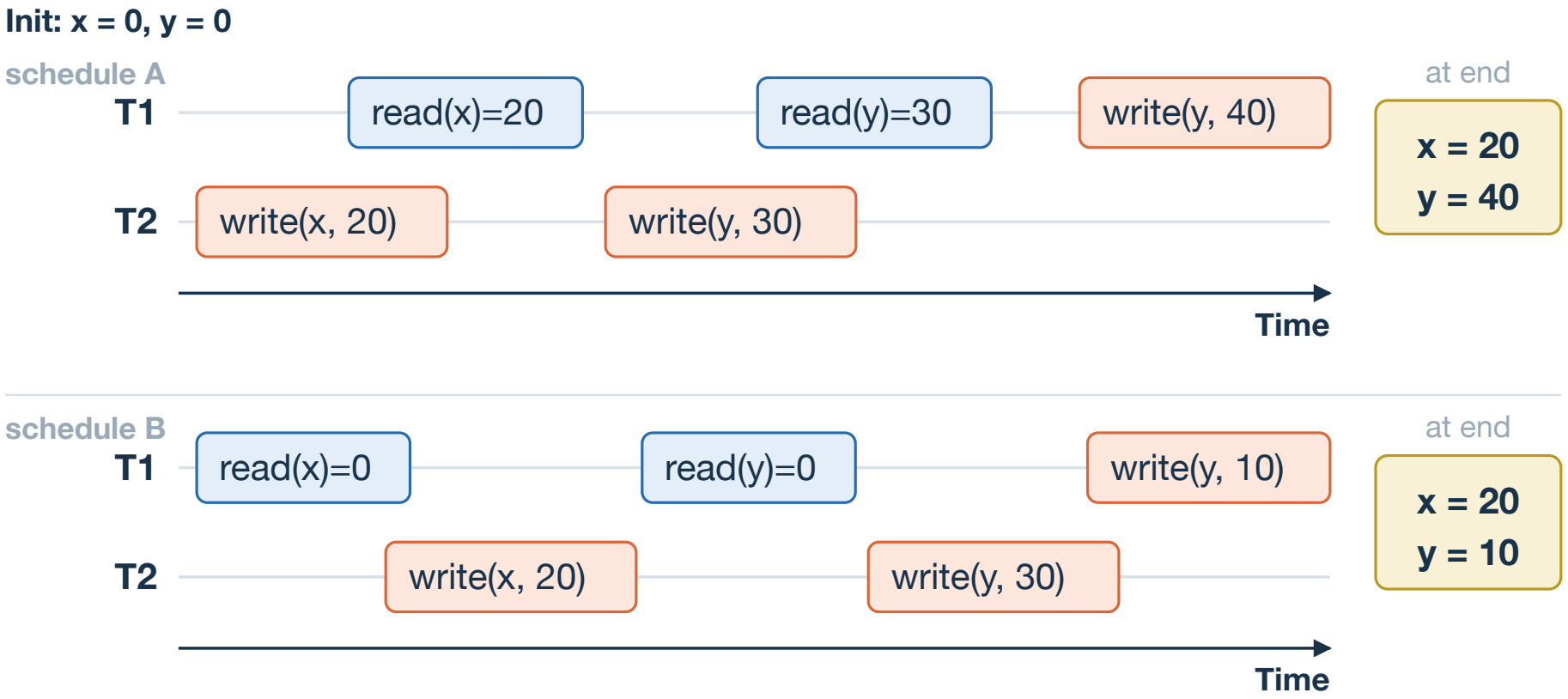
Question: what if run concurrently (concurrent schedules) appears to be the same as a serial one?



Question: what if run concurrently (concurrent schedules) appears to be the same as a serial one?



Question: what if run concurrently (concurrent schedules) appears to be the same as a serial one?



"Appear" means we cannot tell a concurrent execution from a serial one

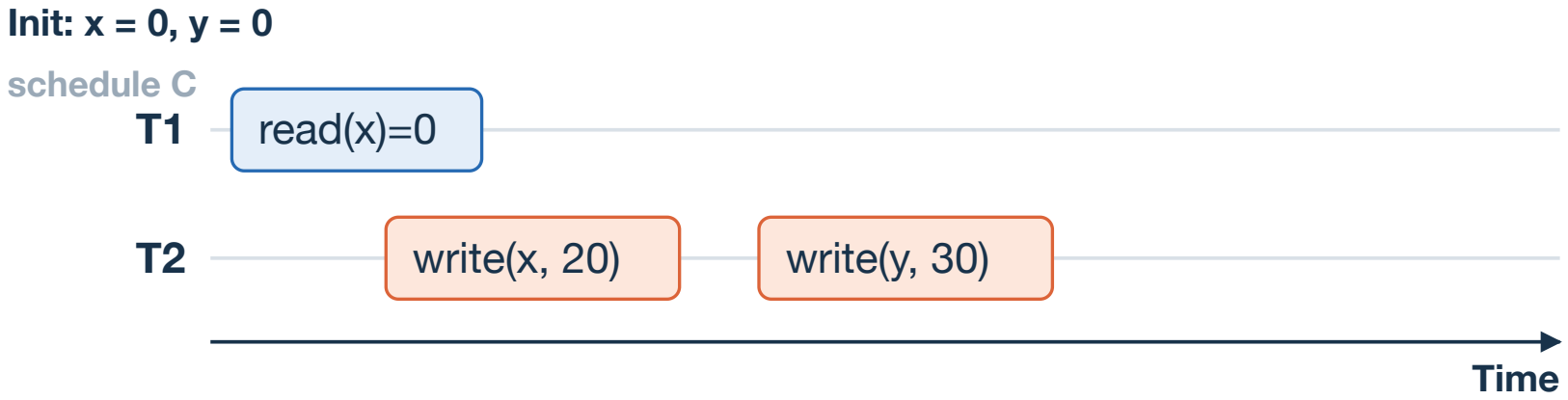
Think: the result is right, but the reads are impossible



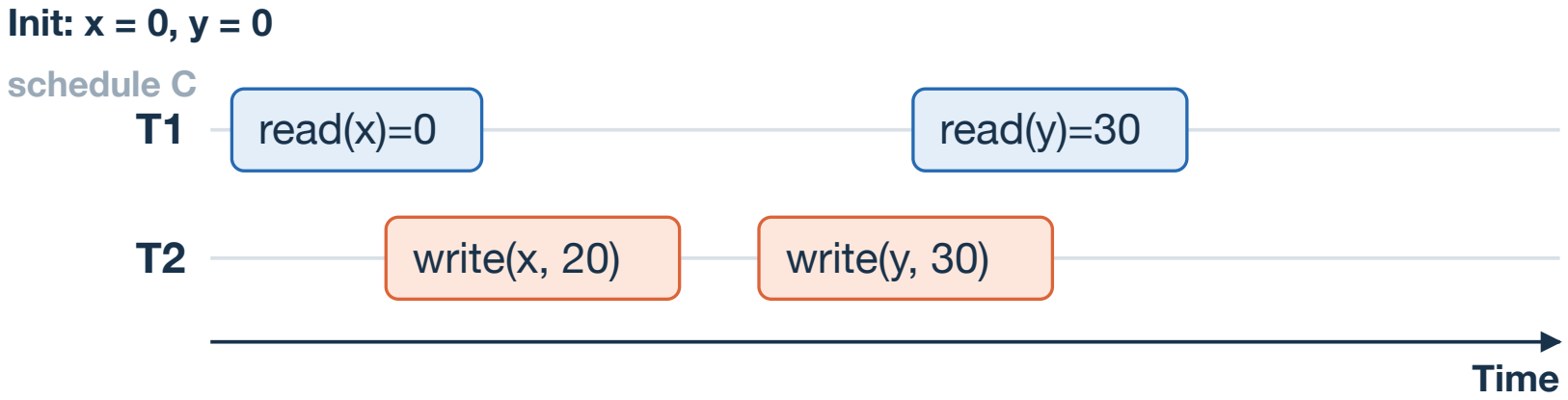
Think: the result is right, but the reads are impossible



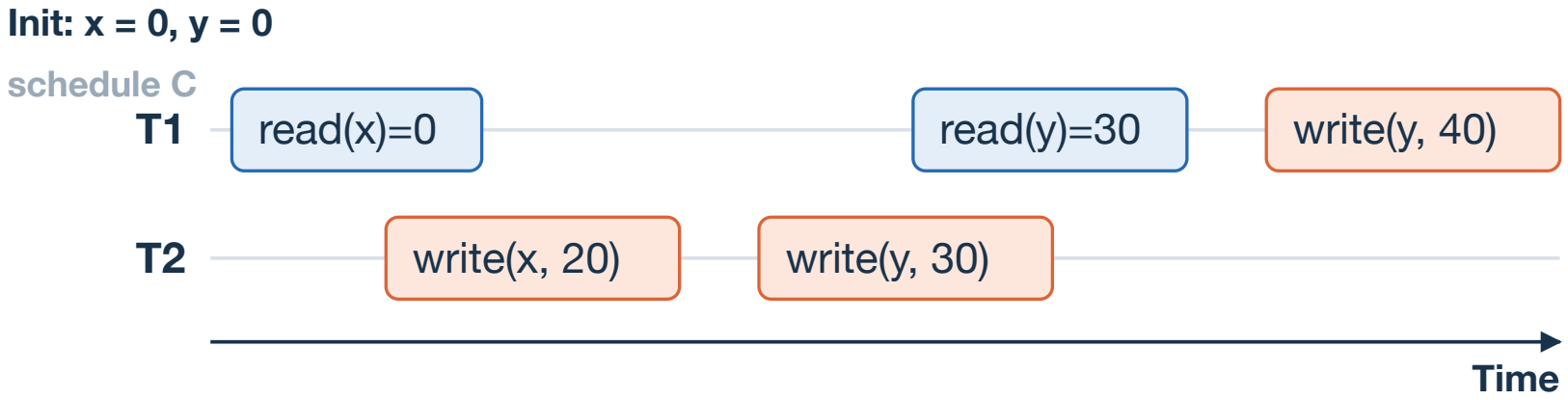
Think: the result is right, but the reads are impossible



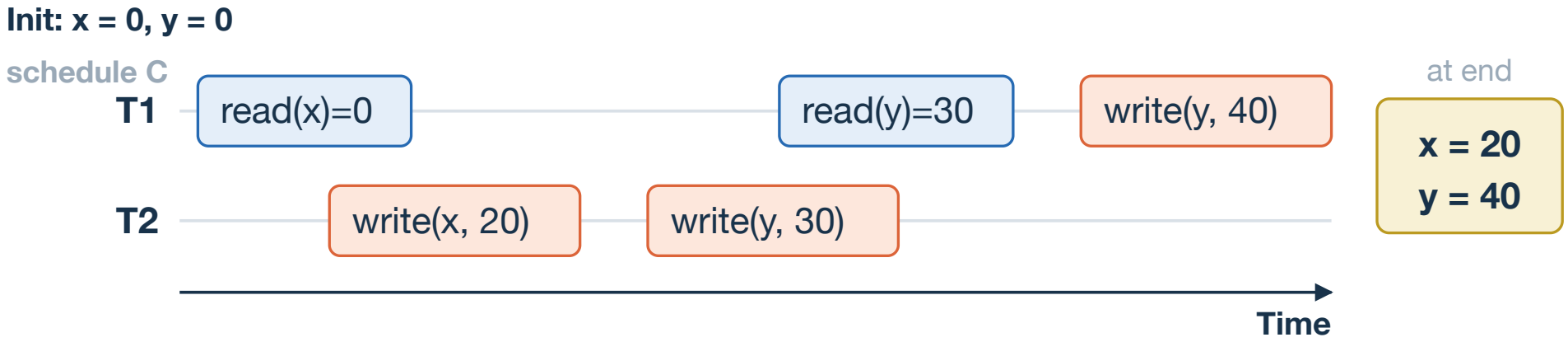
Think: the result is right, but the reads are impossible



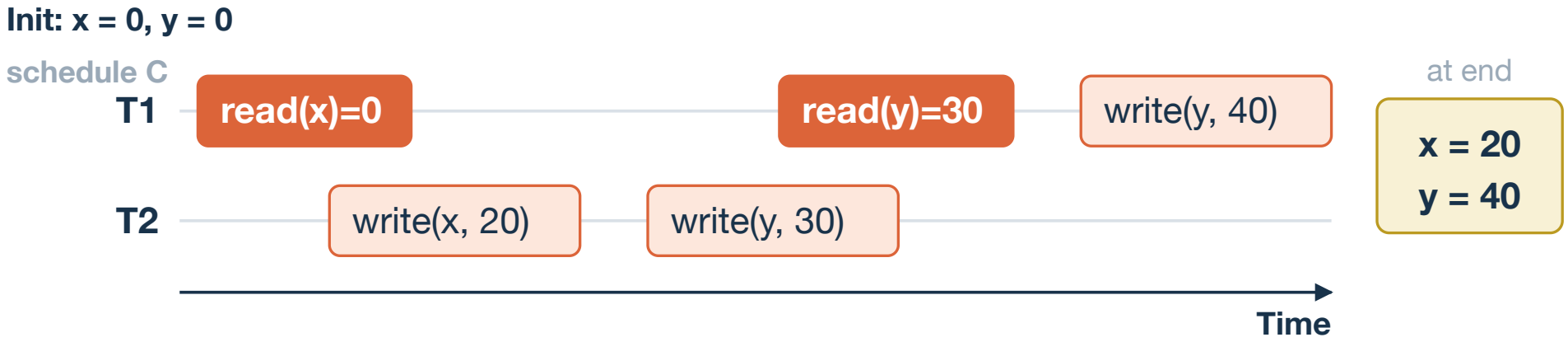
Think: the result is right, but the reads are impossible



Think: the result is right, but the reads are impossible



Think: the result is right, but the reads are impossible



The final state is x=20, y=40, the same as the serial T2 → T1

But T1 reads x=0 and y=30; those reads are not possible in a sequential schedule

Think: is that ok?

- No. appear needs to also ensure view is correct

View serializability

A schedule is view serializable if the final written state as well as the intermediate reads are the same as in some serial schedule

Formally, two schedules S and S' are view equivalent if

- Condition #1: if Ti in S reads an initial value for x, so does Ti in S'
- Condition #2: if Ti in S reads the value written by Tj for some x, so does Ti in S'
- Condition #3: if Ti in S does the final write to x, so does Ti in S'

A schedule is view serializable if it is view equivalent to some serial schedule

Question: is view serializability possible to check?

The definition is clear: there exists some serial schedule it is view equivalent to

Question: what does it cost us to decide that?

We need to enumerate all possible schedules, currently very time consuming!

- And it is still unclear whether it can be done in practice

Conflict serializability

Two operations conflict if all three hold

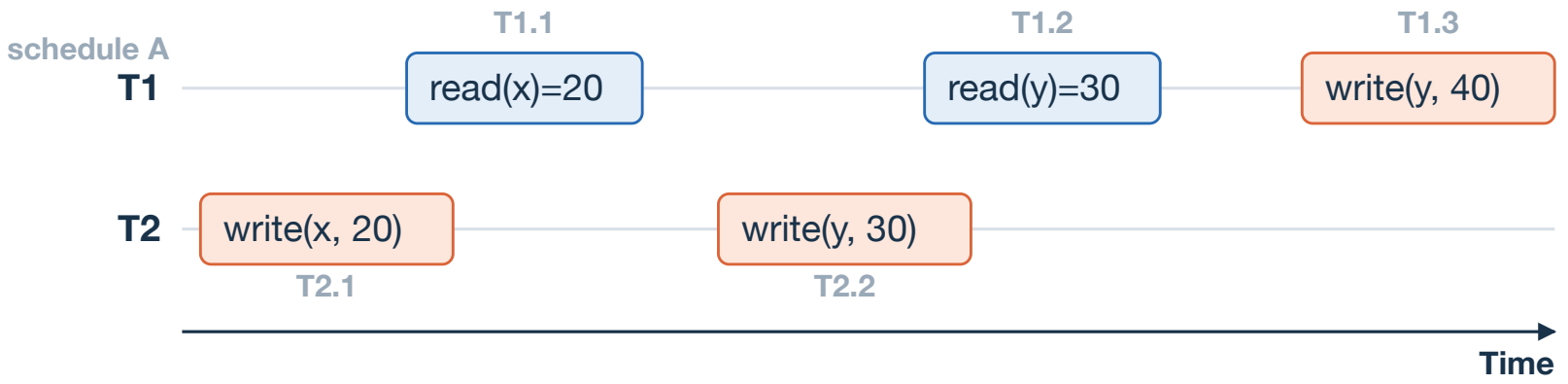
- They operate on the same data object
- At least one of them is a write
- They belong to different transactions (actions)

A schedule is conflict serializable if the order of its conflicts is the same as the order of conflicts in some sequential schedule

The conflicts in our example

Three pairs in total: one on x, two on y

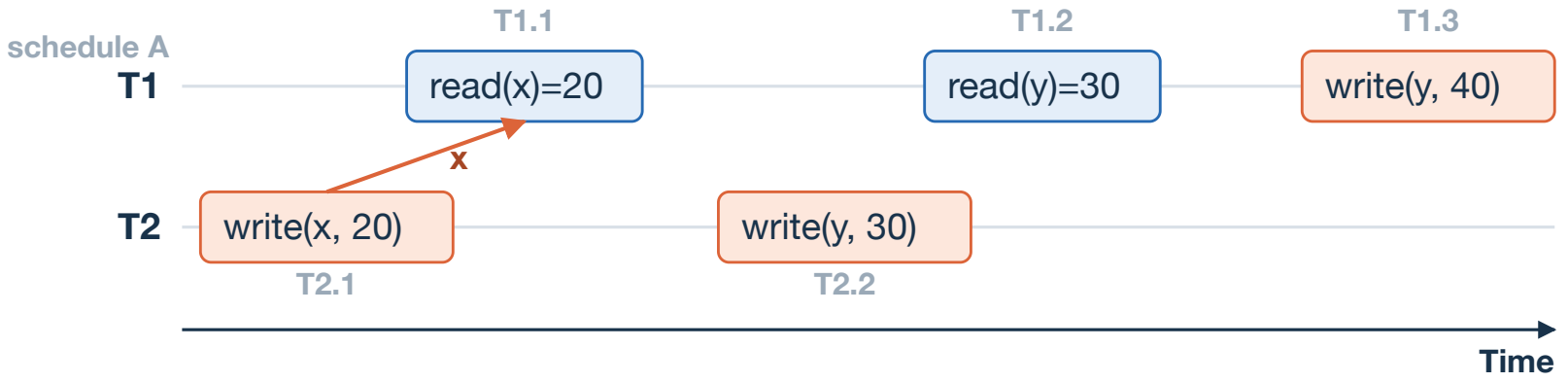
- The arrow points from the step that happens first



The conflicts in our example

Three pairs in total: one on x, two on y

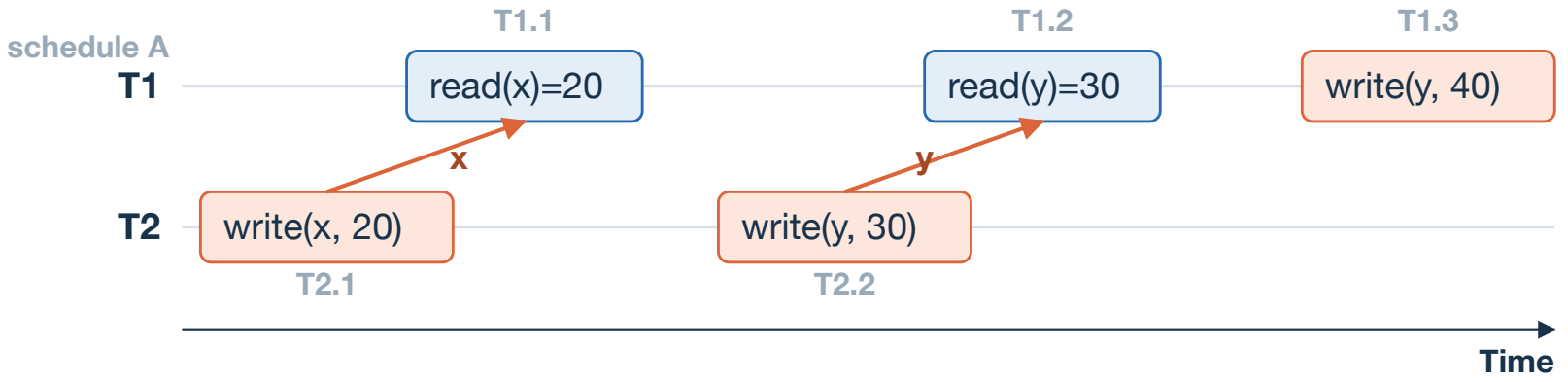
- The arrow points from the step that happens first



The conflicts in our example

Three pairs in total: one on x, two on y

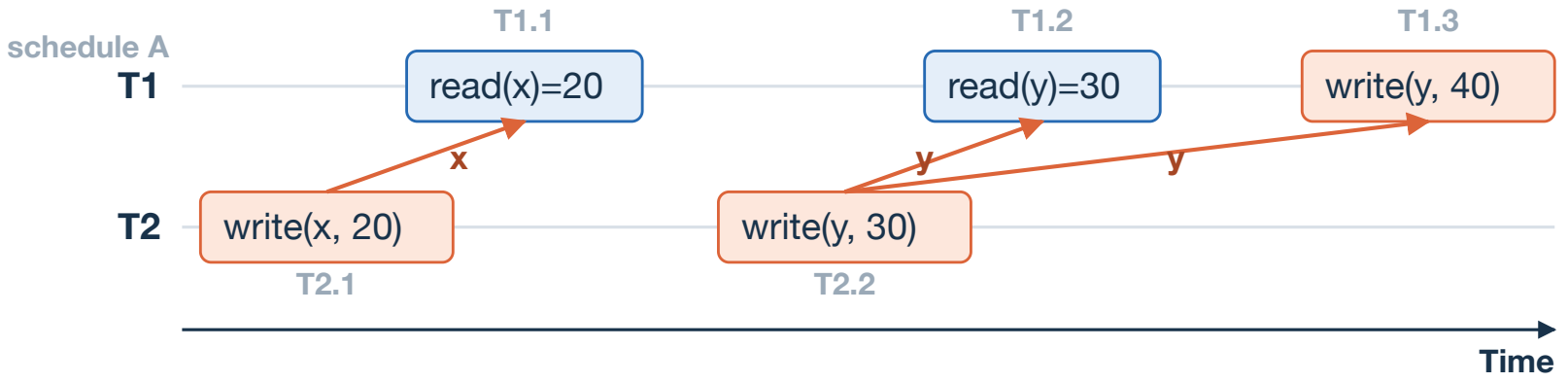
- The arrow points from the step that happens first



The conflicts in our example

Three pairs in total: one on x, two on y

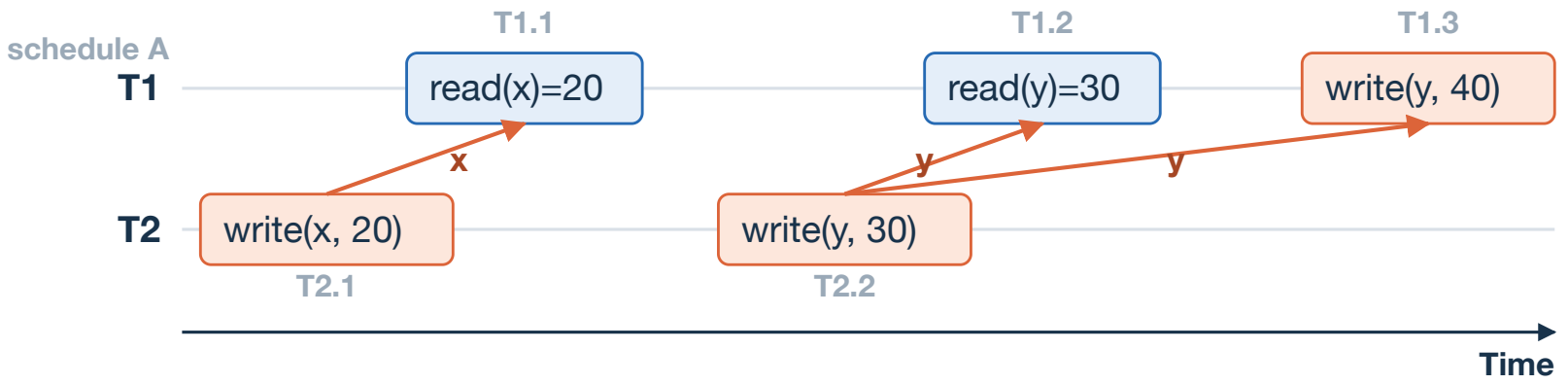
- The arrow points from the step that happens first



Conflict graph

Nodes are transactions, edges are directed

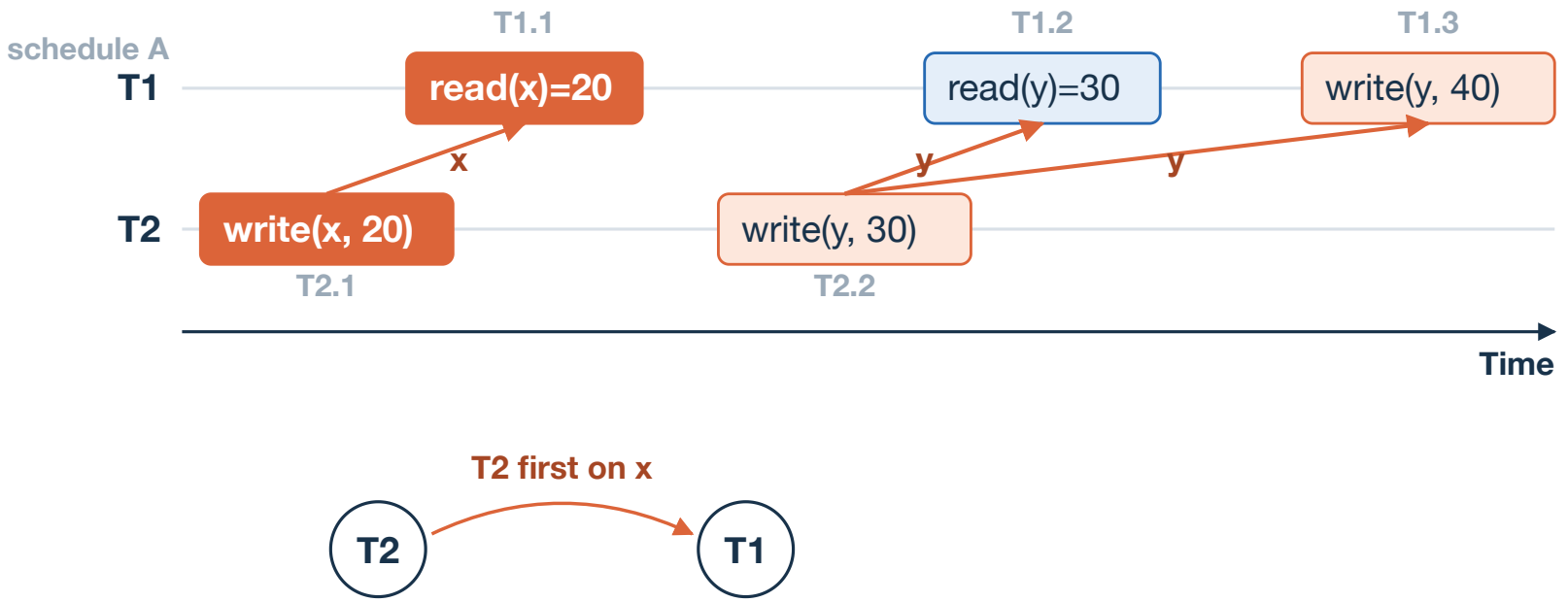
- Ti → Tj iff the two conflict, and the first step of that conflict is Ti's



Conflict graph

Nodes are transactions, edges are directed

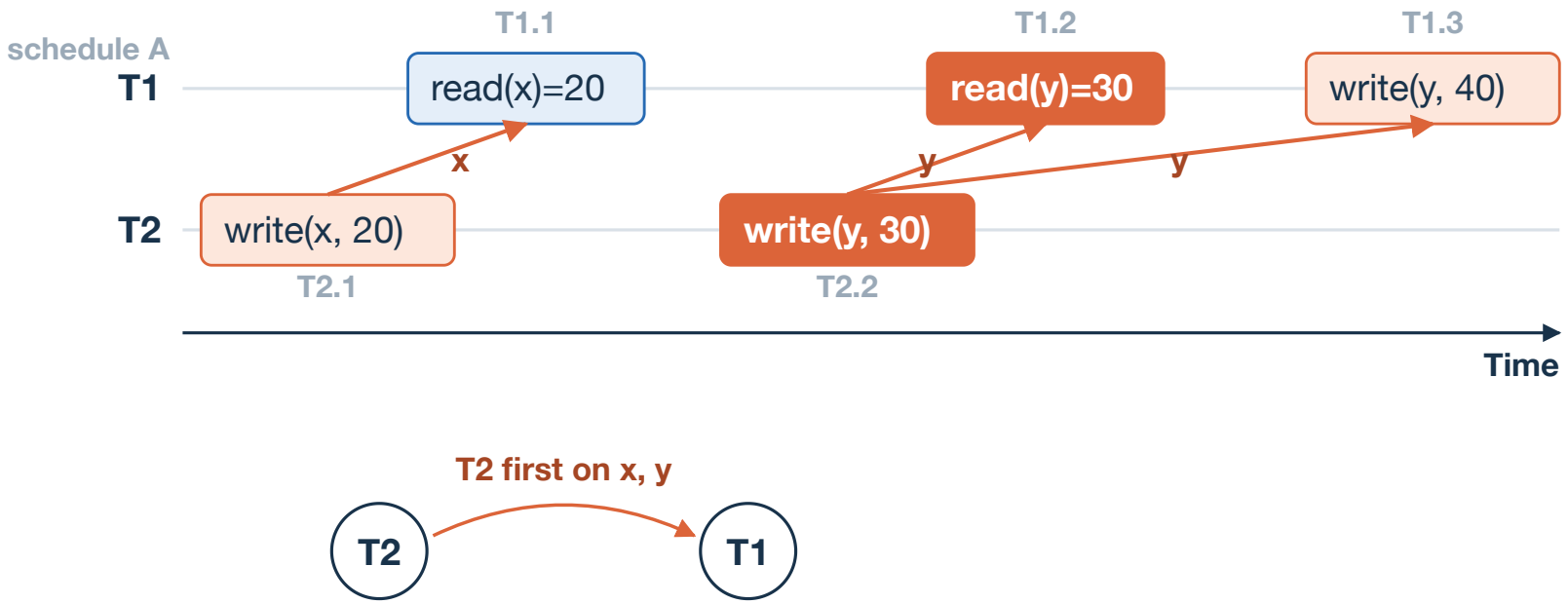
- Ti → Tj iff the two conflict, and the first step of that conflict is Ti's



Conflict graph

Nodes are transactions, edges are directed

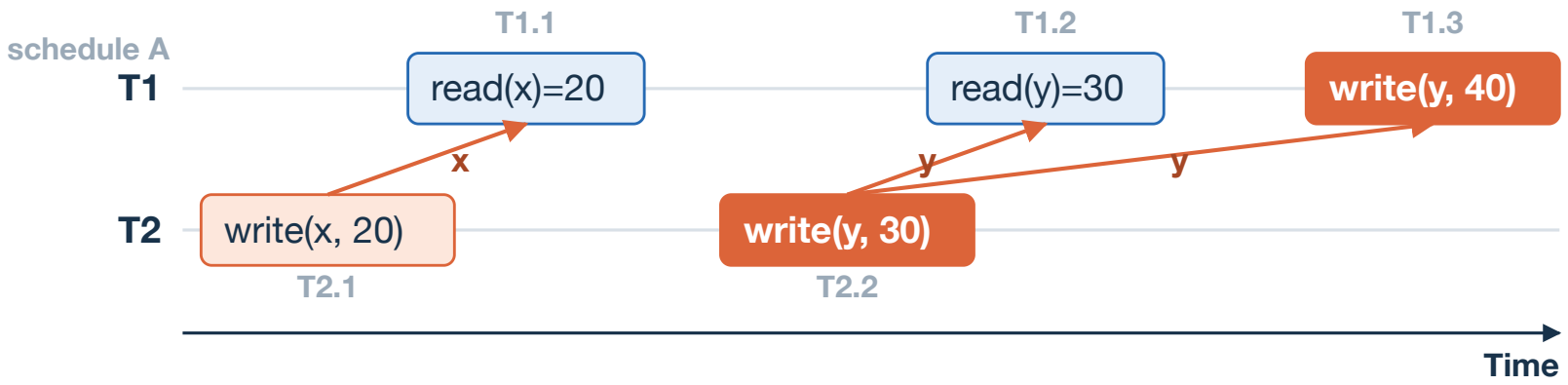
- Ti → Tj iff the two conflict, and the first step of that conflict is Ti's

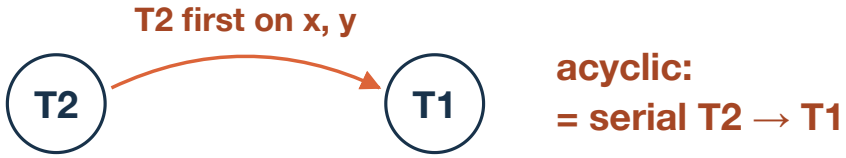


Conflict graph

Nodes are transactions, edges are directed

- Ti → Tj iff the two conflict, and the first step of that conflict is Ti's



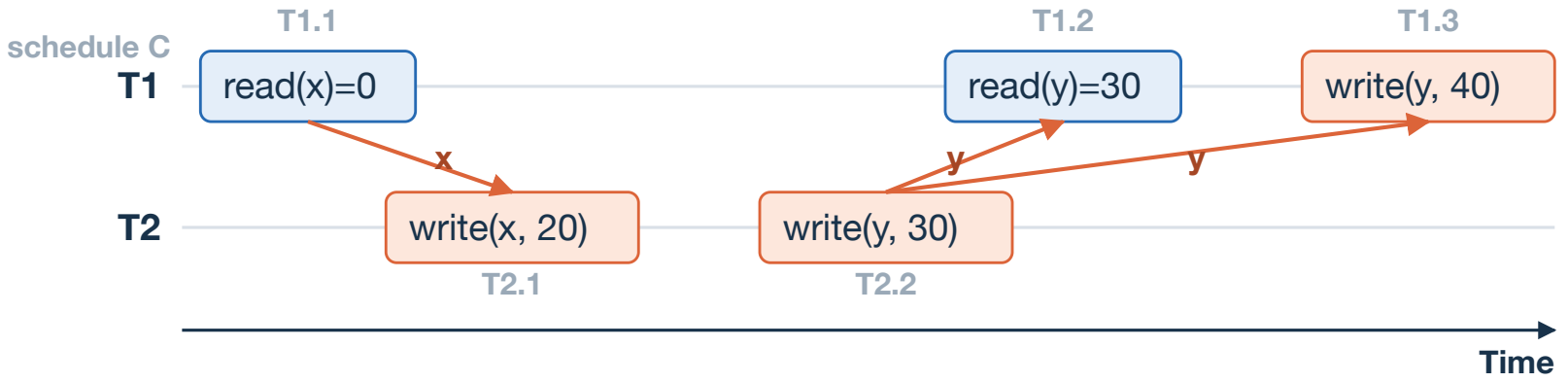


acyclic ⟺ conflict serializable

The conflict graph of schedule C

The pair on x puts T1 first, the pairs on y put T2 first

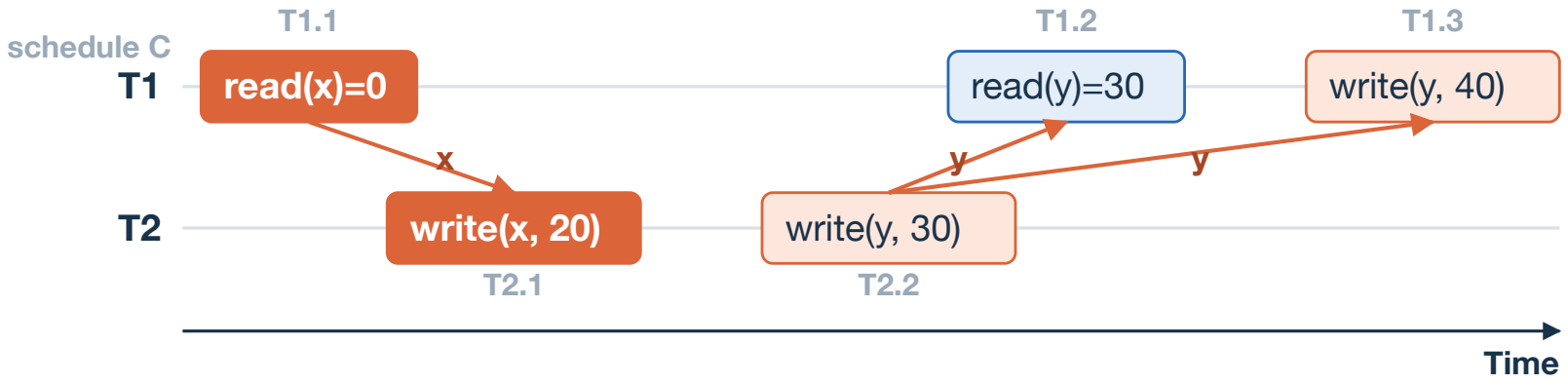
- Two edges, and they close into a cycle

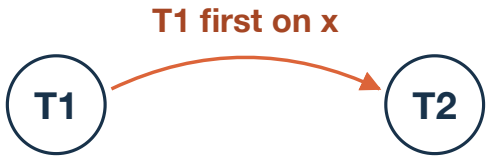


The conflict graph of schedule C

The pair on x puts T1 first, the pairs on y put T2 first

- Two edges, and they close into a cycle

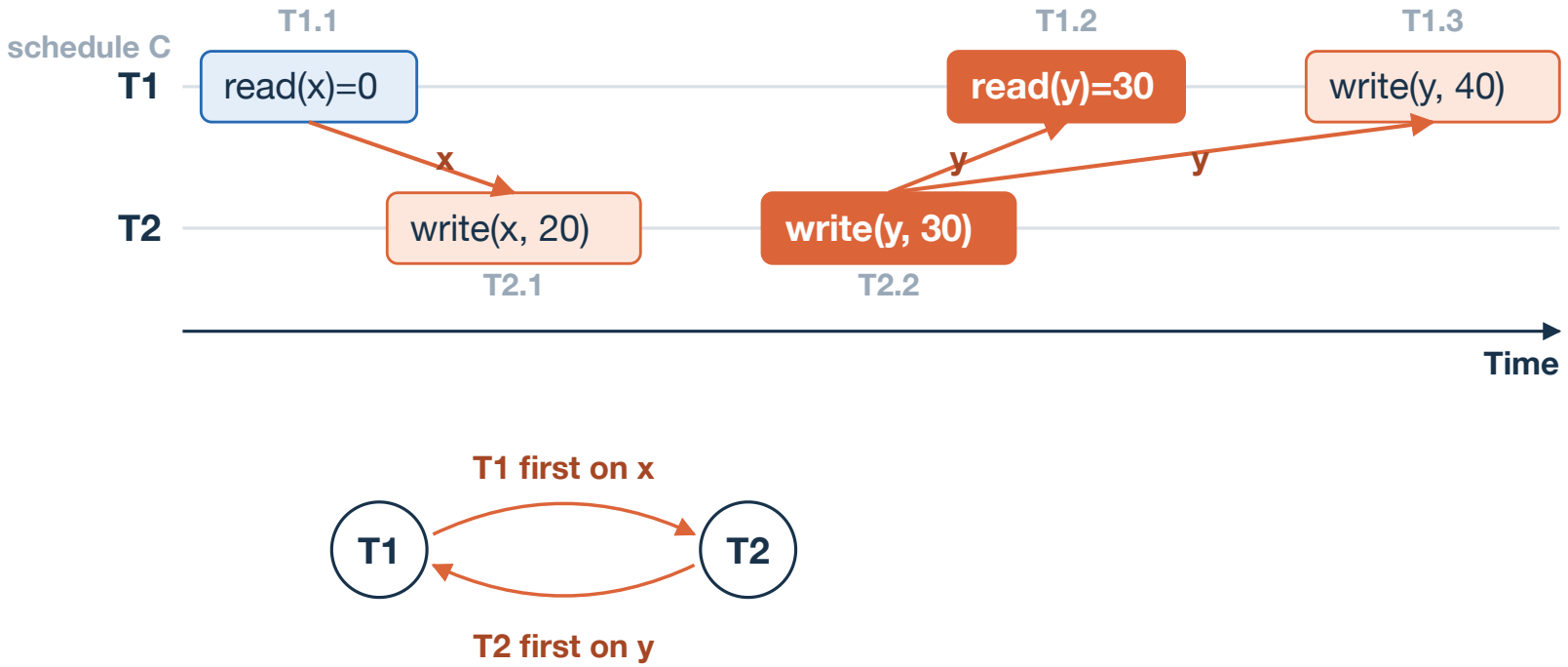




The conflict graph of schedule C

The pair on x puts T1 first, the pairs on y put T2 first

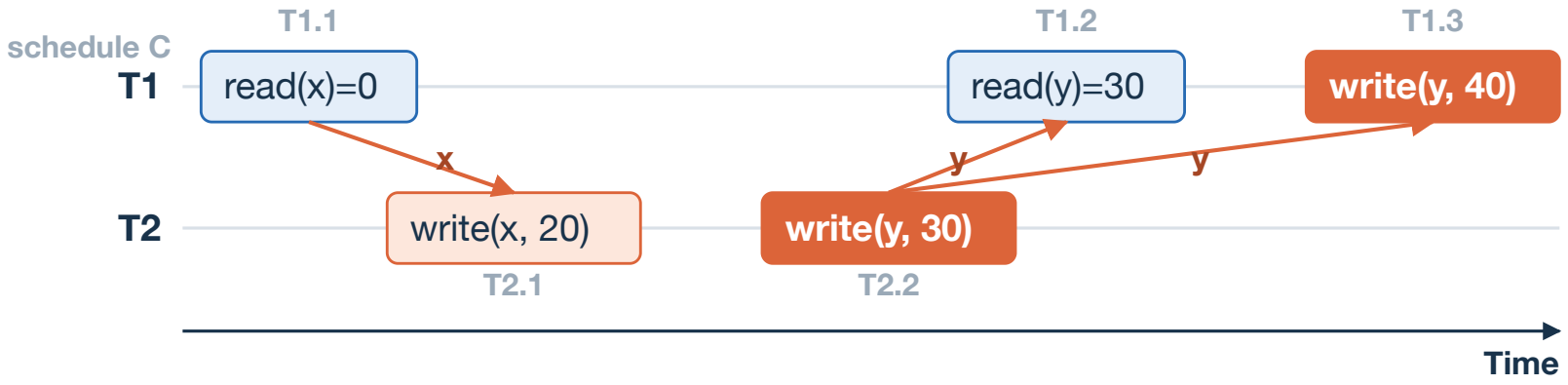
- Two edges, and they close into a cycle

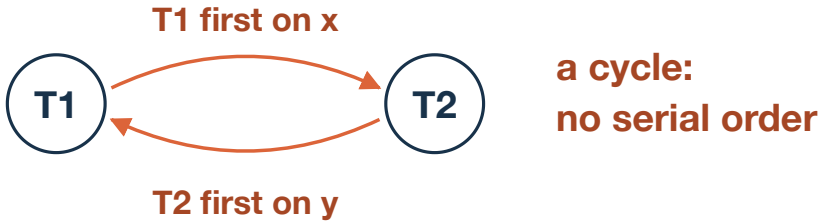


The conflict graph of schedule C

The pair on x puts T1 first, the pairs on y put T2 first

- Two edges, and they close into a cycle



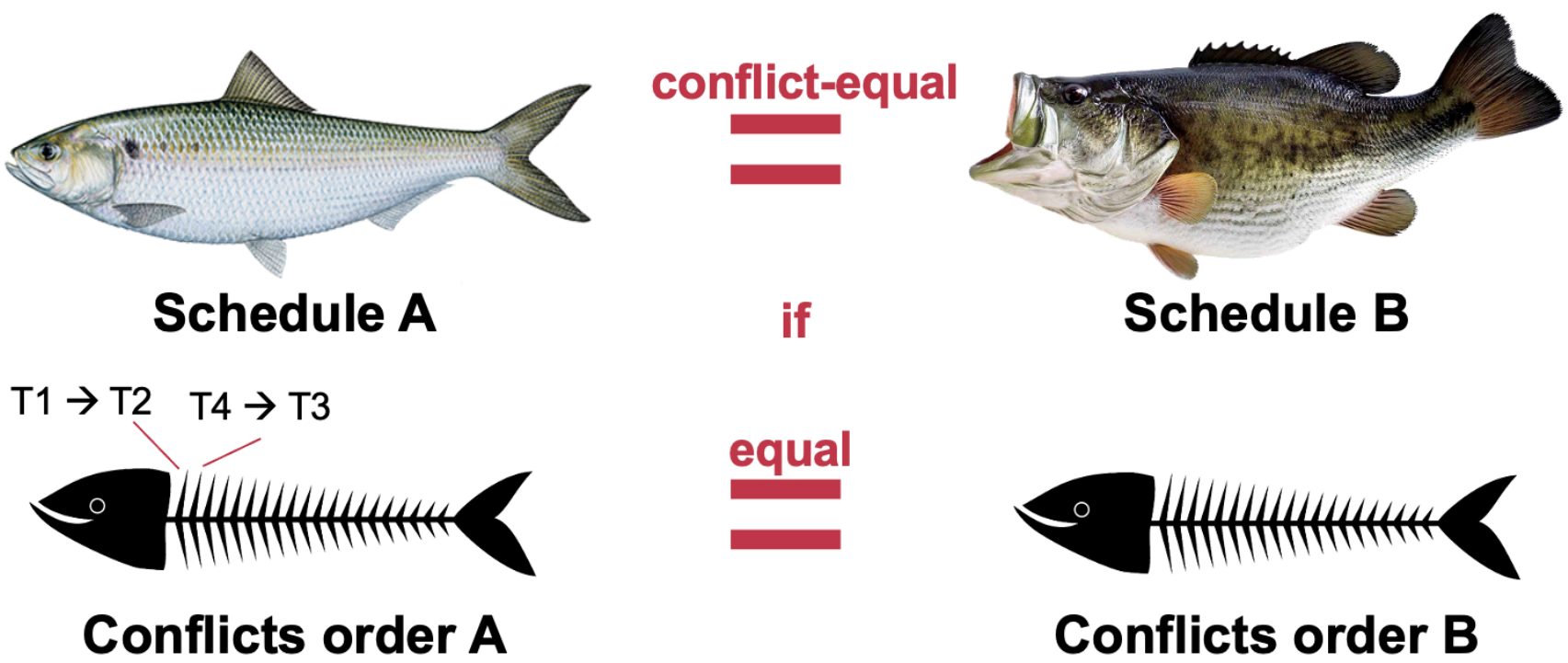


a cycle:
no serial order

Neither serial order matches it, so C is not conflict serializable

Conflict equivalence

If conflict-order-A equals conflict-order-B, then schedule A conflict-equals schedule B



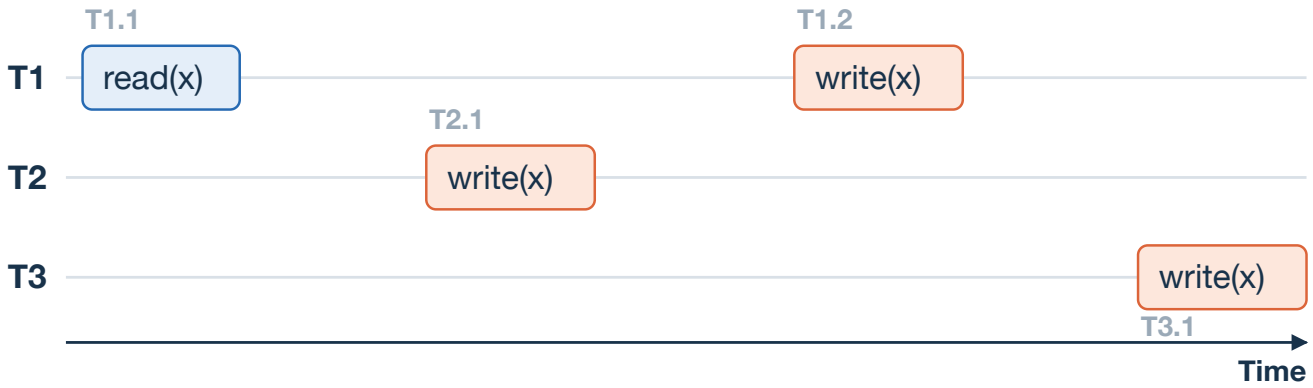
[1]

The two schedules may look nothing alike; what has to match is the conflict order

[1] Source: original course slides

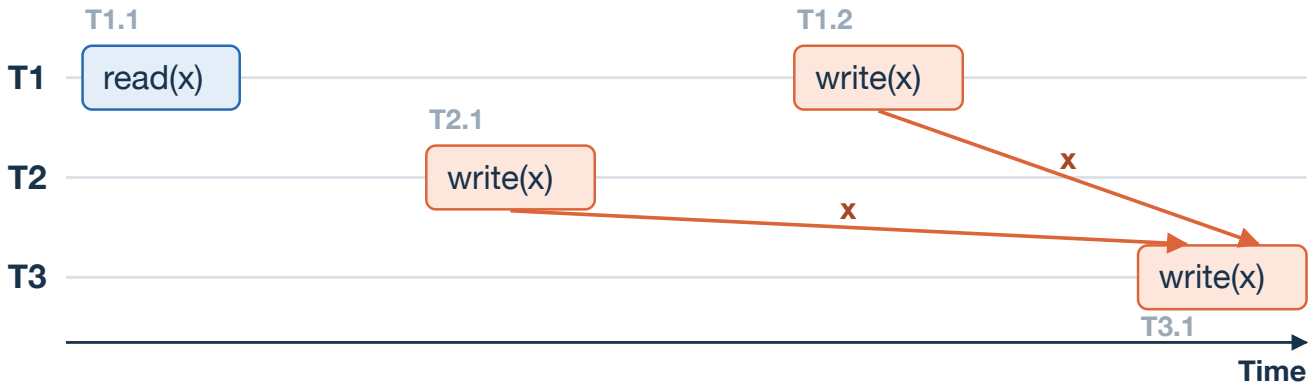
A schedule that is view but not conflict serializable

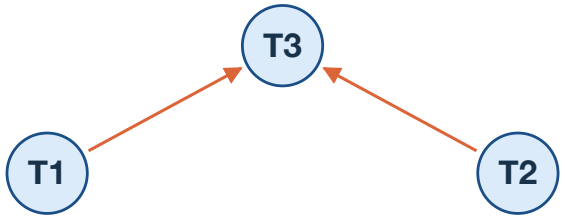
Three transactions, every step on x



A schedule that is view but not conflict serializable

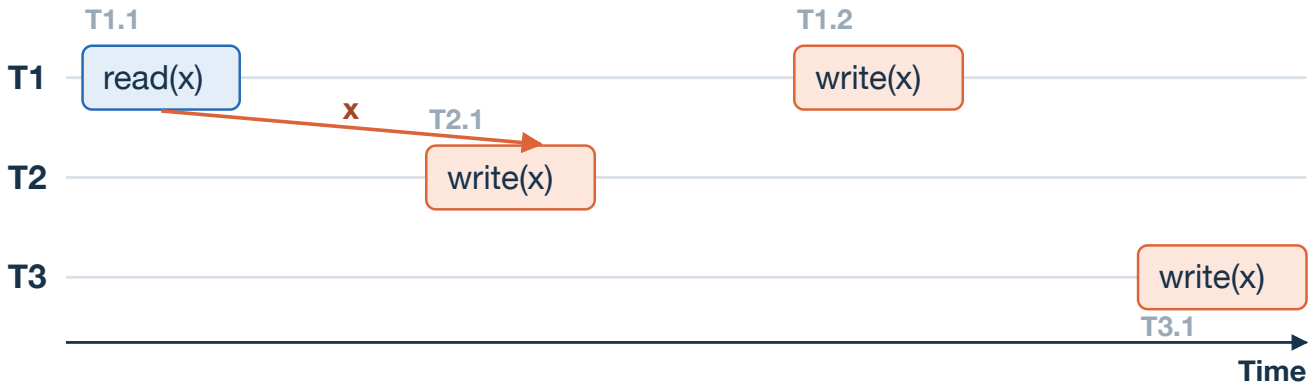
Three transactions, every step on x

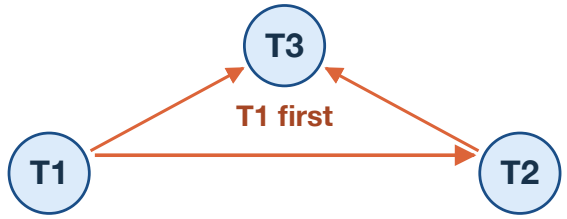




A schedule that is view but not conflict serializable

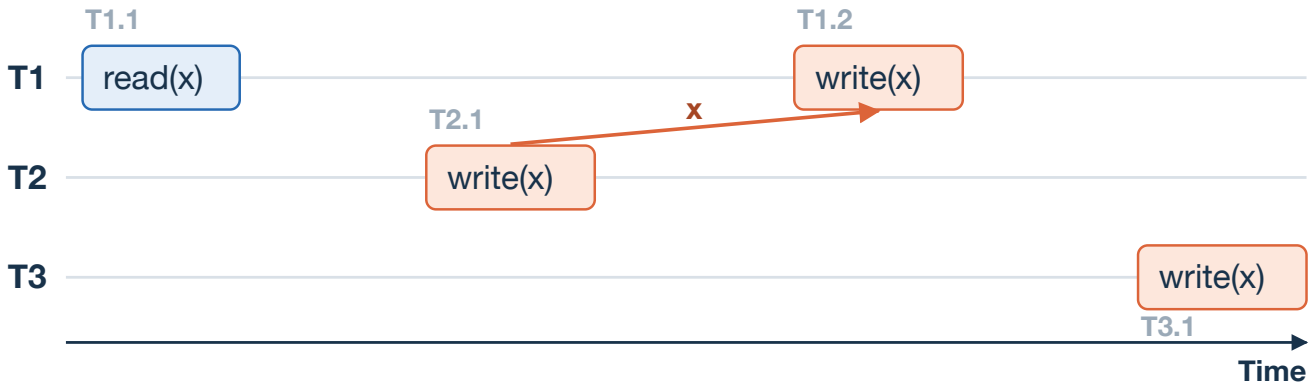
Three transactions, every step on x

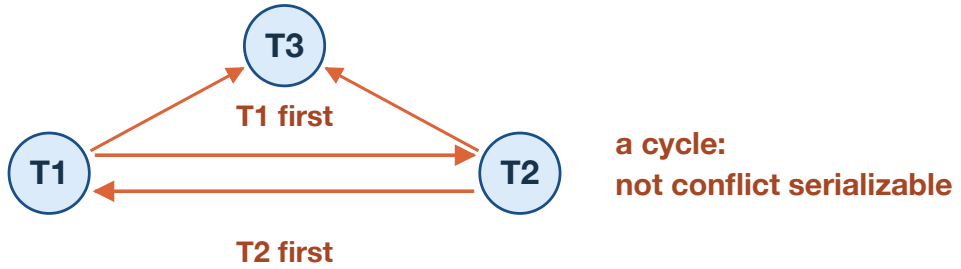




A schedule that is view but not conflict serializable

Three transactions, every step on x

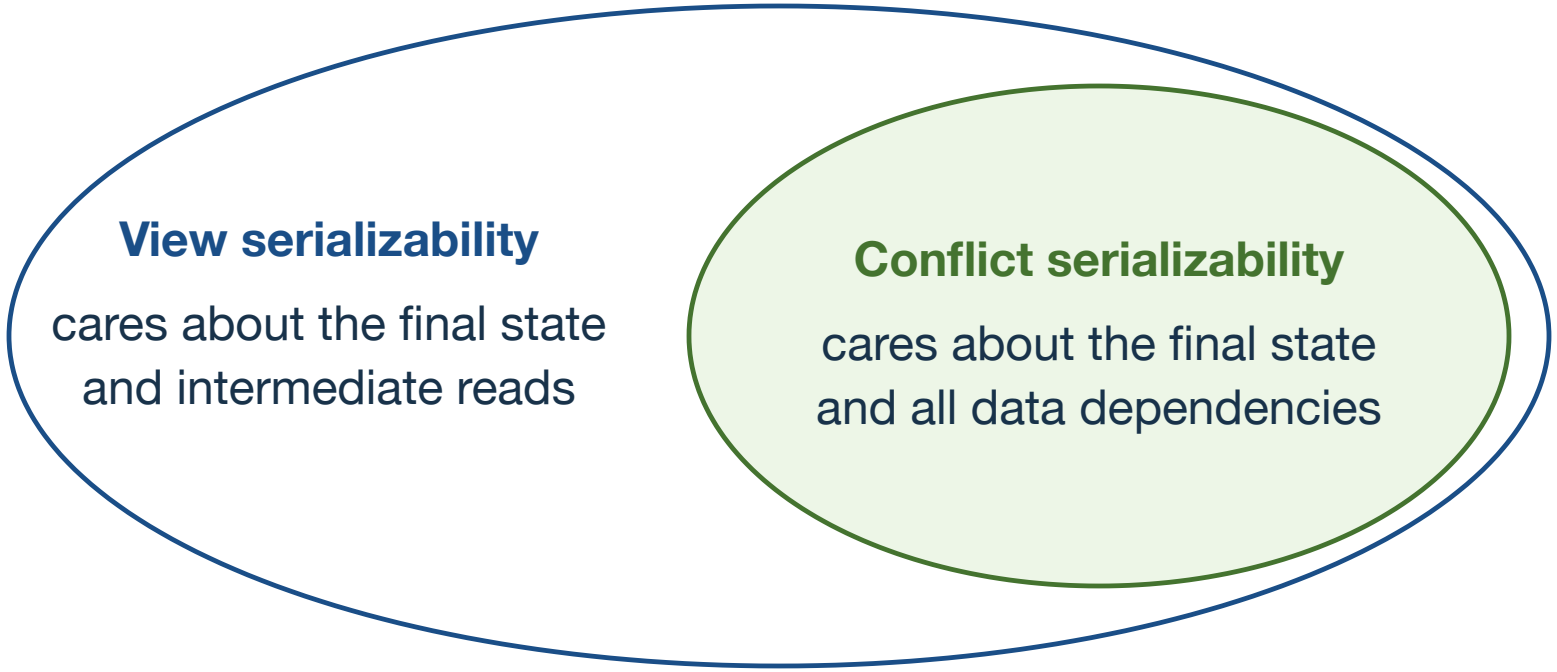




But T1 → T2 → T3 gives the same reads and the same final write, so it is view serializable

- Question: why shouldn't we allow this schedule?

View serializability vs. conflict serializability



A conflict serializable schedule is always view serializable, but not the other way round

- View serializability cares about the final state and the intermediate reads
- Conflict serializability cares about the final state and all the data dependencies

Why systems use conflict serializability

It has two advantages

- Easy to test: check whether the conflict graph is acyclic, a polynomial-time graph algorithm
- Easy to generate: concurrency control protocols such as 2PL produce conflict serializable schedules directly

The cost: it is stricter than view serializability, and rejects some acceptable schedules

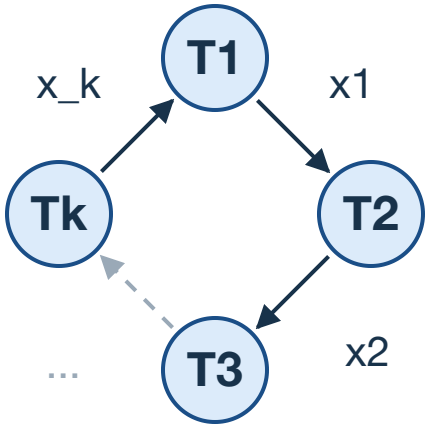
- But there is no easy way to generate those schedules anyway

Proof of 2PL (by contradiction)

Assume a 2PL execution has a conflict cycle: T1 → T2 → … → Tk → T1

- Mutual exclusion: conflicting accesses require a lock handoff
- 2PL: every successful lock acquisition precedes every lock release in that transaction

Assumed conflict cycle



Derived event order

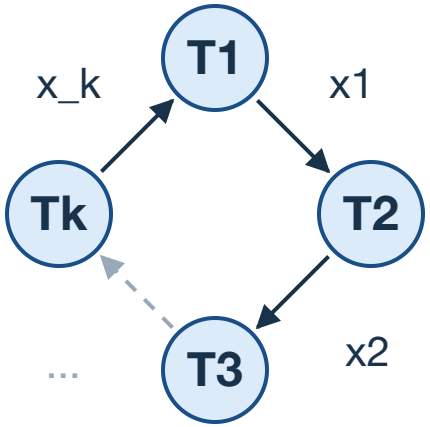
`T1 releases x1.lock`

Proof of 2PL (by contradiction)

Assume a 2PL execution has a conflict cycle: T1 → T2 → ... → Tk → T1

- Mutual exclusion: conflicting accesses require a lock handoff
- 2PL: every successful lock acquisition precedes every lock release in that transaction

Assumed conflict cycle



Derived event order

`T1 releases x1.lock`

↓

`T2 acquires x1.lock`

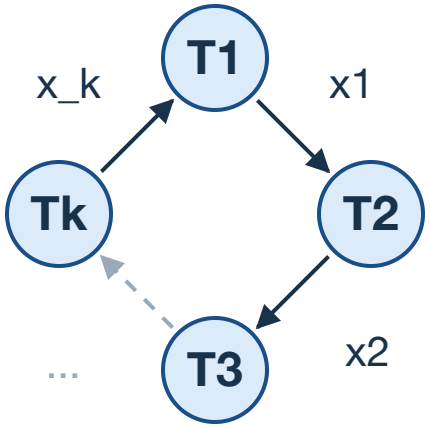
mutual exclusion

Proof of 2PL (by contradiction)

Assume a 2PL execution has a conflict cycle: T1 → T2 → … → Tk → T1

- Mutual exclusion: conflicting accesses require a lock handoff
- 2PL: every successful lock acquisition precedes every lock release in that transaction

Assumed conflict cycle



Derived event order

```
T1 releases x1.lock
↓ T2 acquires x1.lock
↓ T2 releases x2.lock
```

mutual exclusion

2PL in T2

Allowed in T2

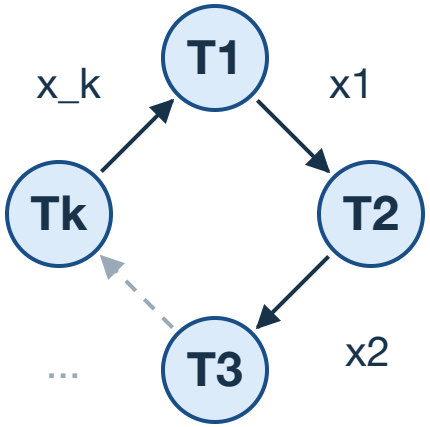
```
acquire x2 → wait for x1
acquire x1 → release x2
```

Proof of 2PL (by contradiction)

Assume a 2PL execution has a conflict cycle: T1 → T2 → ... → Tk → T1

- Mutual exclusion: conflicting accesses require a lock handoff
- 2PL: every successful lock acquisition precedes every lock release in that transaction

Assumed conflict cycle



Allowed in T2

```
acquire x2 → wait for x1
acquire x1 → release x2
```

Derived event order

```
T1 releases x1.lock
↓
T2 acquires x1.lock
↓
T2 releases x2.lock
↓
...
↓
Tk releases x_k.lock
```

mutual exclusion

2PL in T2

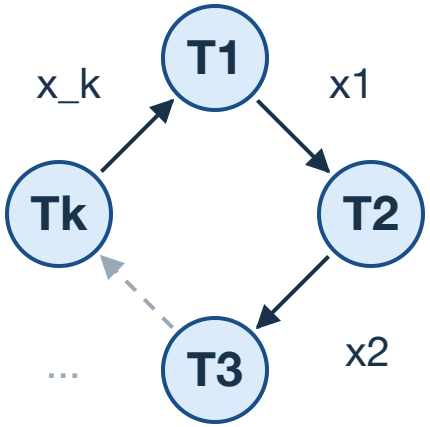
repeat along cycle

Proof of 2PL (by contradiction)

Assume a 2PL execution has a conflict cycle: T1 → T2 → ... → Tk → T1

- Mutual exclusion: conflicting accesses require a lock handoff
- 2PL: every successful lock acquisition precedes every lock release in that transaction

Assumed conflict cycle



Allowed in T2

```
acquire x2 → wait for x1
acquire x1 → release x2
```

Derived event order

```
T1 releases x1.lock
↓ T2 acquires x1.lock
↓ T2 releases x2.lock
↓ ...
↓ Tk releases x_k.lock
↓ T1 acquires x_k.lock
```

mutual exclusion

2PL in T2

repeat along cycle

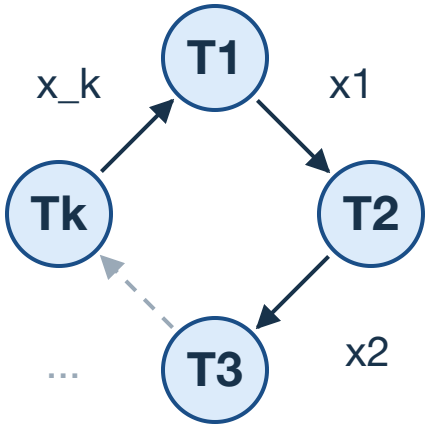
mutual exclusion

Proof of 2PL (by contradiction)

Assume a 2PL execution has a conflict cycle: T1 → T2 → ... → Tk → T1

- Mutual exclusion: conflicting accesses require a lock handoff
- 2PL: every successful lock acquisition precedes every lock release in that transaction

Assumed conflict cycle



Allowed in T2

```
acquire x2 → wait for x1
acquire x1 → release x2
```

Derived event order

```
T1 releases x1.lock
↓ T2 acquires x1.lock
↓ T2 releases x2.lock
↓ ...
↓ Tk releases x_k.lock
↓ T1 acquires x_k.lock
```

mutual exclusion

2PL in T2

repeat along cycle

mutual exclusion

acquire after release **violates 2PL**

T1 releases, then acquires: **violates 2PL. No cycle ⇒ conflict serializable.**

We keep saying before-or-after atomicity is for correctness.

But what is correctness?

Correctness is application-level semantics

```
Transfer(bank, a, b, amt):    # correct
    bank[b] += amt
    bank[a] -= amt
```

```
Transfer(bank, a, b, amt):
    bank[b] += amt * 2                # buggy: should be amt
    bank[a] -= amt
```

Correctness is hard for a computer system to enforce alone, because it depends on the application

- A developer can write a buggy program, so it can be problematic even with one single-threaded execution

So reasoning about concurrent correctness needs an assumption: the single-threaded implementation of each action is correct

Correctness and serializability

Assumption: programmers are pro at writing single-threaded programs

- What do we mean by a correct single-threaded action?

In a single-threaded context, a tx moves the data from one consistent state to another

$$C_i \rightarrow tx \rightarrow C_j$$

If the execution of actions guarantees serializability, it reduces to some serial sequence

$$C_0 \rightarrow tx_0 \rightarrow C_1 \rightarrow tx_1 \rightarrow \cdots \rightarrow C_n$$

If C_0 is consistent, then C_n must be consistent

The issue of 2PL: it is pessimistic

2PL is pessimistic: it assumes conflicts can happen, and coordinates before an action runs

- A transaction (TX) must acquire the lock of a record before reading or writing it
- If it cannot get the lock, it waits for the conflicting TX to release it

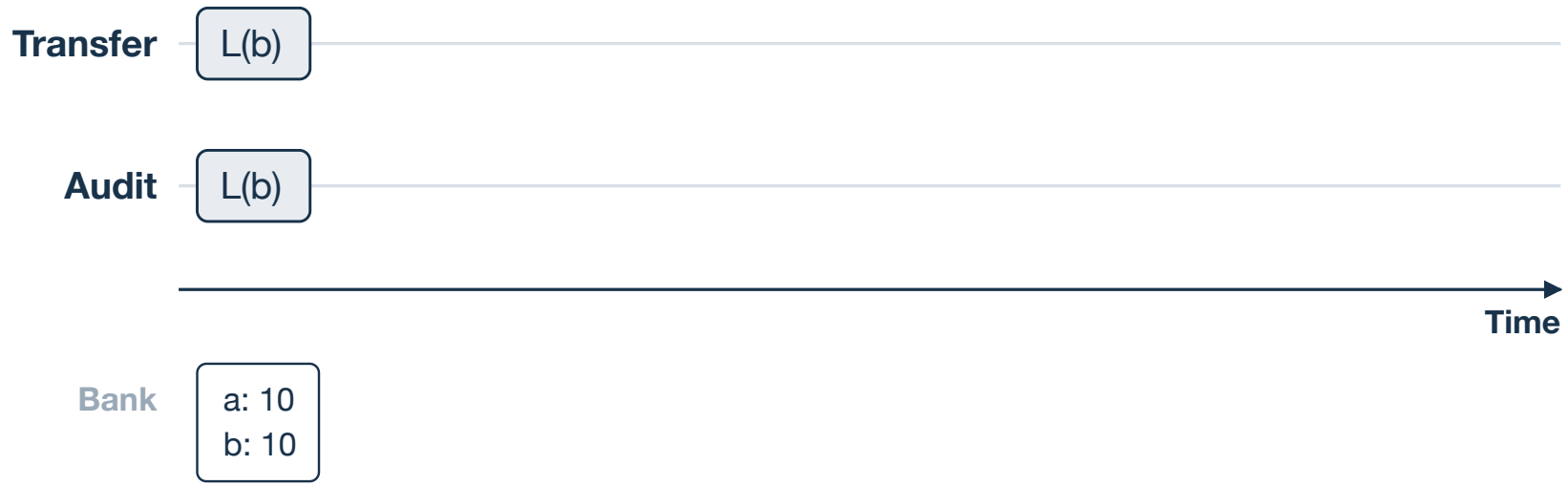
Question: what can happen if two TXs are waiting for each other?

That Transfer + Audit under 2PL

```
Transfer(bank, a, b, amt):
    bank[b] += amt
    bank[a] -= amt
```

```
Audit(bank):
    sum = 0
    for acct in bank: sum += bank[acct]
    print(sum)
```

Audit waits on `lock[b]` until Transfer releases both locks, so Sum = 20

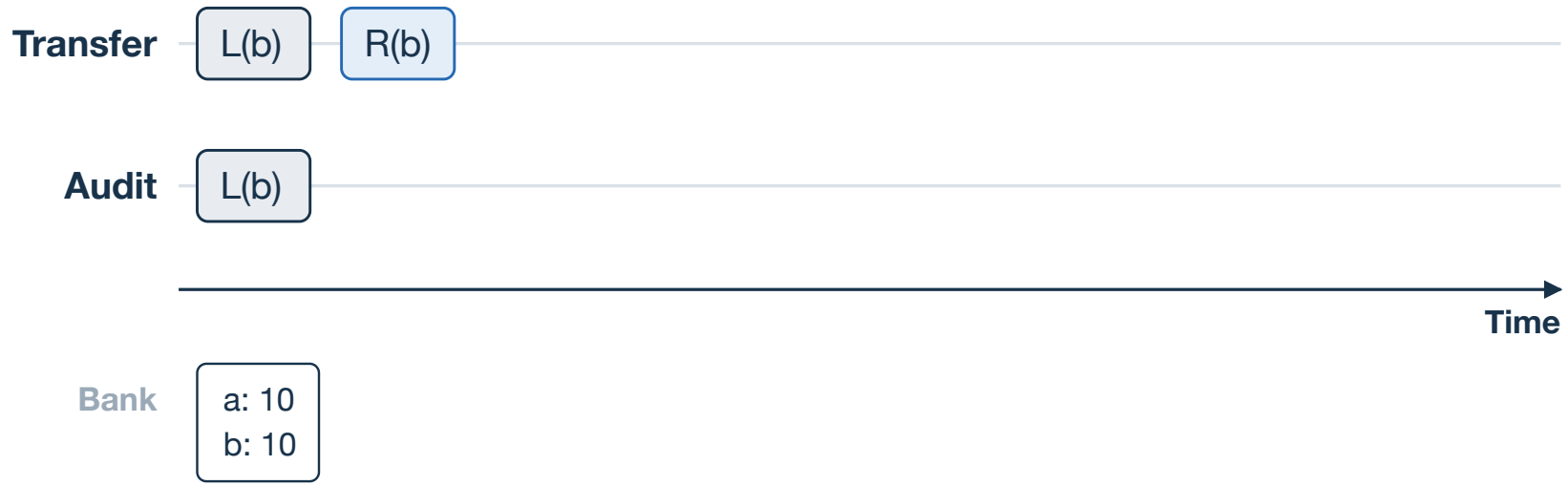


That Transfer + Audit under 2PL

```
Transfer(bank, a, b, amt):
    bank[b] += amt
    bank[a] -= amt
```

```
Audit(bank):
    sum = 0
    for acct in bank: sum += bank[acct]
    print(sum)
```

Audit waits on `lock[b]` until Transfer releases both locks, so Sum = 20



That Transfer + Audit under 2PL

```
Transfer(bank, a, b, amt):
    bank[b] += amt
    bank[a] -= amt
```

```
Audit(bank):
    sum = 0
    for acct in bank: sum += bank[acct]
    print(sum)
```

Audit waits on `lock[b]` until Transfer releases both locks, so Sum = 20

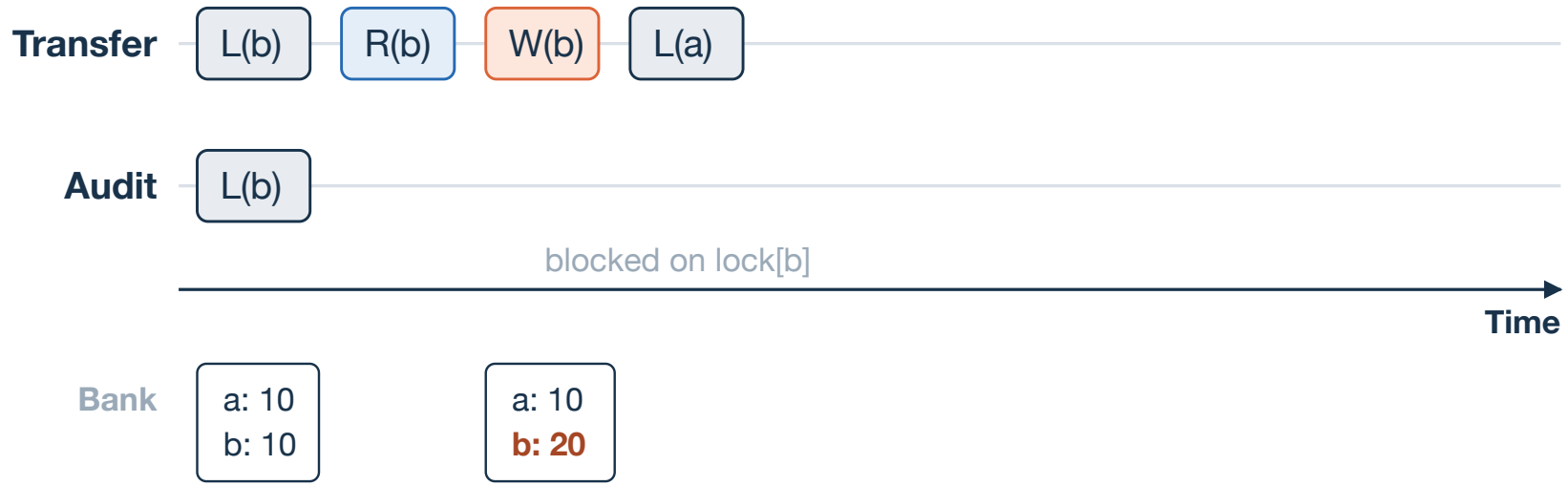


That Transfer + Audit under 2PL

```
Transfer(bank, a, b, amt):
    bank[b] += amt
    bank[a] -= amt
```

```
Audit(bank):
    sum = 0
    for acct in bank: sum += bank[acct]
    print(sum)
```

Audit waits on `lock[b]` until Transfer releases both locks, so Sum = 20

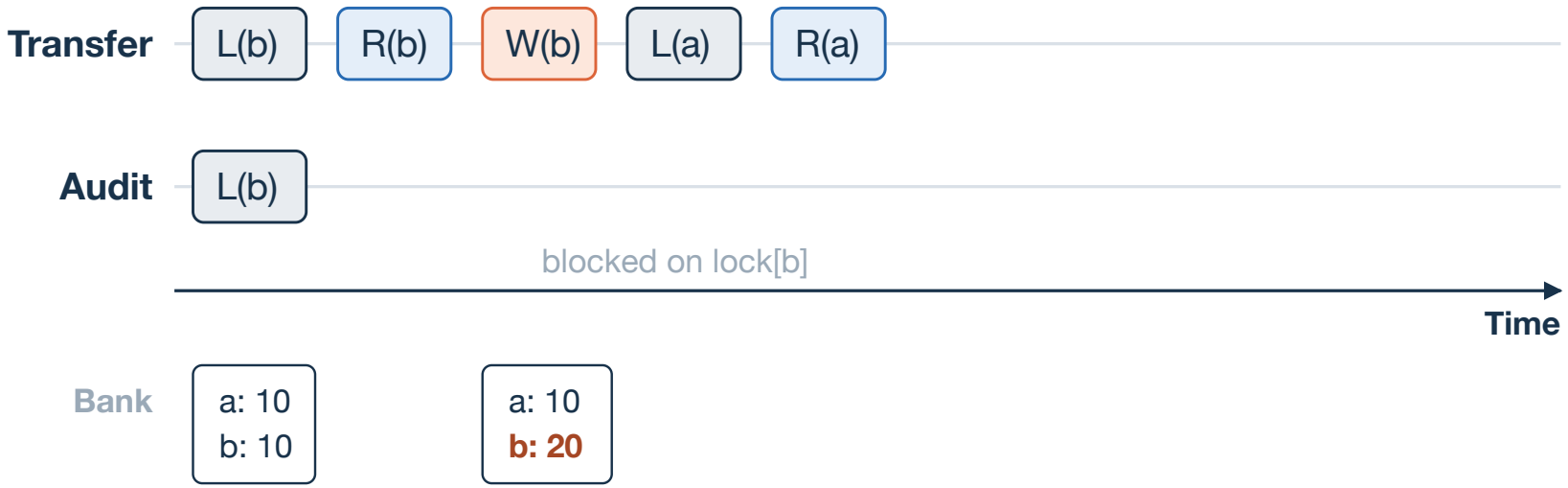


That Transfer + Audit under 2PL

```
Transfer(bank, a, b, amt):
    bank[b] += amt
    bank[a] -= amt
```

```
Audit(bank):
    sum = 0
    for acct in bank: sum += bank[acct]
    print(sum)
```

Audit waits on `lock[b]` until Transfer releases both locks, so Sum = 20

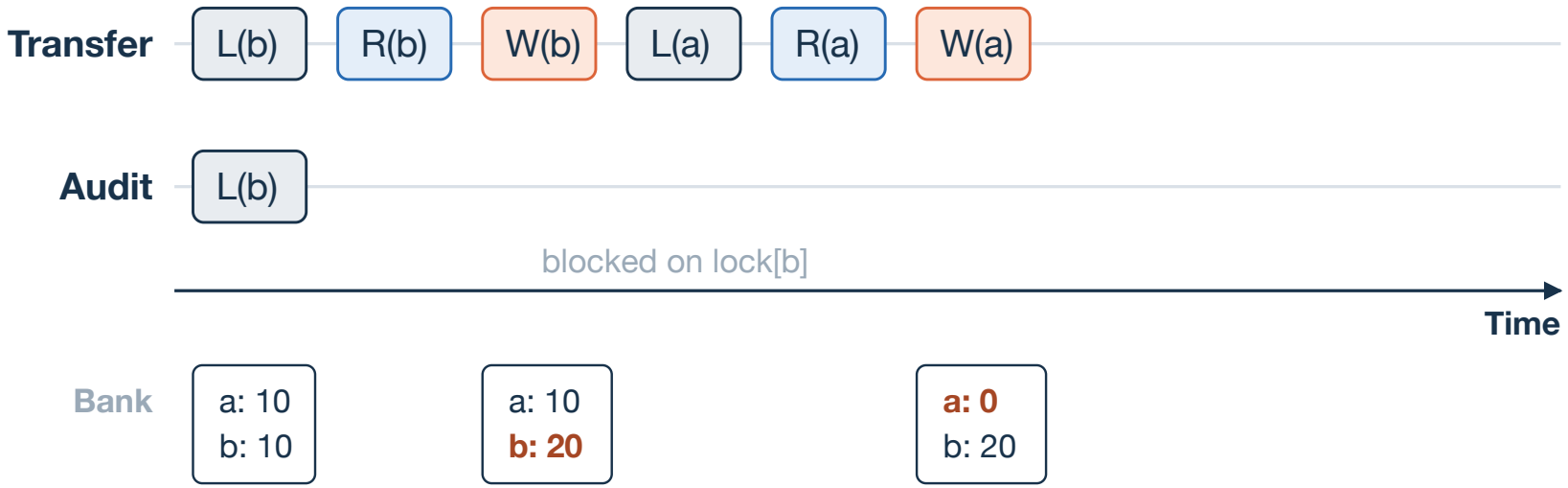


That Transfer + Audit under 2PL

```
Transfer(bank, a, b, amt):
    bank[b] += amt
    bank[a] -= amt
```

```
Audit(bank):
    sum = 0
    for acct in bank: sum += bank[acct]
    print(sum)
```

Audit waits on `lock[b]` until Transfer releases both locks, so Sum = 20

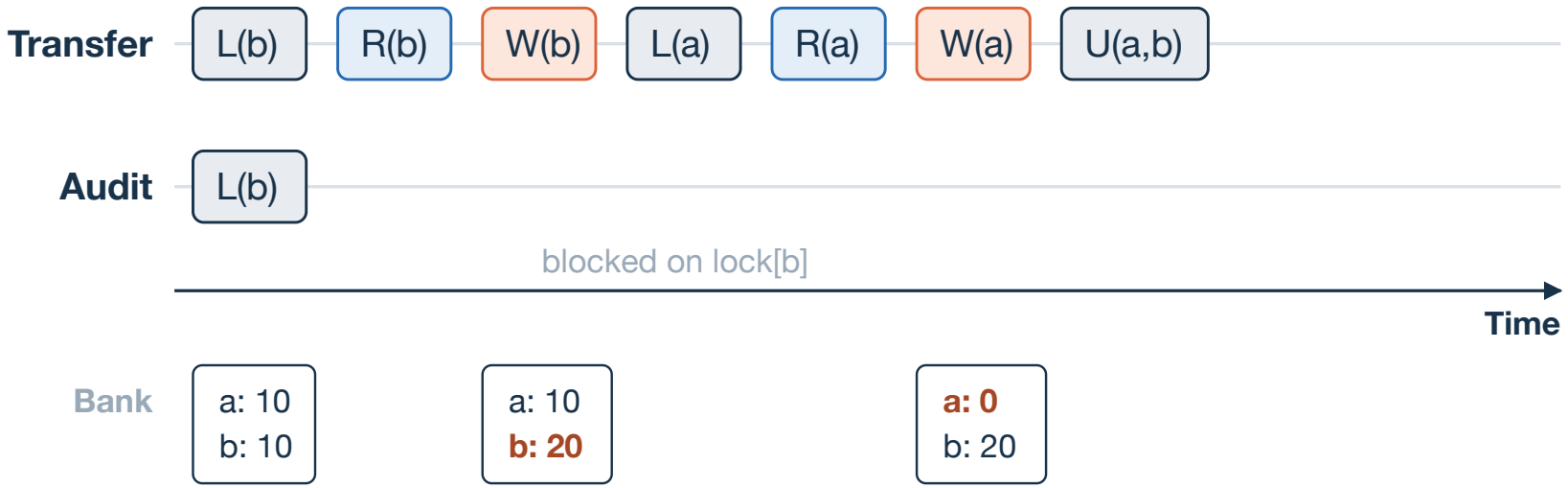


That Transfer + Audit under 2PL

```
Transfer(bank, a, b, amt):
    bank[b] += amt
    bank[a] -= amt
```

```
Audit(bank):
    sum = 0
    for acct in bank: sum += bank[acct]
    print(sum)
```

Audit waits on `lock[b]` until Transfer releases both locks, so Sum = 20

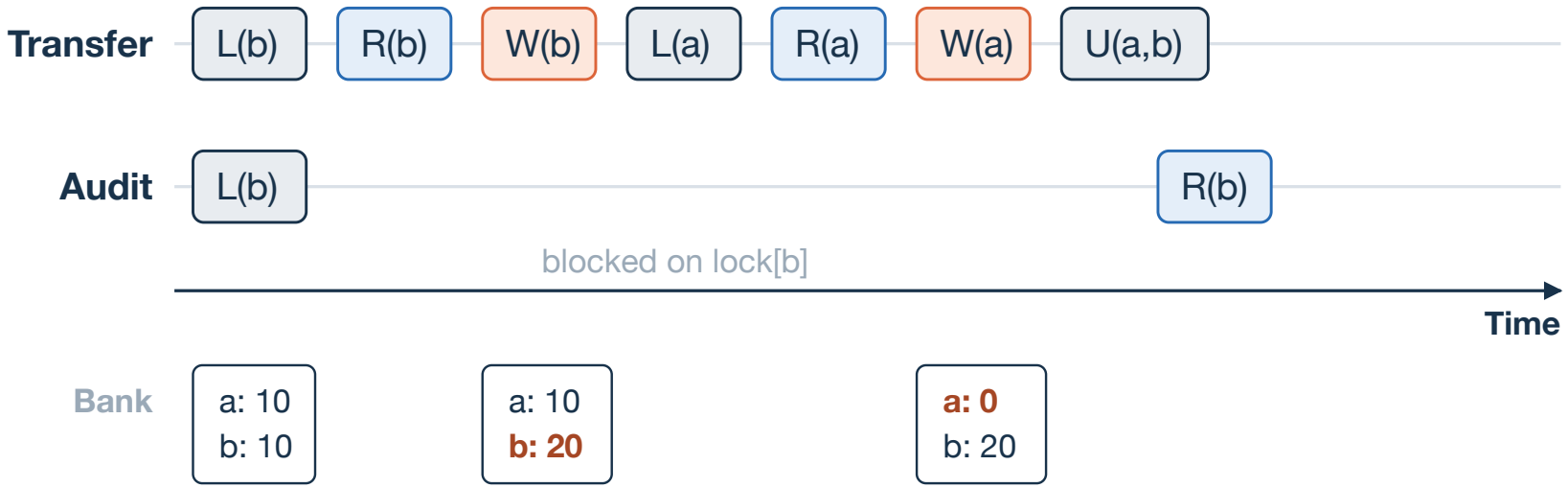


That Transfer + Audit under 2PL

```
Transfer(bank, a, b, amt):
    bank[b] += amt
    bank[a] -= amt
```

```
Audit(bank):
    sum = 0
    for acct in bank: sum += bank[acct]
    print(sum)
```

Audit waits on `lock[b]` until Transfer releases both locks, so Sum = 20

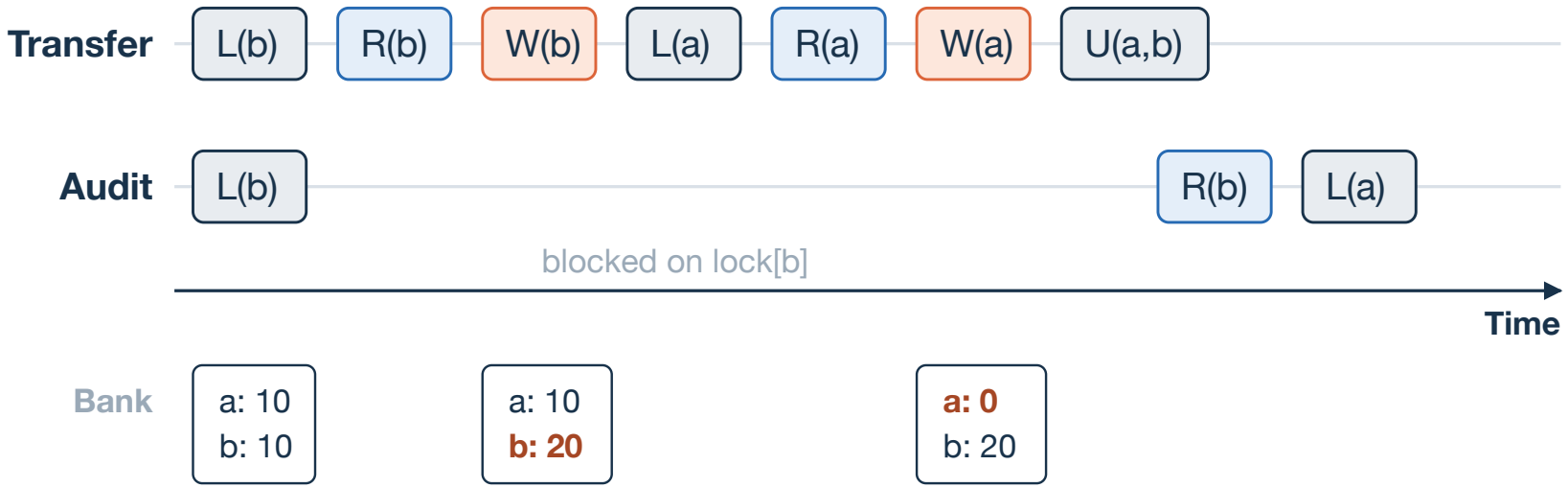


That Transfer + Audit under 2PL

```
Transfer(bank, a, b, amt):
    bank[b] += amt
    bank[a] -= amt
```

```
Audit(bank):
    sum = 0
    for acct in bank: sum += bank[acct]
    print(sum)
```

Audit waits on `lock[b]` until Transfer releases both locks, so Sum = 20

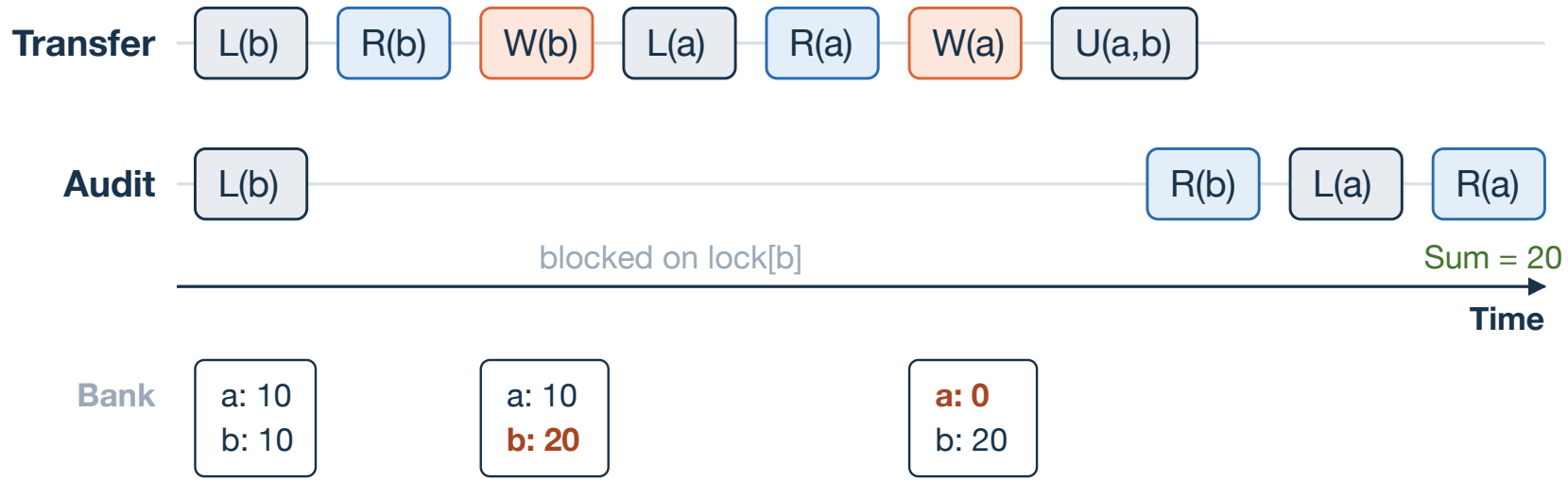


That Transfer + Audit under 2PL

```
Transfer(bank, a, b, amt):
    bank[b] += amt
    bank[a] -= amt
```

```
Audit(bank):
    sum = 0
    for acct in bank: sum += bank[acct]
    print(sum)
```

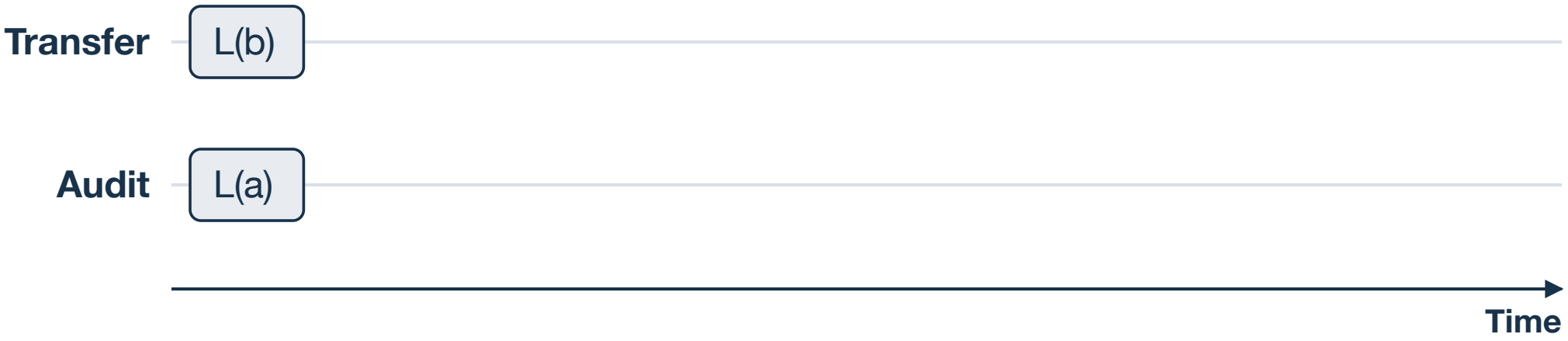
Audit waits on `lock[b]` until Transfer releases both locks, so Sum = 20



Deadlock: what if Audit first acquires `lock[a]`?

Transfer takes `lock[b]` first, Audit takes `lock[a]` first, and each reads its own record

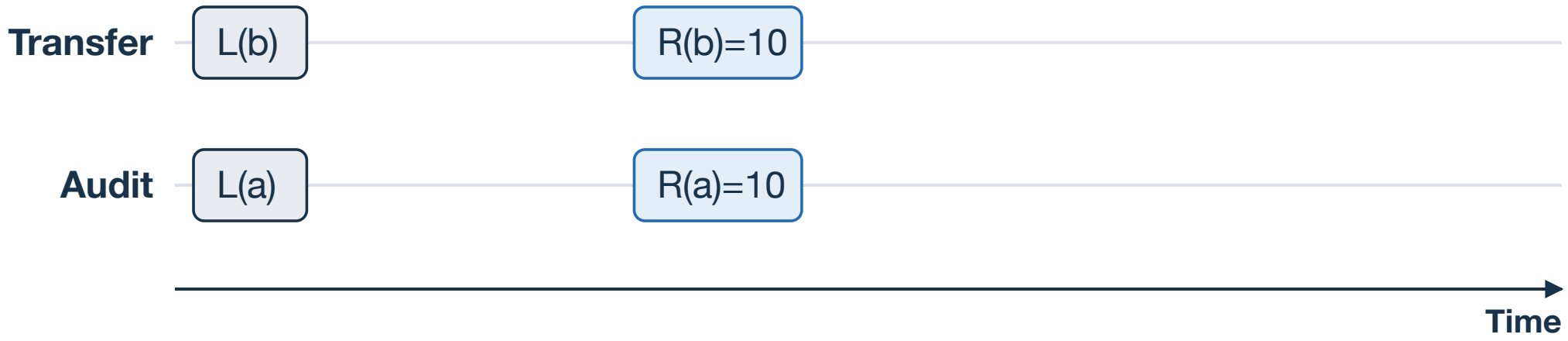
- Then Transfer needs `lock[a]` and Audit needs `lock[b]`
- Each of the two locks is held by the other side, so neither can get it: deadlock



Deadlock: what if Audit first acquires `lock[a]`?

Transfer takes `lock[b]` first, Audit takes `lock[a]` first, and each reads its own record

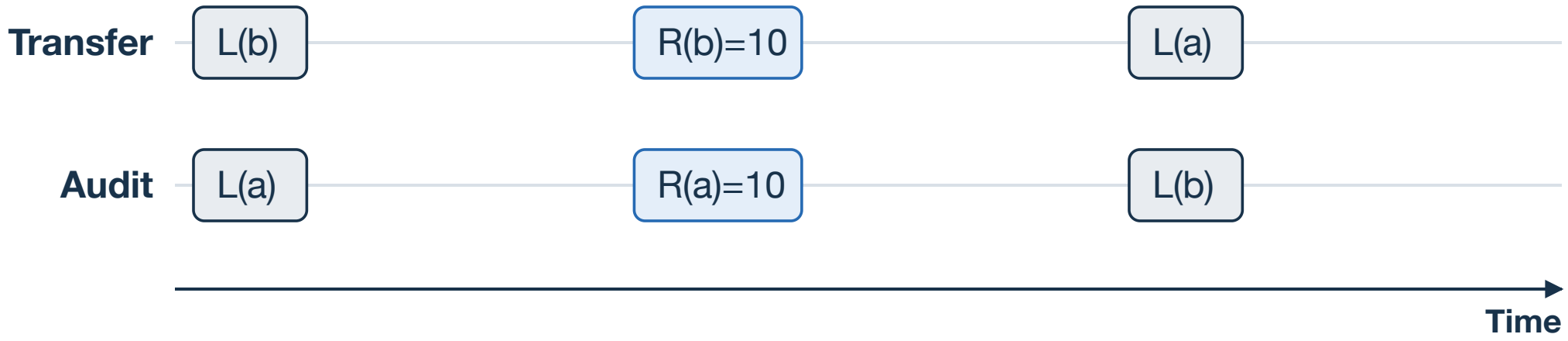
- Then Transfer needs `lock[a]` and Audit needs `lock[b]`
- Each of the two locks is held by the other side, so neither can get it: deadlock



Deadlock: what if Audit first acquires `lock[a]`?

Transfer takes `lock[b]` first, Audit takes `lock[a]` first, and each reads its own record

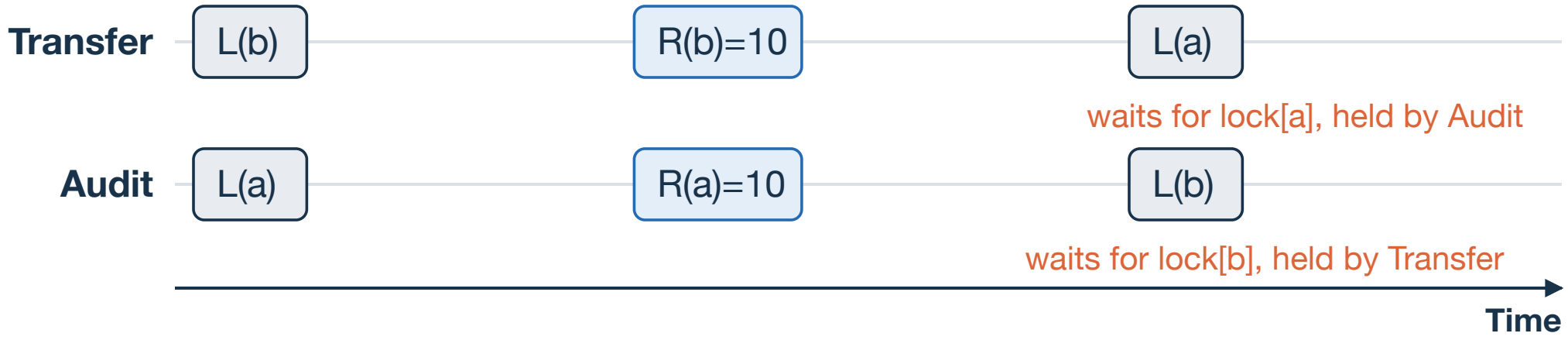
- Then Transfer needs `lock[a]` and Audit needs `lock[b]`
- Each of the two locks is held by the other side, so neither can get it: deadlock



Deadlock: what if Audit first acquires `lock[a]`?

Transfer takes `lock[b]` first, Audit takes `lock[a]` first, and each reads its own record

- Then Transfer needs `lock[a]` and Audit needs `lock[b]`
- Each of the two locks is held by the other side, so neither can get it: deadlock



Can Transfer or Audit finish the execution?

Resolving deadlock

Solution #1: acquire locks in a pre-defined order

- One consistent order, so the cycle never appears
- Cost: does not support general TXs, since a TX must know its read/write set before execution

Solution #2: detect deadlock by calculating the waiting graph

- If there is a cycle, then there must be a deadlock; abort one TX to break it
- Cost: the detection itself is expensive

Solution #3: use heuristics (e.g., timestamps) to pre-abort a waiting TX

- Cost: false positives, killing a TX that would not have deadlocked
- May also end up in livelocks, where everyone keeps retrying and nobody proceeds

Summary

What we covered today

- Why concurrency: one thread cannot sustain the given workload, and requests queue up
- What concurrency costs: race conditions — a group of reads/writes on shared data gets interleaved with someone else's
- The ideal system property under concurrency: before-or-after atomicity, an action happens either completely before or completely after another
- How to implement before or after: from a global lock to fine-grained locking, then static locking, then 2PL
- Using serializability (view / conflict) to check the correctness of locking

参考文献

[1] Queueing Systems, Volume 1: Theory — Kleinrock, L., 1975. Wiley (P8)

[2] Performance Modeling and Design of Computer Systems: Queueing Theory in Action — Harchol-Balter, M., 2013. Cambridge University Press (P8)